

Sistemi Intelligenti — Appunti di Teoria

Trascrizione completa in LaTeX nativo

Corso di Sistemi Intelligenti, Cristina Baroglio — Università di Torino

Moduli 1–8

Indice

1	Introduzione all'IA e Agenti Intelligenti	2
1.1	Cos'è l'intelligenza artificiale?	2
1.1.1	Un punto di partenza informale	2
1.1.2	IA nell'immaginario collettivo vs. IA reale	2
1.1.3	Definizione di "intelligenza" e problema dell'IA	2
1.1.4	Breve storia: le origini	3
1.1.5	Il Dartmouth Summer Research Project (1956)	3
1.1.6	Il Test di Turing	3
1.1.7	La Stanza Cinese di John Searle (1980)	4
1.1.8	Test di Turing inverso: CAPTCHA	5
1.1.9	Strong AI vs. Weak AI	5
1.1.10	Dall'esempio "attraversare la strada" nasce l'astrazione di agente	5
1.1.11	Le quattro scuole di definizione dell'IA	6
1.1.12	Quando serve davvero l'IA?	6
1.1.13	Discipline di fondamento dell'IA	6
1.1.14	Risoluzione automatica di problemi (anticipazione)	7
1.2	Agenti e ambienti	7
1.2.1	Agente e ambiente: il binomio fondamentale	7
1.2.2	Sequenza percettiva	7
1.2.3	Razionalità	8
1.2.4	Task environment e PEAS	9
1.2.5	Proprietà dell'ambiente (task environment)	10
1.2.6	Agente = architettura + programma	11
1.2.7	Tipologie di agente	11
1.2.8	Tabella riassuntiva delle tipologie di agente	14
1.3	Paradigmi di programmazione e agenti autonomi	14
1.3.1	Dall'approccio tradizionale al dichiarativo	14
1.3.2	Esempio guida: il mondo dei blocchi (Blocks World)	14
1.3.3	Dal problema giocattolo al problema reale	15
1.3.4	Da automazione ad autonomia	15
1.3.5	Gradi di autonomia: tre esempi concreti a confronto	16
1.3.6	Ragionare basta?	16
	Riepilogo e punti chiave — Modulo 1	17
2	Risoluzione di problemi mediante ricerca (non informata e informata)	18
2.1	Stati, azioni e astrazione	18
2.2	Definizione formale di problema di ricerca	19
2.3	Spazio degli stati vs albero/grafico di ricerca	20
2.4	Criteri di valutazione delle strategie di ricerca	20
2.5	Ricerca non informata (blind search)	20
2.5.1	Ricerca in ampiezza (Breadth-First Search, BFS)	21

2.5.2	Ricerca a costo uniforme (Uniform-Cost Search)	21
2.5.3	Ricerca in profondità (Depth-First Search, DFS) senza backtracking	21
2.5.4	Iterative Deepening Depth-First Search (IDDFS)	21
2.5.5	Ricerca bidirezionale	22
2.5.6	Grafi di ricerca e nodi già esplorati	22
2.5.7	Tabella riassuntiva — confronto strategie di ricerca non informata	22
2.6	Ricerca informata (heuristic search)	22
2.6.1	Funzioni di valutazione e best-first search	23
2.6.2	Ricerca greedy (avida)	23
2.6.3	A* (A-star)	23
2.6.4	Ridurre i requisiti di memoria: IDA* e RBFS	24
2.6.5	Funzioni euristiche: il caso di studio dell'8-puzzle	25
	Riepilogo e punti chiave — Modulo 2	26
3	Ricerca con avversario (giochi)	27
3.1	Introduzione: perché studiare i giochi in AI	27
3.2	Tassonomia dei giochi	27
3.3	Caratteristiche formali del gioco (a due giocatori)	28
3.4	Algoritmo Minimax	28
3.5	Teoria delle decisioni: maximax, maximin, minimax regret	30
3.6	Potatura alfa-beta (Alpha-Beta Pruning)	31
3.7	Ricerca con profondità limitata e funzioni di valutazione euristica	32
3.8	Giochi stocastici: cenni (expectiminimax)	33
3.9	Programmi storici che giocano	33
	Riepilogo e punti chiave — Modulo 3	34
4	Constraint Satisfaction Problems (CSP)	35
4.1	Definizione formale di CSP	35
4.2	Tipi di domini e di variabili	35
4.3	Arità dei vincoli	36
4.4	CSP come problema di ricerca nello spazio degli stati	36
4.5	Rappresentare gli stati: generate-and-test vs. ricerca con backtracking	36
4.6	Euristiche per la scelta della variabile	37
4.7	Euristica per la scelta del valore: Least Constraining Value (LCV)	38
4.8	Rendere informata la ricerca: propagazione dei vincoli (inferenza)	38
4.9	Vincoli speciali (globali)	40
4.10	Oltre il backtracking cronologico: backjumping	40
4.11	Assegnamenti NOGOOD e conflict-directed backjumping	40
4.12	Applicazioni ed esempi aggiuntivi	41
	Riepilogo e punti chiave — Modulo 4	41
5	Rappresentazione della conoscenza e Logica proposizionale	43
5.1	Perché rappresentare la conoscenza	43
5.2	Rappresentare la conoscenza tramite la logica	44
5.3	Logica proposizionale	45
5.4	Clausole di Horn, Forward Chaining, Backward Chaining	47
	Riepilogo e punti chiave — Modulo 5	48
6	Logica del primo ordine (FOL)	49
6.1	Perché non basta la logica proposizionale	49
6.2	Dalla logica proposizionale a FOL	49
6.3	Termini e formule atomiche	50
6.4	Quantificatori	50

6.5	Inferenza in FOL: due approcci	52
6.6	Modus Ponens Generalizzato (MPG)	53
6.7	Unificazione	53
6.8	Clausole di Horn del prim'ordine	53
6.9	Risoluzione in FOL: lifting e refutazione	54
6.10	Skolemizzazione	55
6.11	Binary resolution in FOL (lifting della risoluzione)	56
6.12	Costruire una KB in FOL: ingegneria della conoscenza	57
	Riepilogo e punti chiave — Modulo 6	57
7	Tassonomie/Ontologie e Pianificazione automatica	58
7.1	Tassonomie e Ontologie	58
7.1.1	Cos'è una tassonomia	58
7.1.2	Decomposizioni, disgiunzione, partizione	58
7.1.3	Relazioni strutturali: Part-of e Bunch-of	58
7.1.4	T-box e A-box	59
7.1.5	Problematiche delle tassonomie/ontologie	59
7.1.6	Dalla tassonomia all'ontologia	59
7.1.7	Semantic Web e linguaggi per rappresentare ontologie	59
7.1.8	Costruire un'ontologia: metodologia pratica	60
7.1.9	Relazioni fra ontologie	60
7.2	Pianificazione automatica	60
7.2.1	Dalle ontologie alla pianificazione: perché serve un nuovo formalismo	60
7.2.2	Differenza rispetto alla ricerca classica	61
7.2.3	Il Situation Calculus: fondamento logico della pianificazione	61
7.2.4	Il Frame Problem	61
7.2.5	Perseguire goal complessi: scomposizione in sottogoal	62
7.2.6	L'anomalia di Sussman: quando i sottogoal interagiscono	62
7.2.7	Limiti pratici del Situation Calculus	62
	Riepilogo e punti chiave — Modulo 7	63
8	Machine Learning: Classificazione, Alberi di Decisione, Reti Neurali	64
8.1	Il problema della classificazione	64
8.2	Alberi di decisione	66
8.3	Reti neurali	68
	Riepilogo e punti chiave — Modulo 8	71

Capitolo 1

Introduzione all'IA e Agenti Intelligenti

1.1 Cos'è l'intelligenza artificiale?

1.1.1 Un punto di partenza informale

Le slide aprono mostrando esempi reali (DeepSeek, ChatGPT, febbraio 2025) a cui viene posta la stessa domanda (“Chi era Napoleone VII di Baviera?”, personaggio storico **inesistente**). I due sistemi rispondono con sicurezza ma con contenuti diversi e **inventati** (allucinazioni): DeepSeek lo descrive come figlio di Napoleone III morto nel 1879, ChatGPT come figlio di Massimiliano I di Baviera morto nel 1837. Nessuna delle due risposte è vera, ma entrambe sono fluenti e plausibili.

Nel secondo esempio, allo stesso sistema viene chiesto di scrivere un articolo che metta “in buona luce” e poi “in cattiva luce” le lumache: il sistema produce entrambi i testi senza alcuna difficoltà, adattando tono e argomentazione alla richiesta.

Perché questi esempi aprono il corso? Servono a mostrare, prima ancora di dare definizioni, il problema centrale che il modulo affronterà: questi sistemi producono output linguisticamente ineccepibili e convincenti, ma ciò **non implica che “comprendano”** quello che dicono, né che possiedano conoscenza verificata. È un’anticipazione informale del dibattito Turing/Searle che viene formalizzato più avanti.

1.1.2 IA nell’immaginario collettivo vs. IA reale

- **Immaginario comune:** robot antropomorfo, capace di risolvere problemi complessi, capace di imparare (fantascienza).
- **IA realmente diffusa** intorno a noi (spesso in modo “invisibile”, cioè in **contatto quotidiano ma inconsapevole**): servizi di streaming (raccomandazioni), fotocamere/smartphone (riconoscimento volti, cibo, scene), social network (annunci e suggerimenti personalizzati), assistenti virtuali (chat, voce, mappe), supporto alla decisione (es. concessione di mutui), logistica e consegne a domicilio (calcolo percorsi).

Il punto pedagogico: l’IA che usiamo ogni giorno è quasi sempre **specializzata e invisibile**, non un robot antropomorfo generalista.

1.1.3 Definizione di “intelligenza” e problema dell’IA

Il vocabolario definisce l’intelligenza (umana) come il complesso di facoltà psichiche che permettono di pensare, comprendere, spiegare fatti/azioni, elaborare modelli astratti della realtà, farsi intendere dagli altri, giudicare, adattarsi a situazioni nuove e modificarle. È una facoltà propria dell’uomo (con gradazioni riconosciute anche agli animali), che si sviluppa nell’infanzia insieme alla consapevolezza.

Intelligenza artificiale = una forma di intelligenza **non naturale**, ottenuta con procedimenti tecnici (cioè costruita dall'uomo tramite l'informatica, non frutto di sviluppo biologico).

1.1.4 Breve storia: le origini

Anno	Evento
1936	Alan Turing formalizza la Macchina di Turing , pietra miliare dell'informatica teorica
1940	ENIAC, tra i primi calcolatori
1950	Turing propone il Test di Turing : quando un computer può dirsi intelligente?
1951	UNIVAC
1956	Nasce ufficialmente l'IA come disciplina: Dartmouth Summer Research Project

Anni '40 — automazione del calcolo: il computer esegue una sequenza di istruzioni predefinita, senza operatore umano che intervenga passo-passo. Questo pone subito la domanda: *l'automazione (eseguire automaticamente un programma) equivale a intelligenza?*

Le slide insistono molto su questa domanda mostrando esempi progressivi (una calcolatrice esegue un programma ed è automatica — ma non diremmo che è intelligente) per portare lo studente a capire che **automazione** $\neq$ **intelligenza**: eseguire automaticamente istruzioni non basta. Serve qualcos'altro — capacità di adattarsi, di scegliere di fronte a situazioni non previste esplicitamente dal programmatore.

Domanda d'esame. Automazione è intelligenza? — No. Un programma che esegue automaticamente una sequenza di istruzioni predefinite (es. una calcolatrice) è automatico ma non è considerato intelligente: non si adatta, non affronta situazioni nuove, non delibera. L'intelligenza (artificiale) richiede in più la capacità di **adattarsi**, di scegliere l'azione più opportuna in funzione di percezioni e obiettivi, eventualmente anche di **imparare** dall'esperienza. Questo è il punto di svolta che porta dal concetto di “programma automatico” a quello di “agente”.

1.1.5 Il Dartmouth Summer Research Project (1956)

- Il termine “Artificial Intelligence” nasce nel 1956, proposto da **John McCarthy**.
- Un gruppo di circa venti ricercatori di discipline diverse (Solomonoff, Minsky, McCarthy, Shannon, Rochester, Selfridge, Newell, Simon, ecc.) si riunisce per circa due mesi per fare brainstorming sulle “thinking machines”. Notare: né Turing (morto nel 1954) né von Neumann (gravemente malato) parteciparono.
- **“Carta di identità” dell'IA:** nascita a Dartmouth Conference, USA, 1956, nome scelto da McCarthy; prima del 1956 (primi anni '50) si discuteva già se una macchina potesse “pensare” (cibernetica, teoria degli automi, elaborazione dell'informazione complessa, Test di Turing); dopo il 1956 (primi anni '60) primi tentativi applicativi (scacchi, giochi, dimostrazione automatica di teoremi).
- Contesto tecnologico dell'epoca: niente tastiere, dischi magnetici o monitor come li intendiamo oggi; solo al MIT si sperimentava l'input diretto da tastiera (anziché schede perforate), mentre IBM sviluppava i precursori dei dischi magnetici.

1.1.6 Il Test di Turing

Proposto da Alan Turing nel 1950 nell'articolo “Computing Machinery and Intelligence” come **The Imitation Game**: sostituisce la domanda mal posta “può una macchina pensare?” con una domanda operativa.

Come funziona: un intervistatore umano comunica per iscritto con due interlocutori nascosti (uno umano, uno macchina) senza sapere quale sia quale. Può fare qualsiasi domanda. Se, al termine, l'intervistatore non riesce a distinguere in modo affidabile la macchina dall'umano (o giudica erroneamente che la macchina sia umana), la macchina **supera il test**.

Esempio dalle slide:

Q: Please write me a sonnet on the subject of the Forth Bridge.

A: Count me out on this one. I never could write poetry.

Q: Add 34957 to 70764.

A: (pausa di 30 secondi) 105621.

Q: Do you play chess?

A: Yes.

Da notare: la macchina finge persino di essere lenta a fare un calcolo (per sembrare più umana) — il test valuta la capacità di **imitare risposte plausibili**, di comportarsi *come se* fosse un essere umano, non un meccanismo interno particolare.

Domanda d'esame. Il test di Turing cattura davvero l'intelligenza? — Questo è uno dei nodi filosofici centrali del modulo. Il test si basa solo sull'osservazione dei comportamenti esterni (input/output): se gli output sono indistinguibili da quelli umani, la macchina “passa”. Ma **produrre gli output attesi è sufficiente per dire che c'è comprensione?** Le slide rispondono con l'esempio del quiz sulla geografia: due persone (Valeria e Rossella) danno entrambe la risposta corretta “Piemonte” alla domanda “dove si trova Torino?”, ma Valeria la sa perché ha studiato geografia e ha usato conoscenza + ragionamento per collegare la domanda alla risposta, mentre Rossella ha semplicemente tirato a caso ed è stata fortunata. Dall'esterno (stesso input, stesso output) i due comportamenti sono indistinguibili, ma solo uno dei due implica reale comprensione. Questo mostra che **stesso output non implica stessa comprensione**: il Test di Turing, valutando solo il comportamento osservabile, non può distinguere questi due casi, e quindi è un test necessario ma probabilmente non sufficiente per l'intelligenza “vera” (nel senso forte).

1.1.7 La Stanza Cinese di John Searle (1980)

Argomento filosofico contro l'idea che superare il Test di Turing implichi comprensione reale (Searle, *Minds, brains, and programs*, 1980).

Esperimento mentale: una persona che non conosce il cinese è chiusa in una stanza. Riceve dall'esterno frasi scritte in ideogrammi cinesi (input). All'interno della stanza ha un manuale di istruzioni (equivalente a un programma) che le dice, per ogni sequenza di simboli in ingresso, quali simboli produrre in uscita, puramente in base alla loro forma sintattica, senza capirne il significato. Seguendo meccanicamente le istruzioni, la persona restituisce risposte in cinese perfettamente sensate, tanto che chi sta fuori dalla stanza (un madrelingua cinese) crede di conversare con qualcuno che capisce il cinese.

Domanda: quella persona (o il sistema stanza+persona+manuale) **parla davvero cinese? Lo capisce?**

Secondo Searle, no: la persona sta solo manipolando simboli sintatticamente, senza avere accesso alla semantica, all'intenzionalità, alla comprensione. Questo è esattamente ciò che fa un computer che esegue un programma: manipola simboli seguendo regole sintattiche, senza “capire” nulla nel senso in cui lo capisce un umano. Da qui la tesi di Searle: “*Instantiating a computer program is never by itself a sufficient condition of intentionality*” — eseguire un programma non è mai di per sé condizione sufficiente per avere intenzionalità (cioè stati mentali “rivolti verso” un significato).

Questo argomento si collega direttamente all’apertura del modulo (DeepSeek/ChatGPT): questi sistemi possono essere impressionanti nelle risposte, ma la domanda “capiscono quello che dicono, oppure no?” resta aperta e filosoficamente delicata.

1.1.8 Test di Turing inverso: CAPTCHA

CAPTCHA = “Completely Automated Public Turing-test to tell Computers and Humans Apart”. A differenza del test di Turing classico (dove un umano cerca di riconoscere la macchina), qui è **un computer** che pone un test per distinguere se dall’altra parte c’è un umano o un bot, tipicamente per impedire l’accesso automatizzato a form o dati. Sviluppato nel 1997 nel contesto del motore di ricerca AltaVista.

1.1.9 Strong AI vs. Weak AI

Due modi diversi di intendere l’obiettivo della disciplina:

	Strong AI (IA forte)	Weak AI (IA debole)
Obiettivo	Riprodurre realmente l’intelligenza umana	Trovare metodi per risolvere problemi che, se risolti da un umano, richiederebbero intelligenza
Approccio	Studio del pensiero e comportamento umano (scienze cognitive)	Task-oriented: studio del pensiero e comportamento <i>razionale</i> (non necessariamente umano)
Criterio di successo	La macchina deve effettivamente pensare/capire come un umano	Basta che il sistema <i>funzioni</i> producendo la soluzione corretta, indipendentemente dal meccanismo interno

Questa distinzione anticipa le “quattro scuole” di definizione dell’IA e chiarisce perché il corso, seguendo Russell & Norvig, adotterà come riferimento pratico l’approccio del **comportamento razionale** (weak AI, task-oriented), più operativo e ingegnerizzabile.

1.1.10 Dall’esempio “attraversare la strada” nasce l’astrazione di agente

Le slide usano un esempio guida ricorrente: *quanto è difficile attraversare una strada?*

Un **uomo** che attraversa la strada deve: identificare il passaggio pedonale, rilevare possibili ostacoli, rilevare oggetti in movimento, rilevare segnali significativi (es. semaforo), costruire un piano d’azione. L’ambiente in cui opera è **complesso, parzialmente prevedibile, parzialmente collaborativo** (altri conducenti/pedoni possono cooperare o no).

La domanda diventa: **come si programma un agente artificiale** capace di fare le stesse cose, nello stesso tipo di ambiente? Da qui emergono le prime astrazioni fondamentali del corso: il binomio **<agente, ambiente>**.

Definizione di agente: un agente è un’*astrazione che rappresenta un qualsiasi sistema che percepisce il proprio ambiente tramite dei sensori e agisce su di esso tramite degli attuatori*.

Punti chiave impliciti in questa definizione:

- Non esistono agenti che non siano **situati in un ambiente**: agente e ambiente sono un **binomio inscindibile**, non ha senso parlare di un agente “nel vuoto”.
- Tra la percezione (input dai sensori) e l’azione (output verso gli attuatori) c’è una **funzione deliberativa** (rappresentata spesso nei diagrammi con un punto interrogativo “?” dentro l’agente) che decide quale azione eseguire sulla base di ciò che è stato percepito. È proprio il “contenuto” di questo punto interrogativo — come viene implementata la deliberazione — a differenziare i vari tipi di agente che vedremo più avanti (agenti reattivi, basati su modello, su obiettivi, sull’utilità, che apprendono).

1.1.11 Le quattro scuole di definizione dell'IA

Russell & Norvig classificano le definizioni storiche di IA lungo due assi: *pensiero vs. comportamento* e *fedeltà umana vs. razionalità*.

	Riproduce il pensiero	Riproduce il comportamento
come l'uomo	Sistemi che pensano come esseri umani (Haugeland 1985; Bellman 1978)	Sistemi che agiscono come esseri umani (Kurzweil 1990; Rich & Knight 1991)
in modo razionale	Sistemi che pensano razionalmente (Charniak & McDermott 1985; Winston 1992)	Sistemi che agiscono razionalmente (Poole et al. 1998; Nilsson 1998)

Confronto sintetico con l'esempio dell'attraversamento pedonale:

	Pensiero	Comportamento
Umano (approccio forte)	Ragionamento logico esplicito sul fatto che c'è un passaggio pedonale, sgombro, senza auto in arrivo (Modellazione Cognitiva, es. <i>General Problem Solver</i> di Newell & Simon)	Guardo a sinistra e a destra prima di attraversare (valutato col Test di Turing)
Razionale (approccio debole)	Una rete neurale che decide se ci sono le condizioni per attraversare	Un robot con sonar schiva passanti e auto in modo diverso da un umano, ma <i>funzionalmente efficace</i>

Il corso adotta prevalentemente l'ottica “**agire razionalmente**” (in linea con Poole/Nilsson e con Russell & Norvig), perché è la più operativa per progettare e valutare agenti artificiali: non serve che l'agente “pensi come un umano”, basta che scelga sempre l'azione che massimizza il risultato atteso.

1.1.12 Quando serve davvero l'IA?

- **Non è adatta** dove esistono già modelli matematici precisi o metodi algoritmici specifici (in quei casi conviene usare un algoritmo tradizionale).
- **È utile o necessaria** quando si hanno: problemi non deterministici, molteplicità di soluzioni possibili, preferenze tra soluzioni diverse, dati di natura simbolica, conoscenza ampia e incompleta, informazione parzialmente strutturata, necessità di interazione con l'ambiente e con esseri umani.

1.1.13 Discipline di fondamento dell'IA

L'IA è un campo intrinsecamente interdisciplinare: **Filosofia, Matematica, Economia, Neuroscienze, Psicologia, Informatica, Teoria del controllo e cibernetica, Linguistica**.

Due esempi approfonditi dalle slide:

- **Filosofia** — **Aristotele** (*Etica Nicomachea*, Libro III): Aristotele osserva che non deliberiamo sui fini (un medico non delibera *se* debba guarire) ma sui **mezzi** per raggiungerli, una volta posto il fine. Se il fine è raggiungibile con più mezzi, si valuta quale sia il più facile/migliore; si procede a ritroso dai mezzi fino a una “causa prima” (ciò che è ultimo nell'analisi è primo nella costruzione). Questo è considerato il **primo esempio storico di algoritmo di deliberazione backward** (dal goal ai mezzi), concettualmente identico

alla ricerca all'indietro usata nella risoluzione di problemi in IA — implementato circa 2300 anni dopo nel **General Problem Solver (GPS)** di Newell & Simon.

- **Psicologia — Ivan Pavlov:** nasce nel XIX secolo lo studio scientifico del comportamento animale/umano. Pavlov studia l'apprendimento per **riflesso condizionato**: la salivazione del cane è un riflesso naturale (incondizionato) davanti al cibo (stimolo incondizionato); associando ripetutamente un campanello (stimolo neutro) alla presentazione del cibo, il campanello da solo diventa uno **stimolo condizionato** che induce salivazione. Questo mostra come un comportamento possa essere modificato dall'esperienza — idea alla base dell'apprendimento automatico.

1.1.14 Risoluzione automatica di problemi (anticipazione)

Le slide chiudono anticipando i temi successivi del corso: definire cosa sia un problema e una soluzione (distinguendo soluzione da soluzione *ottima*), con tre categorie principali di approcci: (1) ricerca nello spazio degli stati, (2) ricerca in spazi con avversario (giochi a informazione completa), (3) risoluzione di problemi mediante soddisfacimento di vincoli (CSP).

1.2 Agenti e ambienti

1.2.1 Agente e ambiente: il binomio fondamentale

Ribadendo la definizione già introdotta: un **agente** è un'astrazione che rappresenta un qualsiasi sistema che **percepisce** il proprio ambiente tramite **sensori** e **agisce** su di esso tramite **attuatori**. Non esistono agenti scollegati da un ambiente: **agente e ambiente formano un binomio inscindibile**. Il simbolo “?” nel diagramma dell'agente rappresenta la **funzione deliberativa**, cioè il meccanismo interno (non ancora specificato) che, sulla base di ciò che è stato percepito, determina quale azione eseguire.

Esempi di sensori fisici reali (per dare concretezza al concetto): sensori di luce/ottici, infrarossi, suono (microfoni), accelerazione (accelerometri), temperatura, calore, radiazione (contatori Geiger), resistenza/corrente/tensione elettrica, magnetismo, pressione, gas e flusso di liquidi, movimento, orientamento, forza (celle di carico, estensimetri), prossimità, distanza, biometrici, chimici.

1.2.2 Sequenza percettiva

Definizione: la **sequenza percettiva** è la storia completa di tutte le percezioni che un agente ha ricevuto fino a un certo istante.

Un agente sceglie la prossima azione in base alla **situazione in cui si trova**, cioè in base a tutto ciò che ha percepito fino a quel momento (non solo alla percezione istantanea). Il modo più generale (ma anche più oneroso) di realizzare il modulo deliberativo è una **tabella percepito** → **azione** che associa ad ogni possibile sequenza percettiva un'azione. In alcuni casi è possibile prescindere dalla storia intera e basarsi sulla sola percezione corrente; in altri casi serve davvero l'intera sequenza (o una sintesi di essa, come uno “stato interno”).

Esempio: il mondo dell'aspirapolvere

Un piccolo mondo giocattolo con due sole posizioni, A e B, ciascuna delle quali può essere pulita o sporca. Una possibile tabella percepito-azione:

Sequenza percettiva	Azione
[A, pulito]	destra
[A, sporco]	aspira
[B, pulito]	sinistra
[B, sporco]	aspira
[A, pulito],[A, pulito]	destra
[A, pulito],[A, sporco]	aspira
...	...

Le slide mostrano poi una **seconda tabella diversa** per lo stesso ambiente (dove, ad esempio, [A, pulito] produce “aspira” invece di “destra”): rappresenta un **agente diverso**, con un comportamento differente pur nello stesso ambiente. Questo porta alla domanda cruciale: **due aspirapolvere che si comportano diversamente sono confrontabili? Sulla base di cosa?** Serve un criterio oggettivo per stabilire quale dei due comportamenti sia “migliore” — da qui nasce la nozione di misura di prestazione.

1.2.3 Razionalità

“Fare la cosa giusta”

In termini informali, un agente **razionale** è un agente che “fa la cosa giusta”, cioè opera per conseguire il “successo”. Per rendere questa idea precisa serve una **misura di prestazione (performance measure)**: un criterio che valuta la bontà degli **stati** attraversati dall’ambiente, definito in funzione dell’effetto che si desidera ottenere (non della singola azione in sé, ma delle sue conseguenze sullo stato del mondo).

Esempio nel mondo dell’aspirapolvere — una possibile misura di bontà degli stati:

Stato	Bontà
[A pulito], [B pulito]	1
[A pulito], [B sporco]	0.5
[A sporco], [B pulito]	0.5
[A sporco], [B sporco]	0

Definizione formale di comportamento razionale

Il comportamento razionale di un agente dipende da **quattro fattori**: (1) le **azioni** nelle facoltà dell’agente (il repertorio di azioni disponibili), (2) la **misura di prestazione** che definisce il criterio di successo, (3) la **conoscenza dell’ambiente** posseduta dall’agente, (4) la **sequenza percettiva** fino a quel momento.

Definizione: un agente razionale dovrebbe scegliere sempre, per ogni possibile sequenza percettiva, l’azione che **massimizza la misura di prestazione attesa**, date la sequenza percettiva stessa e le informazioni derivabili dalla conoscenza dell’ambiente in possesso dell’agente.

Nota il termine “attesa”: la razionalità si valuta rispetto a ciò che l’agente può ragionevolmente prevedere dato ciò che sa, **non** rispetto all’esito effettivo (che dipende anche da fattori che l’agente non poteva conoscere).

Razionalità vs. onniscienza/chiaroveggenza

	Razionalità	Onniscienza / chiaroveggenza
Ottimizza	il risultato atteso	il risultato reale
Fattori ignoti	possono impedire il risultato ideale	per definizione non esistono

Esempio (torta di compleanno): voglio che al compleanno di mio figlio ci sia la torta più bella e meno costosa. Conosco le offerte dei pasticceri della zona, i gusti di mio figlio, ho abbastanza denaro e so fare acquisti: sulla base di tutto ciò, scelgo la torta migliore che conosco. Due cose però potevano sfuggirmi: (a) la nonna, a mia insaputa, porta in regalo una torta identica (informazione che non potevo conoscere), e (b) esiste una nuova pasticceria con un'offerta migliore di cui non ero a conoscenza. In entrambi i casi il mio comportamento **resta razionale**, anche se il risultato non è quello ottimale in assoluto: la mia sequenza percettiva non includeva quelle informazioni, quindi non potevo tenerne conto. Un agente **onnisciente/chiaroveggente**, che sapesse tutto in anticipo, avrebbe invece ottenuto il risultato migliore in assoluto.

Punto fondamentale: **razionale non significa “di successo” né “onnisciente”**. Significa fare la scelta migliore possibile *dato ciò che si sa*.

Razionalità e apprendimento: se un anno dopo, sapendo ormai che la nonna tende a fare la torta perfetta per il compleanno, io comprassi comunque la torta senza prima verificare, il mio comportamento **non sarebbe più razionale**, perché ora quella conoscenza fa parte della mia sequenza percettiva/esperienza pregressa. Questo mostra che **un agente razionale è tanto più efficace quanto più ha capacità di imparare dall'esperienza** e di modificare il proprio comportamento futuro di conseguenza.

La percezione non è un atto passivo

Esempio (attraversare la strada): è razionale che un robot attraversi la strada senza prima guardare a destra? **No**. Prima di eseguire un'azione, un agente deve valutare se ha raccolto informazioni sufficienti oppure se è necessario compiere ulteriori **atti percettivi** (es. guardare) prima di agire. Questo mostra che la **razionalità dipende dalla sequenza percettiva, ma può anche influenzarla**: un agente razionale può scegliere di percepire di più prima di decidere.

Domanda d'esame. Quale aspirapolvere è più razionale? Sono definibili comportamenti ancora più razionali? — Non si può rispondere senza fissare prima una misura di prestazione esplicita (es. quella della tabella con bontà degli stati) e senza conoscere l'ambiente e le azioni disponibili. Rispetto a una misura che premia il numero di celle pulite nel tempo, un agente che aspira solo quando percepisce “sporco” ed evita mosse inutili è più razionale di uno che, ad esempio, spreca energia muovendosi anche quando la cella corrente è già pulita senza una strategia. Esistono comportamenti “ancora più razionali”, ad esempio agenti che tengono conto anche del consumo energetico, del tempo impiegato, o che imparano il layout dell'ambiente per pianificare percorsi più efficienti: dipende sempre da **quali effetti desiderati** vengono inclusi nella misura di prestazione.

1.2.4 Task environment e PEAS

Task environment: il contesto in cui l'agente è inserito. Può essere **fisico** (reale o simulato, tipico per agenti robotici) oppure, ad esempio, **sociale**, comprendendo le relazioni con altri agenti (tipico per agenti software).

PEAS è l'acronimo che identifica i quattro elementi che definiscono formalmente un task environment, e va sempre specificato quando si progetta un agente:

- **Performance measure** — la misura di prestazione rispetto a cui valutare il successo dell'agente,

- **Environment** — l’ambiente in cui l’agente opera,
- **Actuators** — gli attuatori a disposizione dell’agente per agire,
- **Sensors** — i sensori a disposizione dell’agente per percepire.

Specificare il PEAS è il primo passo, sistematico e imprescindibile, nella progettazione di qualunque agente: obbliga a chiarire esplicitamente cosa l’agente deve ottenere, dove opera, come può agire e cosa può percepire.

1.2.5 Proprietà dell’ambiente (task environment)

Ogni ambiente può essere caratterizzato lungo sette dimensioni indipendenti. Questa classificazione è fondamentale perché **determina quali tecniche di progettazione dell’agente sono necessarie o sufficienti** (ambienti più “difficili” richiedono agenti più sofisticati).

Proprietà	Valore 1	Valore 2
Osservabilità	Completamente osservabile: in ogni istante i sensori danno accesso a tutti gli aspetti dell’ambiente rilevanti per scegliere l’azione	Parzialmente osservabile: i sensori danno accesso solo a parte dell’informazione rilevante
Determinismo	Deterministico: lo stato successivo è determinato univocamente dallo stato corrente e dall’azione eseguita	Stocastico: applicando più volte la stessa azione nello stesso stato si possono raggiungere stati diversi (caso particolare: strategico se lo stocasticismo dipende solo dalle azioni di altri agenti)
Episodicità	Episodico: l’esperienza è divisa in episodi atomici indipendenti, ciascuno = una percezione seguita da una singola azione	Sequenziale: l’attività è composta da più passi collegati, ognuno dei quali influenza in generale i successivi
Dinamicità	Statico: l’ambiente non cambia mentre l’agente “pensa”	Dinamico: l’ambiente può cambiare mentre l’agente sta ancora decidendo
Discretezza	Discreto: stato, tempo, percezioni e azioni sono discreti	Continuo: stato, tempo, percezioni o azioni sono continui
N. di agenti	Singolo agente	Multiagente

Note importanti:

- **Connessione tra parzialmente osservabile e stocastico:** spesso un ambiente appare stocastico proprio perché è parzialmente osservabile — non percependo tutti gli aspetti rilevanti, l’agente non riesce a prevedere con certezza l’esito delle proprie azioni, anche se “in realtà” il mondo sottostante è deterministico. Esempio: una batteria la cui carica reale si consuma in modo continuo, ma l’agente percepisce solo tre stati discreti (bassa/media/alta). La stessa azione “passo”, eseguita quando la carica percepita è “media”, può risultare in “carica bassa” oppure “rimane media”, perché l’agente non vede il livello esatto e continuo di carica residua.
- **Come decidere cosa modellare come agente:** il progettista deve decidere quali entità del mondo trattare come agenti e quali come parte dell’ambiente. Criterio: vanno modellate come agenti le entità il cui comportamento tenta di **massimizzare una propria misura**

di prestazione che dipende anche dal comportamento di altri agenti (interazione strategica). Se un'entità si comporta in modo puramente meccanico/prevedibile senza obiettivi propri, può essere trattata come parte dell'ambiente.

- Il caso più complesso e generale (e più realistico per problemi reali) è un ambiente parzialmente osservabile, stocastico, sequenziale, dinamico, continuo e multiagente.

Esempi di caratterizzazione di ambienti

Ambiente	Osserv.	Agenti	Determ.	Episod.	Dinam.
Parole crociate	totale	singolo	deterministico	sequenziale	statico
Guidare un taxi	parziale	multiagente	stocastico	sequenziale	dinamico
Tutor inglese interattivo	parziale	multiagente	stocastico	sequenziale	dinamico
Analizzatore di immagini	totale	singolo	deterministico	episodico	statico

Questi esempi mostrano bene lo spettro di difficoltà: le parole crociate sono l'ambiente "più semplice" possibile, guidare un taxi è vicino al caso più complesso possibile — coerentemente col fatto che guidare in un ambiente reale è un problema molto più arduo da automatizzare rispetto a risolvere cruciverba.

Esempio guida: il termostato

Percepisce la temperatura della stanza; delibera quale azione eseguire; agisce accendendo/spegnendo il riscaldamento, oppure non facendo nulla (comportamento predefinito). È l'esempio più semplice possibile di ciclo percezione→delibera→azione.

Esempio guida: robot aspirapolvere (Roomba e simili)

Percepisce ostacoli (oggetti, scale) e livello della batteria; delibera quale azione eseguire; agisce muovendosi, fermandosi, girando, oppure andando alla stazione di ricarica (comportamento predefinito quando la batteria è scarica).

Da notare: sia il termostato sia il Roomba hanno un "comportamento predefinito" per i casi non altrimenti gestiti — un modo semplice per garantire che l'agente abbia sempre un'azione definita anche in assenza di condizioni specifiche riconosciute.

1.2.6 Agente = architettura + programma

- **Architettura:** la specifica degli elementi strutturali e funzionali su cui il programma agente viene eseguito (l'hardware/piattaforma, es. un robot fisico o una macchina virtuale).
- **Programma:** la funzione che mette in relazione le percezioni con le azioni.

Distinzione importante tra due nozioni spesso confuse:

- **Funzione agente:** un'astrazione matematica che ha come input **l'intera sequenza percettiva** (la storia completa delle percezioni) e restituisce un'azione. È una descrizione ideale, esaustiva, di come l'agente *dovrebbe* comportarsi per ogni possibile storia di percezioni.
- **Programma agente:** l'implementazione concreta, che tipicamente ha come input solo la **percezione corrente** (eventualmente insieme a uno stato interno che riassume la storia passata, per non dover ricordare l'intera sequenza percettiva esplicitamente).

1.2.7 Tipologie di agente

Le slide presentano cinque tipologie di agente, in ordine di sofisticazione crescente. Questa è una delle tassonomie più importanti del modulo (corrisponde alla struttura del cap. 2 di Russell & Norvig).

Agenti reattivi semplici

Basano la scelta dell'azione **solo sulla percezione corrente**, ignorando la storia passata. Il programma codifica direttamente delle **regole condizione-azione** (“se percepisco X, faccio Y”). È un agente scritto ad hoc per un problema specifico.

Esempio (aspirapolvere reattivo):

```
Aspirapolvere-reattivo(posizione, stato) return azione
{
  if (stato = sporco) then return aspira
  else if (posizione = A) then return destra
  else if (posizione = B) then return sinistra
}
```

Schema generale:

```
Agente-reattivo-semplice(percezione) return azione
{
  stato ← interpreta(percezione)
  regola ← individua-regola(stato, regole)
  azione ← regola-azione(regola)
  return azione
}
```

La percezione viene interpretata per identificare uno stato; lo stato individua la regola applicabile; la regola determina l'azione da eseguire. L'insieme delle regole definisce interamente il comportamento dell'agente.

Limiti degli agenti reattivi semplici: funzionano correttamente solo in ambienti **completamente osservabili**. Esempio: se l'aspirapolvere non ha un sensore di posizione, e la cella corrente è pulita, non sa in che direzione muoversi per essere efficace. Questo può generare **loop infiniti** perché l'agente ripete sempre la stessa scelta in presenza della stessa percezione. Per rompere questi loop occorre introdurre comportamenti **casuali (random)**.

Agenti reattivi basati su modello

In caso di **osservabilità parziale**, un agente puramente reattivo non basta: serve un **modello del mondo**, cioè una rappresentazione interna di **come il mondo evolve** e di **quali effetti hanno le azioni**. L'agente mantiene uno **stato interno** che viene aggiornato combinando: (1) la percezione corrente, (2) lo stato interno precedente, (3) l'ultima azione eseguita, (4) il modello del mondo.

Schema:

```
Agente-reattivo-con-modello(percezione) return azione
{
  stato ← aggiorna-stato(stato, azione, percezione)
  regola ← individua-regola(stato, regole)
  azione ← regola-azione(regola)
}
```

Esempio (batteria): il modello dice che “tre passi consumano una tacca di batteria”. Se la sequenza percettiva ha mostrato “2 tacche” per tre volte consecutive dopo tre azioni “passo”, il modello permette di **prevedere** che dopo il prossimo “passo” la batteria scenderà a 1 tacca, anche se il sensore, in un dato istante, non fornisce l'informazione precisa e continua sul livello di carica. Avere un modello permette quindi di **prevedere gli effetti delle azioni** e scegliere l'azione anche sulla base di questa previsione.

Agenti basati su obiettivi (goal-driven)

L'agente sceglie l'azione da eseguire in base ai propri **obiettivi**: l'azione scelta deve avvicinarlo all'obiettivo o farglielo raggiungere. La decisione può riguardare solo il prossimo singolo passo, oppure può richiedere di guardare più avanti nel tempo, costruendo un **piano** di più passi.

Meccanismo tipico: il **ragionamento ipotetico** — l'agente “simula” mentalmente l'effetto delle azioni possibili per valutare se e quanto lo avvicinano all'obiettivo, prima di eseguirle realmente.

Vantaggio chiave: separando esplicitamente l'obiettivo dal meccanismo di ragionamento, basta **cambiare l'obiettivo** per ottenere comportamenti completamente diversi dallo stesso agente, senza riscrivere le regole di comportamento (a differenza dell'agente reattivo, dove il comportamento è “cablato” nelle regole condizione-azione).

Agenti basati sull'utilità (utility-driven)

Alla nozione di obiettivo (raggiunto/non raggiunto, binario) si aggiunge una **misura di prestazione più fine**, calcolata da una **funzione di utilità**, che valuta quanto “buona” sia una scelta tenendo conto di più fattori contemporaneamente: costo, velocità, difficoltà attuativa, tempo, risorse utilizzate, ecc.

Esempio: per raggiungere una località possono esserci più percorsi alternativi (strade sterrate, percorso cittadino, strada veloce a pagamento). Il problema non è solo *trovare una soluzione* ma trovarne una che **massimizzi la soddisfazione** secondo criteri come rapidità, costo, comodità.

Distinzione obiettivo/utilità:

- **Obiettivo (goal)**: uno stato particolare che si vuole raggiungere, tipicamente descritto in termini di proprietà da soddisfare.
- **Utilità**: una funzione che, dato uno stato (o una sequenza di stati), restituisce una misura numerica di bontà.
- L'agente ha quindi un **duplice problema**: (1) raggiungere l'obiettivo, e (2) farlo massimizzando l'utilità. Da qui la distinzione tra **soluzioni** (qualunque sequenza di azioni che raggiunge il goal) e **buone soluzioni** (quelle che massimizzano l'utilità).

Agenti che apprendono

Aggiungono all'agente “classico” un ulteriore componente dedicato all'**apprendimento**, composto da tre elementi:

1. **Critico**: valuta il livello di prestazione dell'agente e decide se e quando attivare l'apprendimento (confrontando il comportamento con uno standard esterno di prestazione).
2. **Modulo di apprendimento**: modifica effettivamente la conoscenza/il comportamento dell'agente sulla base del feedback del critico.
3. **Generatore di problemi**: suggerisce/causa l'esecuzione di **azioni esplorative**, il cui scopo è esporre l'agente a nuove esperienze (anche non immediatamente ottimali) da cui poter imparare qualcosa di utile per il futuro.

Le slide precisano che non si fanno assunzioni su **quali segnali o tecniche specifiche** l'agente usi per apprendere: il concetto di “agente che apprende” è uno schema architetturale generale, indipendente dall'algoritmo di apprendimento concreto.

1.2.8 Tabella riassuntiva delle tipologie di agente

Tipo	Punti di forza	Limiti
Reattivo semplice	Semplice, veloce, poco costoso	Solo se completamente osservabile; loop infiniti; nessuna memoria
Reattivo con modello	Gestisce l'osservabilità parziale	Il modello va costruito/mantenuto; nessun goal esplicito
Basato su obiettivi	Basta cambiare il goal per cambiare comportamento	Non distingue soluzioni diverse che raggiungono il goal
Basato sull'utilità	Sceglie la <i>migliore</i> tra più soluzioni	Serve saper definire una funzione di utilità adeguata
Che apprende	Migliora nel tempo, si adatta	Complessità architetturale; serve un buon feedback

1.3 Paradigmi di programmazione e agenti autonomi

1.3.1 Dall'approccio tradizionale al dichiarativo

Approccio tradizionale — paradigma imperativo/a oggetti (non-AI software): risolve un singolo compito specifico; è tipicamente strutturato come una **sequenza di passi** che descrivono esplicitamente **come (how)** ottenere il risultato. Esempio dalle slide: una funzione C che inserisce un elemento in una lista ordinata (*ins_ord*), scritta come una sequenza esplicita di operazioni sui puntatori.

Approccio dichiarativo (AI software): separa una **descrizione dichiarativa** del problema (**cosa (what)** si vuole/si sa) da un **programma generale** (motore di inferenza/ricerca) che sa elaborare tale descrizione. Lo **stesso programma generale** può essere applicato a **descrizioni diverse** per risolvere **problemi diversi**, senza essere riscritto. Architettura tipica: una **base di conoscenza (Knowledge Base)** contiene la rappresentazione dichiarativa di un corpo di conoscenze applicabile a molteplici situazioni; le **percezioni** vengono via via integrate in questa conoscenza; un **modulo deliberativo** generale interroga (query) la base di conoscenza per decidere l'azione.

Perché questo cambio di paradigma è importante per l'IA? Perché nei problemi “intelligenti” tipicamente non conosciamo a priori la sequenza esatta di passi che risolve il problema: vogliamo invece che sia il sistema stesso, dato un obiettivo e una descrizione del mondo, a **costruire** la sequenza di passi giusta.

1.3.2 Esempio guida: il mondo dei blocchi (Blocks World)

Il **mondo dei blocchi** è un classico problema giocattolo (*toy problem*) dell'IA. Alcuni blocchi (etichettati A, B, C, D, E, F, G) sono disposti impilati su un piano diviso in postazioni (1, 2, 3, 4). L'agente percepisce la configurazione (stato) **iniziale** dei blocchi.

Passaggi concettuali:

1. **Percezione:** l'agente percepisce la situazione iniziale. Di per sé, in questo momento, **non fa nulla**.
2. **Definizione del goal:** per agire, occorre prima descrivere un **obiettivo**, cioè uno stato di cose che si vuole rendere vero.
3. **Costruzione del piano:** l'agente deve costruire autonomamente la sequenza di passi/azioni $a_1, a_2, \dots, a_k$ che porta dallo stato iniziale (I) allo stato goal (G): $I \rightarrow a_1 \rightarrow a_2 \rightarrow \dots \rightarrow a_k \rightarrow G$.

4. **Conoscenza delle azioni:** per farlo, l'agente deve avere/usare conoscenza su quali azioni sono disponibili (es. "Prendi(blocco)", "Impila(blocco, blocco)", "Mettilo(blocco, posizione)"), su quando ciascuna azione è **applicabile** (precondizioni) e quali sono i suoi **effetti** sul mondo.
5. **Ragionamento:** l'agente deve ragionare per determinare quali azioni, tra tutte quelle applicabili, effettivamente avvicinano (o portano) al goal.

Perché serve ricerca: localmente, nello stato iniziale, potrebbero esserci molte mosse possibili, ma solo alcune di esse sono davvero utili per raggiungere lo stato finale desiderato. Il problema di **come scegliere** tra le mosse possibili è esattamente il problema della **ricerca nello spazio degli stati**.

Soluzione trovata dall'agente (esempio dalle slide): una sequenza concreta come `Prendi(C)`, `Mettilo(C,4)`, `Prendi(B)`, `Impila(B,D)`, `Prendi(C)`, `Impila(C,A)`, ... ecc. Il punto concettuale fondamentale è che **il programma dell'agente costruisce un programma**: elabora la conoscenza sulle mosse possibili per generare autonomamente la sequenza di azioni risolutiva, che il progettista non aveva scritto esplicitamente in anticipo.

1.3.3 Dal problema giocattolo al problema reale

Il mondo dei blocchi è deliberatamente un **toy problem**: stati discreti, ambiente completamente osservabile, deterministico, un solo agente. Cosa succede se ci spostiamo nel mondo reale? — riprendendo l'esempio dell'attraversamento della strada, che è invece un ambiente **complesso, parzialmente prevedibile, parzialmente collaborativo**.

Esempio: identificare un passaggio pedonale da un'immagine reale. Quando una telecamera cattura un'immagine di strisce pedonali, non "vede" delle strisce come le vediamo noi: cattura solo **pixel**, cioè codifiche numeriche di colori con cui l'immagine viene approssimata digitalmente. Fattori che complicano ulteriormente il problema: prospettiva, colore, qualità dell'immagine, necessità di estrarre l'oggetto dal contesto, modi alternativi con cui la stessa scena può essere rappresentata.

Punto concettuale chiave: "vedere" (per un agente artificiale) non significa incamerare passivamente dei dati grezzi, ma **elaborare quei dati fino a trasformarli in informazioni**, secondo modelli che noi umani (e molti animali) sappiamo costruire autonomamente e quasi senza sforzo cosciente.

1.3.4 Da automazione ad autonomia

Automazione: ormai uno standard consolidato in moltissime attività. Richiede di programmare il dispositivo per compiere esplicitamente **ogni singolo passo**. È applicabile bene in domini fortemente **ripetitivi** e prevedibili (es. un robot di saldatura in una catena di montaggio industriale).

Autonomia: un agente artificiale autonomo riceve **compiti ad alto livello** (goal), e l'utente/operatore demanda all'agente stesso il compito di trovare **come** risolverli concretamente — non gli vengono più specificati i singoli passi.

Agente autonomo — caratteristiche:

- In quanto agente, possiede capacità di azione.
- Essendo *autonomo*: riceve solo compiti ad alto livello (non istruzioni passo-passo); ragiona ed esplora alternative (spesso in numero esponenziale); riconosce quando una linea di azione non è più percorribile (vicolo cieco); riconosce se è già stato in una situazione già esplorata (evitando cicli inutili); semplificando, **prima ragiona e poi agisce**.

Autonomia e controllabilità — un equivoco comune: al di fuori della comunità di IA, gli agenti autonomi vengono talvolta percepiti come entità semi-coscienti e incontrollabili la cui autonomia "emerge" al di fuori/al di sopra del programma che le governa. Le slide **correggono**

esplicitamente questo fraintendimento: in IA, gli agenti autonomi sono semplicemente **un modo di concepire i programmi**, in cui il **controllo** (la logica di alto livello, il modello del mondo e degli obiettivi) è chiaramente **separato** dai dettagli implementativi di basso livello. **Un agente fa sempre e soltanto ciò che è stato programmato a fare**: ciò che è stato programmato è “persegui questo obiettivo trovando tu stesso i passi necessari”, non “esegui questi passi specifici”.

Domanda d’esame. Un agente autonomo può fare cose che non erano previste dal suo programma? — No, nel senso tecnico usato in IA. Un agente autonomo esegue sempre il proprio programma; la differenza rispetto a un sistema automatizzato tradizionale è che il programma stesso è scritto per **ragionare e generare piani** in risposta a obiettivi di alto livello, invece di eseguire una sequenza di passi fissata a priori dal programmatore. Le azioni “nuove” che l’agente compie sono il risultato del ragionamento interno (esplorazione di alternative, pianificazione), non un’evasione dal codice.

1.3.5 Gradi di autonomia: tre esempi concreti a confronto

Le slide usano tre sistemi reali per mostrare che **autonomia non è un concetto binario**, ma una questione di **grado**, che dipende dalla complessità dell’ambiente in cui l’agente opera.

Sistema	Ambiente	Grado di autonomia
Metropolitana automatica	“Semplice”: rotaia fissa, partenza/arrivo definiti	Bassa: azione calcolata deterministicamente, nessuna vera decisione strategica
Rover su Marte	Relativamente semplice: piana regolare, niente altri agenti né fenomeni atmosferici complessi	Media/parziale: goal programmabile (solo la destinazione), necessario per il ritardo radio (~14 min)
Auto senza guidatore	Complesso: altri agenti, condizioni atmosferiche variabili, comportamento altrui imprevedibile	Alta: goal fornito dall’operatore, ma l’ambiente è multiagente e fortemente imprevedibile

Tutti e tre condividono lo stesso **ciclo agente** di base: (1) leggi i sensori, (2) applica la funzione di controllo (+ eventualmente il goal) e calcola la prossima azione, (3) applica l’azione calcolata. Ciò che cambia drasticamente tra i tre casi non è la struttura del ciclo, ma la **complessità dell’ambiente** e di conseguenza quanto la “funzione di controllo” debba essere sofisticata.

1.3.6 Ragionare basta?

No. Il ragionamento da solo non è sufficiente per un agente reale, perché tipicamente si opera: in presenza di **incertezza**, in un mondo **non completamente conosciuto**, in presenza di **altri agenti** i cui comportamenti non sono del tutto prevedibili o controllabili.

Per questo occorre **combinare** tre ingredienti insieme, in un ciclo continuo: **azioni** (che modificano concretamente il mondo), **percezioni** (che informano sullo stato del mondo), e **ragionamento** (che decide cosa fare sulla base di percezioni e conoscenza). Nessuno dei tre, da solo, è sufficiente.

Esempio applicativo reale — miniere automatizzate: aziende come Rio Tinto Group (Australia), la miniera di Bingham Canyon (Utah), o EEP Elektro-Elektronik Pranjic (Germania/Cina) utilizzano camion autonomi (es. Komatsu driverless trucks) per trivellazione e brillamento autonomi, controllo di flotte di veicoli, trasporto autonomo del materiale (*autonomous haulage*), evitamento di ostacoli e navigazione.

Approfondimento facoltativo: sistemi multiagente (MAS)

- Un **sistema multiagente (MAS)** è costituito da un insieme di agenti che operano in uno stesso ambiente, potendo **competere o collaborare** nell'uso delle risorse condivise e nel perseguimento dei propri obiettivi.
- Per interagire efficacemente, agenti anche implementati in modo indipendente ed eterogeneo devono condividere: (1) un'**ontologia** del dominio del discorso, (2) un **linguaggio di comunicazione** comune, (3) un **protocollo di interazione** condiviso.
- **FIPA** (Foundation for Intelligent Physical Agents) ha standardizzato, a inizio anni 2000, una semantica formale per gli **atti comunicativi** (speech acts, es. **Request**, **Agree**, **Confirm**, **Cancel**, **Inform**) e diversi protocolli di interazione standard (es. **Contract Net**, **Brokering**, **Query Interaction Protocol**).
- Strumenti software citati: **JADE**, **Jason**, **CartAgO**, **JaCaMo**.

Riepilogo e punti chiave — Modulo 1

- **IA** = intelligenza non naturale, ottenuta con procedimenti tecnici. Nasce ufficialmente nel 1956 (Dartmouth Conference), ma affonda le radici nei lavori di Turing (1936, 1950).
- **Automazione $\neq$ intelligenza**: serve capacità di adattarsi, decidere, eventualmente imparare.
- **Test di Turing**: valuta solo il comportamento osservabile; non garantisce comprensione reale. **Stanza cinese (Searle)**: manipolare simboli sintatticamente non implica comprensione semantica.
- **Strong AI** vs **Weak AI**: il corso adotta l'ottica "agire razionalmente" (weak AI).
- **Agente** = percepisce (sensori) + agisce (attuatori); **sequenza percettiva** = storia delle percezioni; **razionalità** = massimizzare la prestazione *attesa*, non quella reale/onnisciente.
- **PEAS** e le **7 proprietà dell'ambiente** guidano la progettazione; il caso più difficile è parzialmente osservabile + stocastico + sequenziale + dinamico + continuo + multiagente.
- **Cinque tipi di agente**, crescenti in sofisticazione: reattivo semplice → reattivo con modello → basato su obiettivi → basato sull'utilità → che apprende.
- **Paradigma dichiarativo**: separa *cosa* (KB) da *come* (motore generale); esempio guida: mondo dei blocchi, dove "il programma costruisce un programma".
- **Automazione vs. autonomia**: l'autonomia non è comportamento incontrollabile, ma un agente che pianifica invece di eseguire passi fissi; è **graduale**, non binaria (metro → rover → auto autonoma).
- Il ragionamento da solo non basta: serve sempre combinare **azioni** + **percezioni** + **ragionamento**.

Capitolo 2

Risoluzione di problemi mediante ricerca (non informata e informata)

Questo modulo affronta il primo dei tre grandi approcci alla risoluzione automatica di problemi in IA: (1) ricerca nello spazio degli stati (oggetto di questo modulo), (2) ricerca in spazi con avversario (giochi ad informazione completa), (3) risoluzione di problemi mediante soddisfacimento di vincoli (CSP).

L'idea di fondo è che moltissimi problemi si possano modellare come un insieme di **stati** collegati da **azioni**, e che risolvere il problema equivalga a trovare un **percorso** (sequenza di azioni) che porti da uno stato iniziale a uno stato che soddisfa un obiettivo.

2.1 Stati, azioni e astrazione

Alla base di tutto c'è l'idea di astrarre la realtà in un insieme discreto di **stati**, che transitano l'uno nell'altro tramite l'esecuzione di **azioni** (operazioni). Esempio minimale: stato {dentro, fuori}, azione {attraversa_soglia} — rappresentabile come un grafo di transizione di stato.

Caratteristiche tipiche di questa astrazione:

- **stati discreti**: anche se nel mondo fisico un fenomeno è continuo (es. il passaggio graduale attraverso una soglia), lo si discretizza in un numero finito di valori;
- **effetto deterministico delle azioni** (nel caso base — esistono varianti non deterministiche, es. l'azione “passo” che dallo stato “dentro” può portare a due esiti diversi a seconda della posizione non rappresentata nella stanza);
- **dominio statico**: il mondo non cambia se non per effetto delle azioni dell'agente.

Il principio guida dell'astrazione è **rappresentare solo l'informazione rilevante al problema**: per uno stato conta la posizione ma non il panorama; per un'azione conta l'esito raggiunto ma non, ad esempio, l'inquinamento prodotto.

Obiettivi e ricerca

Un **obiettivo** è un risultato verso cui sono diretti gli sforzi, definito da: (1) una **situazione** (es. “essere in un luogo”), (2) eventualmente una **prestazione** associata (es. “...entro un certo tempo”). L'insieme di tutti gli stati che soddisfano la condizione obiettivo è detto **insieme degli stati obiettivo** (o stati *goal/target*).

Un algoritmo di ricerca determina una soluzione che, partendo dallo stato iniziale, raggiunge uno stato obiettivo, utilizzando: (1) una descrizione del problema; (2) un metodo di ricerca nello spazio degli stati. **Una soluzione è un percorso nello spazio degli stati.**

2.2 Definizione formale di problema di ricerca

Un problema di ricerca è definito formalmente come una **tupla di 4 elementi**:

Elemento	Significato
1. Stato iniziale	cattura la situazione da cui si parte per calcolare la soluzione
2. Funzione successore	(azioni + modello di transizione) dato uno stato e un'azione legale in esso, calcola lo stato risultante dall'esecuzione di quell'azione
3. Test obiettivo	determina se uno stato è quello goal
4. Funzione di costo del cammino	assegna un costo numerico a un percorso possibile

L'insieme di tutti gli stati possibili (**spazio degli stati**) è spesso definito in modo implicito/generativo (tramite stato iniziale + funzione successore) piuttosto che enumerato esplicitamente.

Esempio guida: problema di navigazione

Stato iniziale: in(Alessandria). Funzione successore: applicando l'azione go(Ovada) allo stato in(Alessandria) si ottiene lo stato in(Ovada). Test obiettivo: se la destinazione è Genova, verifica se lo stato corrisponde a in(Genova). Funzione di costo del cammino: ad esempio il numero di km percorsi.

Esempio: il mondo dell'aspirapolvere

Due stanze (sinistra S, destra D), ciascuna sporca o pulita; un aspirapolvere che si trova in una delle due stanze e può muoversi o aspirare. Ogni stanza può assumere 4 configurazioni. Test obiettivo: entrambe le stanze pulite. Costo del cammino: ogni azione costa 1.

Esempio: il gioco dell'8 (8-puzzle)

Stato: posizione di ciascuna delle 8 tessere numerate e dello spazio vuoto in una griglia 3×3 . Esistono $181\,440 = 9!/2$ stati raggiungibili (metà delle $9!$ permutazioni, perché solo metà sono raggiungibili da una configurazione data — l'altra metà richiederebbe “sollevare” le tessere). Funzione successore: sposta nello spazio vuoto una delle tessere ad esso adiacenti. Costo del cammino: ogni mossa costa 1.

Toy problem vs Real-world problem

Toy problem: problema artificiale, con formulazione precisa e univoca, usato per illustrare o confrontare metodi di risoluzione (8-puzzle, 8 regine, mondo dell'aspirapolvere, attraversamento del fiume...). **Real-world problem:** problema concreto (configurazione VLSI, itinerari stradali, navigazione robotica...), spesso privo di una formulazione unica e standard.

Domanda d'esame. Formulare il problema delle 8 regine (stati, stato iniziale, funzione successore, test obiettivo, costo). — Occorre posizionare 8 regine su una scacchiera 8×8 in modo che nessuna ne attacchi un'altra. Una formulazione efficiente (*incrementale*) è: **stati** = configurazioni con 0-8 regine sulla scacchiera, una per colonna, nessuna in conflitto; **stato iniziale** = scacchiera vuota; **funzione successore** = aggiungi una regina in una qualsiasi casella libera della colonna successiva che non sia attaccata dalle regine già posizionate; **test obiettivo** = 8 regine sulla scacchiera senza conflitti; **costo del cammino** = 1 per ogni regina piazzata. Questa formulazione riduce enormemente lo spazio di ricerca rispetto a generare tutte le disposizioni di 8 regine su 64 caselle e poi filtrare.

2.3 Spazio degli stati vs albero/grafico di ricerca

È fondamentale distinguere due livelli:

- **Spazio degli stati:** l'insieme (astratto, spesso implicito) di tutti gli stati raggiungibili e le transizioni fra essi, definito dal problema stesso. Esiste indipendentemente dal fatto che venga esplorato.
- **Albero (o grafo) di ricerca:** struttura dati *costruita dall'algoritmo* durante l'esplorazione per trovare una soluzione. Ogni **nodo** dell'albero corrisponde a uno stato, ma un nodo contiene anche altra informazione (riferimento al nodo genitore, azione che lo ha generato, profondità, costo del cammino accumulato). L'albero diventa **grafo** quando lo stesso stato può essere raggiunto da più percorsi diversi.

Formalmente, un grafo di ricerca è una coppia $G = (\{n_i\}, \{e_{ij}\})$: un insieme di nodi e un insieme di archi, dove l'arco e_{ij} rappresenta che n_j è successore di n_i , con un costo c_{ij} associato. L'esistenza di e_{ij} non implica quella di e_{ji} (i grafi sono orientati); se esiste e_{ji} in generale $c_{ji} \neq c_{ij}$.

Durante la ricerca ogni nodo si trova in uno di questi stati: **frontiera** (nodi generati ma non ancora espansi, detti anche nodi APERTI); **espanso/chiuso** (nodo di cui sono già stati generati tutti i successori); non ancora generato. Il criterio con cui si sceglie **quale nodo della frontiera espandere per primo** è ciò che distingue le diverse strategie di ricerca.

2.4 Criteri di valutazione delle strategie di ricerca

Per confrontare le strategie si usano quattro criteri: (1) **Completezza:** garanzia di trovare una soluzione, se esiste. (2) **Ottimalità:** garanzia di trovare la soluzione di costo minimo. (3) **Complessità temporale:** quanto tempo (= quanti nodi generati/espansi) serve. (4) **Complessità spaziale:** quanta memoria (= quanti nodi mantenuti) serve.

Poiché gli algoritmi sono entità astratte, tempo e spazio si misurano in modo **parametrico**, usando la notazione *O*-grande: $T(n)$ è $O(f(n))$ se esiste n_0 e una costante $c > 0$ tali che per ogni $n > n_0$ si abbia $T(n) \leq c \cdot f(n)$.

I parametri standard usati sono: b = fattore di ramificazione (branching factor); d = profondità della soluzione più superficiale; m = profondità massima dello spazio degli stati (può essere infinita).

Gli algoritmi si dividono in: **approcci blind (non informati)** (usano solo la struttura del problema); **approcci informati** (usano anche conoscenza aggiuntiva, euristiche, per guidare la ricerca).

2.5 Ricerca non informata (blind search)

2.5.1 Ricerca in ampiezza (Breadth-First Search, BFS)

Idea: si espande prima il nodo radice, poi tutti i suoi successori (livello 1), poi tutti i discendenti di livello 2, e così via. **Struttura dati:** la frontiera è gestita come una **coda FIFO**. **Nota sulla memoria:** tutti i nodi della frontiera e *tutti i loro antenati* devono restare in memoria, per poter ricostruire il percorso soluzione.

Valutazione BFS: completa (se b finito e soluzione a profondità finita d); ottima solo se il costo del cammino è funzione monotona non decrescente della profondità; complessità temporale $O(b^{d+1})$; complessità spaziale $O(b^{d+1})$.

La complessità spaziale è il vero tallone d'Achille della BFS: con $b = 10$, generando 10.000 nodi/sec e un nodo che occupa 1000 byte, a profondità 12 servirebbero circa **10 petabyte** di memoria e 35 anni di tempo.

2.5.2 Ricerca a costo uniforme (Uniform-Cost Search)

Idea: quando i costi dei passi non sono tutti uguali, occorre espandere sempre il nodo il cui cammino dalla radice ha **costo minimo**. **Struttura dati:** coda con priorità ordinata per costo del cammino accumulato $g(n)$.

Punti di attenzione: costo $\neq$ numero di passi; quando si genera per la prima volta un nodo obiettivo **l'algoritmo non si ferma subito**: prima verifica che non vi siano altri cammini aperti di costo ancora inferiore; quando tutti i costi sono uguali, coincide con la BFS.

Valutazione: completa (se tutti i passi hanno costo > 0); ottima (se costi > 0); complessità $O(b^{1+\lceil C^*/\varepsilon \rceil})$ dove C^* = costo della soluzione ottima, ε = costo minimo delle azioni.

2.5.3 Ricerca in profondità (Depth-First Search, DFS) senza backtracking

Idea: si espande sempre uno dei nodi **più profondi** della frontiera. **Struttura dati:** la frontiera è gestita come una **coda LIFO** (stack).

Valutazione: completa solo se **tutti i cammini sono finiti**; ottimalità **non garantita** in generale; complessità temporale $O(b^m)$ nel caso peggiore; complessità spaziale $O(b \cdot m)$ (versione che genera tutti i successori) oppure $O(m)$ con backtracking classico (un solo successore alla volta).

Il vantaggio della DFS rispetto alla BFS è enorme in termini di spazio: con lo stesso esempio ($b = 10$, $d = 12$) la DFS richiederebbe solo ~ 118 KB contro i 10 petabyte della BFS.

Variante: ricerca a profondità limitata

Si introduce un limite artificiale l : un nodo viene espanso solo se la sua profondità $p \leq l$. Problema 1: si perde completezza se $d > l$. Problema 2: se $l > d$, si perde efficienza. Il problema pratico è che d non è quasi mai noto a priori.

2.5.4 Iterative Deepening Depth-First Search (IDDFS)

Idea: esegue ripetutamente una ricerca a **profondità limitata**, aumentando progressivamente il limite l ($0, 1, 2, 3, \dots$) fino a trovare la soluzione. Ad ogni iterazione l'albero di ricerca viene **ricostruito da zero**.

Perché non è così costoso ripartire da zero?

La maggior parte dei nodi si trova **vicino alla frontiera**, quindi i nodi vicini alla radice vengono rigenerati più volte ma sono relativamente pochi rispetto a quelli generati una sola volta al livello più profondo. Il numero totale di nodi generati è:

$$N_{id} = d \cdot b + (d-1) \cdot b^2 + \dots + (d-i) \cdot b^{i+1} + \dots + 1 \cdot b^d = \sum_{i=1}^d (d-i) b^i,$$

minore del numero di nodi generati dalla BFS: $N_{amp} = b + b^2 + \dots + b^d + (b^{d+1} - b)$.

Esempio numerico ($b = 10, d = 5$): $N_{id} \approx 123\,456$ nodi contro $N_{amp} \approx 1\,111\,100$ nodi.

Valutazione IDDFS: completa (se b finito); ottima (se il costo è non decrescente con la profondità); complessità spaziale $O(b \cdot d)$; complessità temporale $O(b^d)$. È la strategia **preferita** quando lo spazio di ricerca è ampio e la profondità della soluzione non è nota a priori.

2.5.5 Ricerca bidirezionale

Idea: eseguire simultaneamente **due ricerche**: una **forward** dallo stato iniziale, una **backward** dallo stato obiettivo. La ricerca termina quando le due frontiere si intersecano.

Per determinare la ricerca all'indietro occorre saper calcolare i **predecessori** di uno stato — operazione non sempre semplice. Se lo stato obiettivo non è unico, si introduce uno **stato “di comodo”** raggiungibile in un passo da tutti gli stati obiettivo reali.

Valutazione: complessità temporale e spaziale $O(b^{d/2})$ (più precisamente $2 \cdot O(b^{d/2})$): esempio con $b = 2, d = 6$, si passa da $b^d = 64$ a $b^{d/2} = 8$. Completa e ottima se b finito e si usa BFS in entrambe le direzioni.

2.5.6 Grafi di ricerca e nodi già esplorati

In molti problemi lo stesso stato è raggiungibile tramite percorsi diversi: marcare i nodi già visitati e controllare, prima di espandere un nodo, se lo stato corrispondente è già stato visitato, rende la ricerca molto più efficiente.

Esempi classici di problemi di attraversamento

- **Capra, cavolo e lupo**: un uomo deve traghettare pecora, cavolo e lupo con una barca che ne trasporta uno alla volta, senza mai lasciare incustodite sulla stessa riva pecora+cavolo o lupo+pecora.
- **Missionari e cannibali**: 3 missionari e 3 cannibali devono attraversare un fiume con una barca da 1-2 posti, senza che in nessuna riva i cannibali siano mai in maggioranza rispetto ai missionari.
- **Torre di Hanoi**: spostare una torre di dischi dal primo al terzo piolo, potendo sovrapporre solo dischi più piccoli su dischi più grandi.

2.5.7 Tabella riassuntiva — confronto strategie di ricerca non informata

Strategia	Completa	Ottima	Tempo	Spazio
Ampiezza (BFS)	Sì (b finito)	Sì (costo non decr.)	$O(b^{d+1})$	$O(b^{d+1})$
Costo uniforme	Sì (costi > 0)	Sì (costi > 0)	$O(b^{1+\lceil C^*/\varepsilon \rceil})$	idem
Profondità (DFS)	No, se cammini finiti	No	$O(b^m)$	$O(bm) [O(m)]$
Prof. limitata	No, solo se $l \geq d$	No	$O(b^l)$	$O(bl)$
Iterative deepening	Sì (b finito)	Sì	$O(b^d)$	$O(bd)$
Bidirezionale	Sì (b finito)	Sì (costi unif.)	$O(b^{d/2})$	$O(b^{d/2})$

2.6 Ricerca informata (heuristic search)

Domanda d'esame. Le strategie di ricerca blind non sono efficienti. È possibile rendere la ricerca più efficiente utilizzando conoscenza sul problema? — Sì. Le strategie non informate esplorano lo spazio degli stati “alla cieca”. Se disponiamo di conoscenza aggiuntiva sul problema — tipicamente una **stima** di quanto uno stato sia vicino all'obiettivo — possiamo usarla per **ordinare la frontiera** e dare priorità agli stati più promettenti, riducendo drasticamente il numero di nodi generati/espansi. Questa stima si chiama **funzione euristica** $h(n)$.

2.6.1 Funzioni di valutazione e best-first search

Una strategia di ricerca informata ordina la frontiera in base a una **funzione di valutazione** $f(n)$ applicata a ciascun nodo. Questa famiglia di strategie è detta **best-first search**. Spesso $f(n)$ include una componente $h(n)$, detta **euristica**, che stima il costo minimo del cammino residuo dallo stato corrispondente a n fino a uno stato obiettivo.

2.6.2 Ricerca greedy (avida)

$f(n) = h(n)$: si sceglie sempre il nodo che l'euristica stima più vicino all'obiettivo, ignorando completamente il costo già speso per arrivarci.

Problemi della ricerca greedy: l'euristica può ingannare; rischio di **vicoli ciechi** e loop (eredita i difetti della DFS); **non è né completa né ottima** in generale.

2.6.3 A* (A-star)

Idea: combinare i punti di forza della ricerca a costo uniforme (guarda al **passato**) e della ricerca greedy (guarda al **futuro**):

$$f(n) = g(n) + h(n),$$

dove $g(n)$ = costo minimo per raggiungere il nodo n dallo stato iniziale, $h(n)$ = stima del costo residuo.

Confronto con i valori “veri” (ideali): $g^*(n)$ = costo minimo reale per raggiungere n da s ; $h^*(n)$ = costo minimo reale del proseguimento; $f^*(n) = g^*(n) + h^*(n) = C^*$ per i nodi sul cammino ottimo.

Pseudocodice di A*

```
Sia  $s$  il nodo iniziale,  $T$  l'insieme dei nodi obiettivo
segna  $s$  come APERTO e calcola  $f(s)$ 
ripeti:
  seleziona il nodo APERTO  $n$  con  $f(n)$  minima
  se  $n \in T$ : marca  $n$  CHIUSO e termina
  altrimenti: marca  $n$  CHIUSO, applica la funzione successore
  (i successori già CHIUSI ma raggiunti con  $f$  più basso vengono riaperti)
```

Struttura dati: coda a priorità ordinata su $f(n) = g(n) + h(n)$.

Euristiche ammissibili

Una funzione euristica h è **ammissibile** se, per ogni nodo n ,

$$h(n) \leq h^*(n),$$

cioè non sovrastima mai — è sempre ottimistica (o al più esatta). Esempio: la distanza in linea d'aria è ammissibile rispetto alla distanza reale su strada.

Ottimalità di A*

Teorema: se (1) l'euristica h è ammissibile e (2) tutti i passi hanno costo maggiore di $\varepsilon > 0$, allora **A* termina e trova sempre una soluzione ottima**.

Dimostrazione (caso albero di ricerca): supponiamo per assurdo che A* scelga un obiettivo subottimo G_2 al posto di un nodo n su un cammino ottimo verso il vero obiettivo G , con $f^*(G) = g^*(G) + h^*(G) = C^*$.

1. G_2 è un nodo obiettivo, quindi $h(G_2) = 0$.

2. Quindi $f(G_2) = g(G_2) + h(G_2) = g(G_2)$.
3. Poiché G_2 è per ipotesi subottimo, $f(G_2) = g(G_2) > C^*$.
4. Sia n un nodo qualunque sul cammino ottimo: in un albero, $g^*(n) = g(n)$.
5. Poiché h è ammissibile, $h(n) \leq h^*(n)$.
6. Quindi $f(n) = g(n) + h(n) \leq g^*(n) + h^*(n) = f^*(n) = C^*$.
7. Combinando (3) e (6): $f(n) \leq C^* < f(G_2)$.
8. A^* sceglie sempre il nodo APERTO con f minima: fra n e G_2 , A^* sceglierà sempre n , mai G_2 — contraddizione. $\square$

Caso grafo di ricerca: serve la consistenza (monotonicità)

Nei grafi esiste **molteplicità di cammini**: il primo cammino trovato verso uno stato non è necessariamente quello di costo minimo. Per i grafi occorre l'ipotesi più forte di **euristica consistente (o monotona)**: per ogni nodo n e ogni suo successore n' generato tramite un'azione a ,

$$h(n) \leq c(n, a, n') + h(n')$$

(disuguaglianza triangolare).

Dimostrazione che con h consistente i valori $f(n)$ sono non decrescenti lungo un cammino:

1. $g(n') = g(n) + c(n, a, n')$ per definizione.
2. $f(n') = g(n') + h(n')$ per definizione.
3. Sostituendo: $f(n') = g(n) + c(n, a, n') + h(n')$.
4. Per la disuguaglianza triangolare, $c(n, a, n') + h(n') \geq h(n)$, quindi $f(n') \geq g(n) + h(n) = f(n)$. $\square$

Poiché A^* espande i nodi in ordine non decrescente di f , il primo incontro con un nodo obiettivo durante l'espansione corrisponde già alla soluzione ottima.

Monotonicità vs ammissibilità

La monotonicità (consistenza) è una proprietà **più forte** dell'ammissibilità: ogni euristica monotona è anche ammissibile, ma non vale il viceversa in generale.

Ammissibile non vuol dire informativo

$h(n) = 0$ è banalmente sempre ammissibile, ma è totalmente **priva di informazione utile**: degrada A^* a una ricerca "cieca" basata solo su $g(n)$. Se in aggiunta tutti i passi hanno costo uniforme pari a 1, **A^* con $h = 0$ diventa equivalente alla ricerca in ampiezza**.

Ottimalità in senso forte di A^*

A^* è **ottimamente efficiente** per qualunque euristica data: non esiste alcun altro algoritmo ottimo (che usi la stessa euristica) in grado di garantire l'espansione di un numero di nodi inferiore a quello espanso da A^* . Il difetto pratico è che il numero di nodi espansi cresce **esponenzialmente** con la profondità della soluzione ottima, e A^* mantiene in memoria **tutti** i nodi generati.

2.6.4 Ridurre i requisiti di memoria: IDA* e RBFS

IDA* (Iterative Deepening A^*): unisce l'idea di A^* con quella dell'iterative deepening, usando come limite di taglio ad ogni iterazione il valore di $f(n)$ invece della profondità.

RBFS (Recursive Best-First Search, Korf 1993): funziona come una DFS ricorsiva, ma mantiene un **upper bound dinamico** B che rappresenta la stima di costo della migliore alternativa

attualmente nota, permettendo di abbandonare un ramo non appena esso smette di essere il più promettente.

Ogni chiamata ricorsiva ha tre argomenti: un nodo N , un valore $F(N)$ e un upper bound B . $f(n) = g(n) + h(n)$ è statica; $F(N)$ è dinamica: $F(N) = f(N)$ se esplorato per la prima volta, altrimenti $F(N) = \min(F(\text{figli di } N))$.

Valutazione RBFS: ottima se h ammissibile; complessità spaziale **lineare**, $O(b \cdot d)$; difetto: non tiene traccia dei cammini ripetuti e non sfrutta memoria aggiuntiva disponibile.

2.6.5 Funzioni euristiche: il caso di studio dell'8-puzzle

Generando casualmente lo stato iniziale, in media servono **22 mosse** per risolverlo, con un **branching factor medio pari a 3**. L'albero esaustivo conterrebbe circa 3^{22} nodi, oltre 30 miliardi. Il grafo esaustivo (senza duplicati) contiene "solo" circa 180.000 stati. Passando al 15-puzzle, lo spazio esplode fino a circa 10^{13} stati.

Due euristiche ammissibili classiche

- h_1 = **numero di tessere fuori posto**. Ammissibile perché ogni tessera fuori posto dovrà essere spostata almeno una volta.
- h_2 = **distanza di Manhattan**. Somma, su tutte le tessere, della distanza fra la posizione corrente e quella desiderata. Ammissibile perché ogni mossa può avvicinare una tessera al proprio posto di **al più una posizione** per volta.

Effective branching factor (fattore di ramificazione effettivo) b^*

Dato un problema risolto con A^* , siano N = numero di nodi generati, d = profondità della soluzione trovata. b^* è definito come il branching factor che avrebbe un albero uniforme di profondità d che contenga esattamente $N + 1$ nodi:

$$N + 1 = 1 + b^* + (b^*)^2 + \dots + (b^*)^d.$$

Quanto più b^* è vicino a 1, tanto migliore è l'euristica.

Esperimento citato: 1200 problemi del 15-puzzle, profondità 2–24, risolti con IDDFS e A^* con h_1 e h_2 : h_2 **produce sistematicamente meno nodi e un b^* più basso** di h_1 .

Valutazione teorica: euristiche dominanti

Siano h_1 e h_2 due euristiche ammissibili tali che per ogni nodo n , $h_2(n) \geq h_1(n)$. Si dice che h_2 **domina** h_1 . Si dimostra che A^* espande tutti e soli i nodi con $f(n) < C^*$, cioè $h(n) < C^* - g(n)$: se $h_2(n) \geq h_1(n)$, l'insieme dei nodi che soddisfano la condizione con h_1 è un **soprainsieme** di quello con h_2 . Quindi **un'euristica più informata (dominante) porta A^* a espandere un numero di nodi minore o uguale**.

Costruzione sistematica di euristiche ammissibili: problemi rilassati

Un **problema rilassato** si ottiene rimuovendo (parte de)i vincoli del problema originale. Il grafo degli stati del problema rilassato è un **supergrafo** di quello originario. Poiché il supergrafo contiene comunque tutte le soluzioni ottime del problema originale, **il costo della soluzione ottima nel problema rilassato è un'euristica ammissibile per il problema originale**.

Nell'8-puzzle: rimuovendo solo il vincolo "B deve essere vuota" si ottiene un'euristica vicina a h_2 ; rimuovendo anche il vincolo di adiacenza si ottiene h_1 . Il programma **Absolver II** (Prieditis, 1993) genera automaticamente euristiche ammissibili per astrazione.

Combinare euristiche non comparabili

Se si dispone di più euristiche ammissibili $h_1, \dots, h_k$ nessuna delle quali domina le altre, si può costruire un'euristica composta

$$h(n) = \max\{h_1(n), h_2(n), \dots, h_k(n)\},$$

che è **ammissibile** ed è **dominante per costruzione** su ciascuna delle euristiche che la compongono.

Alternativa: apprendere le euristiche induttivamente

Si arricchisce ogni stato con un insieme di *feature* (es. numero di celle fuori posto), si generano casualmente molti problemi, li si risolve raccogliendo i dati, e si applica un metodo di apprendimento induttivo per estrarre automaticamente una funzione euristica dai dati raccolti.

Riepilogo e punti chiave — Modulo 2

- Un **problema di ricerca** è formalizzato da 4 elementi: stato iniziale, funzione successore, test obiettivo, funzione di costo del cammino. Lo **spazio degli stati** va tenuto distinto dall'**albero/grafico di ricerca**.
- Le strategie **non informate** si distinguono per come gestiscono la frontiera: FIFO (ampiezza), coda a priorità su $g(n)$ (costo uniforme), LIFO (profondità), stack con limite (profondità limitata), stack ricostruito iterativamente (iterative deepening), doppia frontiera (bidirezionale). **Iterative deepening** è **generalmente la scelta migliore** quando d non è nota.
- Le strategie **informate** sfruttano $h(n)$. La **greedy** ($f = h$) non è completa né ottima. **A*** ($f = g + h$) combina costo uniforme e greedy.
- **A* con euristica ammissibile è ottimo su alberi**; sui **grafi** serve la **consistenza/monotonicità**. A* è **ottimamente efficiente**, ma mantiene tutti i nodi in memoria; **IDA*** e **RBFS** riducono lo spazio a $O(bd)$.
- La qualità di un'euristica si misura con l'**effective branching factor** b^* ; un criterio teorico complementare è la **dominanza**. Le euristiche ammissibili si costruiscono sistematicamente tramite **problemi rilassati**.

Capitolo 3

Ricerca con avversario (giochi)

3.1 Introduzione: perché studiare i giochi in AI

Marvin Minsky (1968) osservava che “i giochi non vengono scelti perché sono chiari e semplici, ma perché ci danno la massima complessità con le minime strutture iniziali”. I giochi sono quindi un banco di prova ideale per l’AI: regole semplici e stato ben definito, ma spazio di ricerca enorme e presenza di un **avversario** che rende il problema strutturalmente diverso dalla ricerca “in solitaria” vista nei moduli precedenti.

Il tema dei giochi attraversa più discipline: **Matematica** (teoria dei grafi, complessità computazionale), **Computer Science** (AI, database, calcolo parallelo), **Economia** (teoria dei giochi, economia cognitiva/sperimentale), **Psicologia** (fiducia, percezione del rischio).

Ambienti competitivi

Un **ambiente multi-agente** è competitivo quando gli obiettivi degli agenti sono **conflittuali**: il conseguimento dell’obiettivo di un agente impedisce (almeno in parte) il conseguimento di quello degli altri. In questo contesto, i problemi di ricerca con avversario sono detti **giochi**.

3.2 Tassonomia dei giochi

I giochi si classificano lungo due assi indipendenti:

Condizioni di scelta (informazione): **informazione perfetta/completa** (lo stato del gioco è totalmente esplicito e visibile a tutti i giocatori, es. scacchi, dama, Otello, Forza4, tris) vs **informazione imperfetta** (lo stato è solo parzialmente noto ad ogni giocatore, es. Mastermind, Scarabeo, Bridge, Poker).

Effetti della scelta (determinismo): **deterministici** (lo stato successivo è determinato unicamente dalle azioni degli agenti) vs **stocastici** (lo stato successivo dipende anche da fattori esterni casuali, es. lancio di dadi in Backgammon e Monopoli).

	Informazione perfetta	Informazione imperfetta
Deterministici	Scacchi, Go, Dama, Otello, Tris	Mastermind, Scarabeo, Bridge, Poker
Stocastici	Backgammon, Monopoli	Risiko

Il Modulo si concentra sui **giochi ad informazione completa (perfetta)**, tipicamente deterministici e a due giocatori, a **somma zero**.

Gioco a somma zero

Un gioco a somma zero è un contesto di interazione multi-agente in cui la perdita (o il guadagno) di un agente è esattamente compensata dal guadagno (perdita) degli altri agenti.

Nei giochi a due giocatori a somma zero, se l'utilità di uno stato terminale per il giocatore A è u , l'utilità dello stesso stato per B è $-u$: è sufficiente memorizzare un solo valore per conoscere automaticamente anche l'altro.

Perché il tris è un caso interessante da studiare

Osservando lo sviluppo di una partita a tris si notano fenomeni chiave: ogni giocatore, quando è il suo turno, sceglie tra più mosse possibili (branching); l'ambiente è multi-agente (l'evoluzione dello stato è solo parzialmente controllabile); una mossa che apre a una vittoria "a tre passi" può essere più rischiosa di una vittoria immediata; in una posizione di svantaggio conviene comunque "prolungare il più possibile" la partita.

Differenze rispetto alla ricerca (in)formata "classica"

Ricerca classica	Ricerca con avversario
Obiettivo: trovare un cammino a costo minimo	Obiettivo: determinare una strategia (funzione mossa per ogni stato), non una singola sequenza fissa
Si usa $g(x)$, il costo del cammino	$g(x)$ non si usa: costo delle azioni assunto uniforme
I nodi terminali hanno un costo/valore	I nodi terminali sono categorizzati come vittoria, sconfitta, parità
Il nodo successore è sempre scelto dall'agente	Anche l'avversario muove

3.3 Caratteristiche formali del gioco (a due giocatori)

Due giocatori, convenzionalmente chiamati **MAX** e **MIN**; MAX muove per primo. Ciascun giocatore non conosce in anticipo la mossa che farà l'altro, ma **conosce l'insieme delle mosse possibili**. A partire dallo stato iniziale si costruisce un **albero di gioco**. I giocatori valutano gli stati tramite una funzione di **utilità**. **Ipotesi di pessimismo**: si assume che l'avversario sia "infallibile" e giochi sempre la mossa che gli porta il massimo guadagno possibile. Un **ply** è un mezzo turno (un turno completo = 2 ply).

Albero di gioco: definizione

La **radice** è lo stato iniziale (turno di MAX); ogni **nodo interno** rappresenta uno stato di gioco, etichettato come nodo MAX o nodo MIN; gli **archi** rappresentano le mosse legali; le **foglie** sono gli stati terminali, cui è associata un'**utilità**.

3.4 Algoritmo Minimax

Idea e strategia ottima

La **strategia ottima** per un agente, assumendo un avversario perfetto, massimizza l'utilità garantita. L'agente costruisce mentalmente l'intero albero di gioco, assumendo che: nei nodi MAX venga scelta la mossa che **massimizza** il valore per MAX; nei nodi MIN venga scelta la mossa che **minimizza** il valore per MAX.

Definizione ricorsiva: valore-minimax

$$\text{Minimax}(n) = \begin{cases} \text{Utilità}(n) & \text{se } n \text{ è terminale} \\ \max_{s \in \text{Succ}(n)} \text{Minimax}(s) & \text{se } n \text{ è un nodo MAX} \\ \min_{s \in \text{Succ}(n)} \text{Minimax}(s) & \text{se } n \text{ è un nodo MIN} \end{cases}$$

Il valore minimax è calcolato **ricorsivamente per ogni nodo** dell'albero, dal basso verso l'alto. Da qui il nome “minimax”: ogni giocatore, minimizzando il guadagno massimo dell'altro, minimizza automaticamente la propria perdita.

Pseudocodice

```
funzione MINIMAX-DECISION(stato) restituisce una mossa
return arg maxa ∈ Azioni(stato) MIN-VALUE(Risultato(stato,a))

funzione MAX-VALUE(stato) restituisce un valore di utilità
if TEST-TERMINALE(stato) then return Utilità(stato)
v ← -∞
for each a in Azioni(stato) do v ← max(v, MIN-VALUE(Risultato(stato,a)))
return v

funzione MIN-VALUE(stato) restituisce un valore di utilità
if TEST-TERMINALE(stato) then return Utilità(stato)
v ← +∞
for each a in Azioni(stato) do v ← min(v, MAX-VALUE(Risultato(stato,a)))
return v
```

Esempio passo-passo

Consideriamo un piccolo albero di gioco a 4 livelli (2 turni completi, 4 ply: MAX-MIN-MAX-MIN), con branching factor 3 e 9 foglie:

MAX (radice)								
MIN(A)			MIN(B)			MIN(C)		
3	12	8	2	4	6	14	5	2

Valutazione bottom-up: nodo MIN(A) = min(3, 12, 8) = 3; nodo MIN(B) = min(2, 4, 6) = 2; nodo MIN(C) = min(14, 5, 2) = 2; radice (MAX) = max(3, 2, 2) = 3 ⇒ MAX sceglierà il ramo verso MIN(A).

Il valore minimax della radice è **3**: è il massimo risultato che MAX può *garantirsi* contro un avversario che gioca in modo ottimale (non è detto sia il massimo assoluto raggiungibile nell'albero — qui 14 — perché per raggiungerlo MAX dovrebbe “sperare” in un errore di MIN).

Complessità

Temporale: $O(b^m)$; spaziale: $O(bm)$ se i successori vengono generati uno alla volta.

Proprietà

Completo: sì, su alberi/grafi finiti. Ottimale: sì, ma solo a condizione che **sia MAX sia MIN giochino in modo ottimale** (se l'avversario reale non gioca ottimamente, la strategia minimax resta comunque *sicura*).

Estensione a più di due giocatori

Minimax si estende a giochi con più di due giocatori sostituendo il valore scalare di ogni nodo con un **vettore di utilità** $\langle u_1, \dots, u_k \rangle$. Un fenomeno tipico che emerge è la possibile **formazione di alleanze**.

Domanda d'esame. Perché in un gioco a due giocatori a somma zero basta un solo valore per nodo, mentre con più giocatori serve un vettore? — Nella somma zero a due giocatori l'utilità di un giocatore è sempre l'opposto di quella dell'altro ($u_2 = -u_1$), quindi un solo numero descrive completamente la situazione di entrambi. Con tre o più giocatori questa relazione di opposizione non vale più (soprattutto se si formano alleanze), quindi occorre tracciare separatamente l'utilità di ciascun giocatore in ogni nodo.

3.5 Teoria delle decisioni: maximax, maximin, minimax regret

Scenario di esempio: tabella dei payoff

Si deve scegliere un investimento tra tre alternative, sapendo che il mercato può salire, restare stabile o scendere:

alternative	sale	stabile	scende
fondo1	40	45	5
fondo2	70	30	-20
azioni	55	40	-10

Approccio maximax (ottimistico)

Si guarda, per ciascuna scelta, il **payoff più alto ottenibile**: fondo1→45, fondo2→**70**, azioni→55 ⇒ si sceglie **fondo2**. Approccio **ottimistico**.

Approccio maximin (pessimistico/conservativo)

Si guarda, per ciascuna scelta, il **payoff peggiore possibile**: fondo1→5, fondo2→ -20, azioni→ -10 ⇒ si sceglie **fondo1**. È la stessa logica prudente su cui si basa **minimax** nei giochi con avversario.

Approccio minimax regret

Regret = Best payoff (nello scenario) – Real payoff (della scelta fatta).

alternative	sale	stabile	scende
fondo1	30	0	0
fondo2	0	15	25
azioni	15	5	15

Per ciascuna alternativa si prende il **regret massimo**: fondo1→30, fondo2→25, azioni→15. Si sceglie l'alternativa con il **minimo tra i regret massimi**: **azioni**, con rimpianto massimo pari a 15.

Collegamento con i giochi con avversario

Nei giochi, la “dinamica esterna non controllabile” è costituita proprio dalle **mosse dell’avversario**. I giocatori razionali sono **pessimisti/conservativi** per definizione: per questo l’algoritmo Minimax è concettualmente lo stesso approccio **maximin** applicato ricorsivamente lungo l’albero di gioco.

3.6 Potatura alfa-beta (Alpha-Beta Pruning)

Motivazione

Minimax richiede una visita **esaustiva** dell’albero, con complessità temporale esponenziale $O(b^m)$: in giochi reali è del tutto improponibile. L’idea della potatura alfa-beta è che se durante la visita si scopre che un ramo non potrà mai influenzare la decisione finale alla radice, si può **evitare di espanderlo**.

Definizione di α e β

- α = il **massimo lower bound** trovato finora per MAX lungo il cammino (inizialmente $-\infty$, aggiornato dai nodi **MAX**);
- β = il **minimo upper bound** trovato finora per MIN lungo il cammino (inizialmente $+\infty$, aggiornato dai nodi **MIN**).

Ha senso continuare a esplorare un nodo se e solo se il suo valore stimato è compreso nell’intervallo $[\alpha, \beta]$.

Quando si pota un ramo

Se in un certo nodo $\beta \leq \alpha$, nessun valore del sottoalbero rimanente potrà mai influenzare la decisione: si pota. **Beta-pruning**: in un nodo MIN quando trova un successore con $v \leq \alpha$. **Alfa-pruning**: simmetricamente in un nodo MAX quando trova un successore con $v \geq \beta$.

Pseudocodice

```
funzione MAX-VALUE(stato,  $\alpha$ ,  $\beta$ ) restituisce un valore
if TEST-TERMINALE(stato) then return Utilità(stato)
 $v \leftarrow -\infty$ 
for each  $a$  in Azioni(stato) do
 $v \leftarrow \max(v, \text{MIN-VALUE}(\text{Risultato}(\text{stato}, a), \alpha, \beta))$ 
if  $v \geq \beta$  then return  $v$       (beta cutoff)
 $\alpha \leftarrow \max(\alpha, v)$ 
return  $v$ 

funzione MIN-VALUE(stato,  $\alpha$ ,  $\beta$ ) restituisce un valore
if TEST-TERMINALE(stato) then return Utilità(stato)
 $v \leftarrow +\infty$ 
for each  $a$  in Azioni(stato) do
 $v \leftarrow \min(v, \text{MAX-VALUE}(\text{Risultato}(\text{stato}, a), \alpha, \beta))$ 
if  $v \leq \alpha$  then return  $v$       (alfa cutoff)
 $\beta \leftarrow \min(\beta, v)$ 
return  $v$ 
```

MAX-VALUE modifica il lower bound (α), **MIN-VALUE** modifica l’upper bound (β).

Equivalenza con Minimax e guadagno in complessità

Alfa-beta pruning e Minimax sono **equivalenti**: trovano entrambi la stessa mossa ottima e attribuiscono al nodo radice la stessa valutazione. Alfa-beta raggiunge lo stesso risultato **espandendo molti meno nodi**: nel caso migliore, la complessità temporale passa da $O(b^m)$ a $O(b^{m/2})$. Esempio: con $b = 2$, $m = 8$, si passa da $2^8 = 256$ nodi a $2^4 = 16$ nodi espansi.

Importanza dell'ordinamento delle mosse

L'efficacia della potatura **dipende fortemente dall'ordine** in cui i successori vengono considerati: se il successore più promettente (**killer move**) viene esaminato per ultimo, il guadagno si riduce drasticamente. La complessità $O(b^{m/2})$ si ottiene solo con ordinamento ottimale (branching effettivo $\approx \sqrt{b}$, es. scacchi $b = 35 \rightarrow \approx 6$); ordine casuale (caso medio) $\rightarrow O(b^{3m/4})$.

Domanda d'esame. Perché l'ordinamento delle mosse è così cruciale per alfa-beta? — Perché la potatura si verifica solo quando i bound α e β si “incrociano” abbastanza presto durante l'esplorazione dei fratelli di un nodo. Se il valore migliore (killer move) viene trovato subito, i bound si stringono rapidamente e i fratelli successivi vengono scartati senza doverli espandere. Se invece la mossa migliore viene trovata per ultima, nessuna potatura può avvenire: nel caso peggiore alfa-beta degenera esattamente nella stessa complessità di Minimax, $O(b^m)$.

Come trovare le killer moves

- (1) **Apprendimento**: il sistema “ricorda” le esperienze passate.
- (2) **Combinazione con iterative deepening**: le informazioni ottenute alle iterazioni più superficiali orientano quelle profonde.
- (3) **Tabelle di trasposizione**.

Trasposizioni

In molti giochi, sequenze di mosse diverse nell'ordine possono condurre allo **stesso stato finale** (**trasposizioni**). Si mantiene una **hash table**: se lo stato è già stato raggiunto, non lo si esplora di nuovo. Quando lo spazio eccede la memoria disponibile, si mantengono solo le voci usate più di frequente o più di recente (LRU).

3.7 Ricerca con profondità limitata e funzioni di valutazione euristica

Il problema del real-time

Alfa-beta deve comunque arrivare fino agli stati terminali. In giochi con alberi molto profondi questo può risultare troppo lento. Serve introdurre dei **test di cutoff**.

Funzione di valutazione euristica

Si usa una **funzione di valutazione lineare** che combina un insieme di caratteristiche pesate:

$$\text{eval}(s) = \sum_{i=1}^k w_i \cdot f_i(s),$$

dove f_i sono le feature dello stato (es. negli scacchi: numero di pedoni, alfieri, torri ecc.) e w_i i pesi associati.

Alfa-beta con cutoff (versione con profondità limitata)

Si sostituisce il test di terminalità con un test di taglio generico: `if [TEST-TAGLIO(stato, profondità)] then return eval(stato)`. Strategie: (1) profondità massima predefinita; (2) iterative deepening (si esegue la ricerca a profondità crescenti, restituendo la miglior mossa trovata allo scadere del tempo).

Problema dell'orizzonte

I tagli artificiali introducono il **problema dell'orizzonte**: l'algoritmo “non vede oltre” il punto di taglio, ma la situazione può ribaltarsi rapidamente subito dopo.

Quiescenza

Concetto proposto da Berliner (1973): la decisione di terminare o continuare la ricerca in un nodo deve dipendere dalla **quiescenza** della funzione di valutazione in quel nodo, cioè dalla stabilità nel tempo del segno della valutazione. Nodi **quiescenti** (stabili) possono essere tagliati in sicurezza; nodi **non quiescenti** richiedono ulteriore esplorazione.

3.8 Giochi stocastici: cenni (expectiminimax)

Nei giochi stocastici (es. Backgammon, Monopoli, Risiko) lo stato successivo dipende anche da fattori esterni non controllabili (lancio di dadi). Concettualmente, questo richiederebbe di estendere l'albero di gioco con **nodi di caso** (chance nodes), il cui valore si calcola come **valore atteso** (media pesata sulle probabilità) dei valori dei successori — da qui il nome *expectiminimax*.

3.9 Programmi storici che giocano

Checkers: Samuel, Chinook. **Othello**: Logistello. **Backgammon**: TD-gammon. **Go**: Alpha-Go. **Bridge**: Bridge Baron, GIB. **Scacchi**: DeepBlue.

DeepBlue

Primo calcolatore a battere un Campione del Mondo in carica (Garry Kasparov) a scacchi, con cadenza di tempo da torneo (10 febbraio 1996). Hardware: sistema a parallelismo massivo con 30 nodi RS/6000, 480 processori VLSI dedicati; capacità 200 milioni di posizioni al secondo. Algoritmo: **iterative deepening + alpha-beta search con tabella delle trasposizioni**. Di routine raggiungeva profondità 14, in certi casi fino a 40. Funzione di valutazione: oltre 8000 feature; database con 700.000 partite di gran maestri.

AlphaGo

Sviluppato da Google, primo programma a battere un maestro umano a Go **senza handicap** (2015). Approccio: combinazione di **deep learning (reti neurali)** e **ricerca su alberi**; le reti furono addestrate su un dataset di 30.000.000 di mosse umane e poi ulteriormente raffinate tramite **self-play**.

Un parallelo con la memoria umana

Legge di Miller (1956): la capacità della memoria a breve termine umana è stimata in 7 ± 2 elementi. **Cowan (2001)**: revisione più recente, stima 4 ± 1 elementi.

Riepilogo e punti chiave — Modulo 3

- I **giochi** sono problemi di ricerca **multi-agente competitiva**. Questo modulo tratta soprattutto giochi **deterministici a informazione completa, a due giocatori, a somma zero**.
- In un **gioco a somma zero**, basta un solo valore di utilità per nodo. L'**albero di gioco** alterna turni di **MAX** e **MIN** (l'avversario, assunto infallibile).
- **Minimax** calcola ricorsivamente il valore di ogni nodo: $O(b^m)$ in tempo, $O(bm)$ in spazio; completo e ottimale (se entrambi giocano ottimo).
- **Maximax** (ottimistico), **maximin** (pessimistico, identico a Minimax), **minimax regret** (minimizza il rimpianto massimo).
- La **potatura alfa-beta** pota quando $\beta \leq \alpha$: stesso risultato di Minimax, ma nel caso migliore $O(b^{m/2})$. L'efficacia dipende **criticamente dall'ordinamento delle mosse**: nel caso peggiore degenera in $O(b^m)$.
- Nei giochi reali si usano **cutoff a profondità limitata** con **funzioni di valutazione euristica**, mitigando il **problema dell'orizzonte** con la **quiescenza**.
- **DeepBlue** e **AlphaGo** sono applicazioni storiche di questi principi.

Capitolo 4

Constraint Satisfaction Problems (CSP)

I problemi visti nei moduli precedenti (ricerca cieca ed euristica) trattano gli stati come “scatole nere”: l'unica cosa che l'algoritmo di ricerca può fare è generare successori, testare l'obiettivo, eventualmente stimare una distanza tramite euristica. Nei CSP si apre la scatola: **lo stato ha una struttura interna esplicita** (un insieme di variabili con valori), e i vincoli fra le variabili sono rappresentati in modo dichiarativo. Questo permette di costruire algoritmi generali di risoluzione e tecniche di potatura molto più efficaci della ricerca cieca.

Esempi di ambiti in cui i vincoli sono centrali: design di oggetti, allocazione di risorse, progettazione di circuiti, pianificazione di percorsi robotici.

4.1 Definizione formale di CSP

Un **constraint satisfaction problem (CSP)** è definito da: un insieme di **variabili** $X_1, \dots, X_n$; un insieme di **domini** $D_1, \dots, D_n$; un insieme di **vincoli** $C_1, \dots, C_m$ che restringono le combinazioni di valori ammissibili; opzionalmente, una **funzione obiettivo** da massimizzare/minimizzare.

Stati e assegnamenti

Gli **stati** di un CSP corrispondono a tutti i possibili **assegnamenti** di valori alle variabili. Un assegnamento è detto: **completo** se assegna un valore a tutte le variabili; **consistente** se non viola alcun vincolo; **soluzione** se è completo e consistente.

Esempio guida: colorazione della mappa dell'Australia

Colorare i 7 territori dell'Australia (WA, NT, SA, Q, NSW, V, T) con i colori $\{R, G, B\}$ in modo che due territori confinanti non abbiano lo stesso colore.

Variabili: WA, NT, SA, Q, NSW, V, T. **Domini:** uguali per tutte, $D = \{R, G, B\}$. **Vincoli** (binari): $WA \neq NT$, $WA \neq SA$, $NT \neq SA$, $NT \neq Q$, $SA \neq Q$, $SA \neq NSW$, $SA \neq V$, $Q \neq NSW$, $NSW \neq V$ (la Tasmania T è un'isola: non confina con nessuno).

Una soluzione completa è $\{WA=R, NT=G, SA=B, Q=R, NSW=G, V=R, T=B\}$: qualunque permutazione dei tre colori applicata a una soluzione è ancora una soluzione (simmetria del problema).

I vincoli binari si rappresentano naturalmente come archi di un **grafo di vincoli**, i cui nodi sono le variabili: questa rappresentazione è centrale per gli algoritmi di propagazione (arc consistency, AC-3).

4.2 Tipi di domini e di variabili

Domini finiti: è possibile enumerare esplicitamente i vincoli. Il caso particolare in cui il dominio è $\{\text{vero}, \text{falso}\}$ dà luogo a CSP **booleani**, la cui decisione è NP-completa (es. 3-SAT).

Domini infiniti (discreti): non si possono enumerare i vincoli; si usano linguaggi di specifica (es. $\text{Lavoro1} + 5 < \text{Lavoro2}$). **Domini continui:** se i vincoli sono disuguaglianze lineari che definiscono una regione convessa, il CSP si risolve con la **programmazione lineare** in tempo polinomiale.

4.3 Arità dei vincoli

Vincoli unari: coinvolgono una sola variabile (es. $T \neq G$). **Vincoli binari:** coinvolgono due variabili; sono gli archi del grafo di vincoli (es. $WA \neq NT$). **Vincoli n-ari:** coinvolgono un numero qualsiasi di variabili; si rappresentano con **ipergrafi** (es. $\text{diverse}(WA, NT, SA)$). Spesso possono essere riscritti come insieme equivalente di vincoli binari.

Esempio classico di vincolo n-ario: criptoaritmetica

$$\begin{array}{r} \text{SEND} \\ + \text{MORE} \\ \hline \text{MONEY} \end{array}$$

Ogni lettera rappresenta una cifra diversa. Dominio di S e M: $\{1, \dots, 9\}$; dominio delle altre lettere: $\{0, \dots, 9\}$. Vincoli: “a lettere diverse corrispondono valori diversi” (alldifferent) più il vincolo aritmetico globale, non scomponibile in vincoli binari:

$$1000S + 100E + 10N + D + 1000M + 100O + 10R + E = 10000M + 1000O + 100N + 10E + Y.$$

Vincoli vs. criteri di preferenza

Una **soluzione** deve soddisfare **tutti** i vincoli veri e propri; una soluzione può **violare** uno o più **criteri di preferenza** (soft constraints), che permettono di ordinare le soluzioni ammissibili.

4.4 CSP come problema di ricerca nello spazio degli stati

Stato iniziale = assegnamento vuoto $\{\}$; **funzione successore** = assegna un valore a una delle variabili non ancora assegnate; **test obiettivo** = l'assegnamento corrente è completo; **costo di cammino** = costante per ogni passo. Poiché ogni passo valorizza esattamente una variabile precedentemente non assegnata, **la profondità massima dell'albero di ricerca è n .**

Perché l'ordine di assegnamento non conta: commutatività

Uno stato è un assegnamento di valori $\{X_1=v_1, \dots, X_n=v_n\}$, e l'ordine con cui i valori sono stati assegnati è **irrilevante ai fini del risultato**: si dice che i **CSP sono commutativi**. Questo permette agli algoritmi di scegliere sempre prima una variabile e poi un valore per essa.

4.5 Rappresentare gli stati: generate-and-test vs. ricerca con back-tracking

Approccio a forza bruta: generate-and-test

Usa solo informazioni di stato, ignorando i vincoli durante la generazione: genera un assegnamento completo, controlla la consistenza, ripete se necessario. Può richiedere di esplorare **l'intero spazio degli assegnamenti completi**.

Esempio, 8 regine: con variabili $Q_1..Q_8$ = posizione (1..64) $\rightarrow d^n = 64^8 \approx 2.81 \times 10^{14}$ assegnamenti; con variabili $Q_1..Q_8$ = riga della regina nella colonna i , dominio $\{1..8\} \rightarrow 8^8 =$

16 777 216 assegnamenti. Con 16 regine si arriverebbe a $16^{16} \approx 1.8 \times 10^{19}$ configurazioni, stimate in ~ 1200 anni di computazione.

Ricerca in profondità con backtracking

Si esplora lo spazio degli assegnamenti con una **depth-first search**, usando i vincoli per **potare** appena si genera un conflitto. Senza sfruttare la commutatività, al primo livello il fattore di ramificazione sarebbe $n \cdot d$, con $n! \cdot d^n$ foglie possibili (es. Australia: $7! \cdot 3^7 = 11\,022\,480$). Sfruttando la commutatività, il branching scende a d per livello, foglie d^n (Australia: $3^7 = 2187$).

Pseudocodice del backtracking

```
funzione BACKTRACKING-SEARCH(csp) restituisce soluzione o fallimento
restituisce BACKTRACK({}, csp)

funzione BACKTRACK(assegnamento, csp) restituisce soluzione o fallimento
se assegnamento è completo allora restituisci assegnamento
var ← SELECT-UNASSIGNED-VARIABLE(csp)
per ogni valore in ORDER-DOMAIN-VALUES(var, assegnamento, csp) fai
se valore è consistente con assegnamento allora
aggiungi {var = valore} ad assegnamento
inferenze ← INFERENCE(csp, var, valore)
se inferenze ≠ fallimento allora
aggiungi inferenze ad assegnamento
risultato ← BACKTRACK(assegnamento, csp)
se risultato ≠ fallimento allora restituisci risultato
rimuovi {var = valore} e le inferenze da assegnamento
restituisce fallimento
```

Limiti del backtracking semplice: il thrashing

Il backtracking di base è soggetto a **thrashing**: lo stesso errore può ripetersi identico in più punti diversi dell'albero di ricerca, perché l'algoritmo non "ricorda" perché ha fallito. Per migliorare servono risposte a tre domande: (1) l'assegnamento ha impatto sulle variabili non ancora assegnate? (inferenza/propagazione); (2) l'ordinamento dei valori influisce? (euristica del valore); (3) è possibile evitare fallimenti simili in futuro? (backjumping).

4.6 Euristiche per la scelta della variabile

Minimum Remaining Values (MRV, o euristica "fail-first")

Sceglie la variabile con il **minor numero di valori legali rimasti**. L'idea è "fallire il prima possibile".

Degree heuristic (euristica di grado)

MRV non aiuta sempre (es. nella scelta della prima variabile, o in caso di pareggio). Si usa allora l'**euristica di grado**: si sceglie la variabile **coinvolta nel maggior numero di vincoli** con le altre variabili non ancora assegnate. Le due euristiche si usano in combinazione: MRV come criterio principale, degree heuristic come tie-breaker.

4.7 Euristica per la scelta del valore: Least Constraining Value (LCV)

L'euristica del **valore meno vincolante** predilige il valore che **lascia più libertà** (più opzioni residue) alle variabili adiacenti nel grafo dei vincoli.

Domanda d'esame. Perché per la scelta della variabile si usa “la più vincolata” (MRV) mentre per la scelta del valore si usa “il meno vincolante” (LCV)? Non è contraddittorio? — Non è contraddittorio perché i due criteri rispondono a obiettivi diversi. Scegliere la variabile più vincolata serve a **individuare il prima possibile i vicoli ciechi** (fail-first). Scegliere invece, per quella variabile, il valore meno vincolante serve a **massimizzare la probabilità di successo** del ramo, lasciando il maggior numero possibile di opzioni aperte. In sintesi: si sceglie “dove rischiare di fallire subito” (variabile) e poi “come minimizzare il rischio di fallire” (valore).

4.8 Rendere informata la ricerca: propagazione dei vincoli (inferenza)

Il passo successivo è alternare fasi di **esplorazione** a fasi di **inferenza**: dopo ogni assegnamento si propagano le conseguenze sui domini delle altre variabili, eliminando valori che non potranno mai portare a una soluzione consistente. Le proprietà di consistenza locale principali, in ordine crescente di forza (e di costo): (1) **Forward checking**, (2) **Node consistency** (vincoli unari), (3) **Arc consistency** (vincoli binari), (4) **Path consistency** (triplette di variabili).

Forward checking

Quando si assegna un valore a una variabile, si eliminano subito dai domini dei vicini diretti i valori ormai incompatibili.

Esempio numerico (Australia): assegnamenti $WA=R$, poi $Q=G$, poi $V=B$:

	WA	NT	Q	NSW	V	SA	T
INIZIO	RGB	RGB	RGB	RGB	RGB	RGB	RGB
$WA=R \rightarrow$	R	GB	RGB	RGB	RGB	GB	RGB
$Q=G \rightarrow$	R	B	G	RB	RGB	B	RGB
$V=B \rightarrow$	R	B	G	R	B	$\emptyset$	RGB

Quando SA arriva a dominio vuoto, si ha fallimento immediato e si fa backtracking.

Limiti del forward checking

Non cattura tutte le inconsistenze: propaga solo dal nodo appena assegnato verso i suoi vicini diretti, senza controllare le conseguenze fra coppie di vicini fra loro. Esempio: dopo $WA=R$ e $Q=G$, i domini residui di NT e SA restano entrambi $\{B\}$: questo forza un'inconsistenza (NT e SA confinanti dovrebbero avere lo stesso colore), ma il forward checking non se ne accorge.

Node consistency

Una variabile è node consistent quando rispetta tutti i vincoli unari che la riguardano.

Arc consistency

Un grafo di vincoli è **arc consistent** quando rispetta tutti i vincoli binari. Un **arco** $X \rightarrow Y$ è **consistente** quando: per ogni valore possibile di X esiste **almeno un** valore di Y ad esso consistente. Quando un arco non è consistente, si possono eliminare valori dal dominio della variabile sorgente.

Algoritmo AC-3

Sviluppato nel 1977 da Alan Mackworth. Si può usare come **preprocessing**, oppure intrecciato con il backtracking (algoritmo **MAC**, Maintaining Arc Consistency).

```
funzione AC-3(csp) restituisce false se un dominio è svuotato, true altrimenti
queue ← insieme di tutti gli archi  $(X_i, X_j)$  del csp
finché queue non è vuota:
    rimuovi un arco  $(X_i, X_j)$  da queue
    se REVISE(csp,  $X_i, X_j$ ) allora
        se  $D_i$  è vuoto allora restituisci false
    per ogni  $X_k$  vicino di  $X_i$ ,  $X_k \neq X_j$ : aggiungi  $(X_k, X_i)$  a queue
    restituisci true

funzione REVISE(csp,  $X_i, X_j$ ) restituisce true se il dominio di  $X_i$  è stato
modificato
revised ← false
per ogni  $x$  in  $D_i$ :
    se non esiste alcun valore  $y$  in  $D_j$  tale che  $(x, y)$  soddisfa il vincolo:
        elimina  $x$  da  $D_i$ ; revised ← true
    restituisci revised
```

Costo computazionale di AC-3

Un CSP con n variabili e vincoli binari ha al più n^2 archi. Sia d il numero massimo di valori nel dominio. Il tempo nel caso peggiore è $O(n^2 d^3)$. **AC-3 è incompleto**: esistono assegnamenti globalmente inconsistenti che l'arc consistency da sola non rileva.

Incompletezza di AC-3

Esempio: un grafo a 3 nodi tutti collegati a due a due (triangolo), con dominio {bianco, nero} e vincolo “colori diversi” fra ogni coppia. Il grafo è **arc consistent**, **eppure non esiste soluzione**, perché con solo 2 colori non si può colorare un triangolo (3-clique) con tutte le coppie di colori diversi.

Path consistency

Proprietà **più forte** dell'arc consistency: una coppia di variabili $\{X_1, X_2\}$ è path consistent rispetto a X_3 quando per ogni assegnamento $\{X_1=a, X_2=b\}$ consistente esiste un valore di X_3 compatibile con entrambi. L'algoritmo **PC-2** (Mackworth) propaga la path consistency in modo analogo ad AC-3, con complessità maggiore.

Generalizzazione: k-consistency

Un CSP è **k-consistent** quando: per ogni sottoinsieme di $k - 1$ variabili e per ogni loro assegnamento consistente, è sempre possibile trovare un assegnamento consistente per una qualunque k -esima variabile. 1-consistency = node consistency; 2-consistency = arc consistency; 3-consistency = path consistency.

Risultato teorico: un CSP fortemente k -consistent (con k pari al numero di variabili n) può essere risolto **senza mai fare backtracking**, in tempo $O(nd)$ — ma decidere se un CSP è fortemente k -consistent ha esso stesso costo esponenziale.

4.9 Vincoli speciali (globali)

Alldifferent $(X_1, \dots, X_n)$

Impone che tutte le variabili elencate assumano valori tutti diversi fra loro. Se le n variabili possono assumere complessivamente meno di n valori distinti, il vincolo non può mai essere soddisfatto (pigeonhole principle).

Atmost $(N, A_1, \dots, A_k)$

Vincolo di risorse: le attività $A_1, \dots, A_k$ possono complessivamente impegnare **al più** N risorse.

4.10 Oltre il backtracking cronologico: backjumping

Il limite del backtracking cronologico

Quando il backtracking standard raggiunge un vicolo cieco, torna indietro **sempre e solo** alla variabile assegnata al passo immediatamente precedente, indipendentemente dal fatto che quella variabile abbia o meno causato il conflitto.

Esempio: assegnamento corrente $\{Q=R, NSW=G, V=B, T=R\}$. Si sceglie SA: tutti i valori vanno in conflitto (Q, NSW, V, tutti confinanti con SA, coprono l'intero spettro dei colori). Il backtracking cronologico tornerebbe indietro su T — ma T non ha nessun ruolo nel conflitto.

Backjumping: backtracking non cronologico

Ogni variabile mantiene un **conflict set**: l'insieme degli assegnamenti che hanno contribuito a creare il conflitto. Nell'esempio, il conflict set di SA è $\{Q=R, NSW=G, V=B\}$. Il **backjumping** usa il conflict set per saltare direttamente alla variabile realmente responsabile (nell'esempio: V).

Relazione fra backjumping e forward checking: il forward checking, se arricchito per costruire i conflict set, rileva **esattamente gli stessi conflitti** che rilevarebbe il backjumping puro. Quindi **usare backjumping insieme a forward checking è ridondante**.

4.11 Assegnamenti NOGOOD e conflict-directed backjumping

NOGOOD

Un **assegnamento NOGOOD** in un CSP è un **assegnamento parziale** che, pur non essendo ancora un vicolo cieco immediato, **non può mai essere esteso a una soluzione** — lo si scopre solo esplorando ulteriormente. Esempio: $\{WA=R, NSW=R\}$ è NOGOOD.

Conflict-directed backjumping

Aggiorna dinamicamente i conflict set mano a mano che si esplora. Regola di aggiornamento: sia X_j la variabile corrente su cui tutti i valori falliscono. Si fa backjump alla variabile X_i aggiunta più di recente a $\text{conf}(X_j)$, aggiornando:

$$\text{conf}(X_i) \leftarrow \text{conf}(X_i) \cup \text{conf}(X_j) - \{X_i\}.$$

Esempio numerico completo (Australia): assegnando $WA=R$, $NSW=R$, $T=B$, $NT=B$, $Q=G$, si arriva a dover assegnare SA , ma nessun valore funziona. I conflict set costruiti via FC:

$$\text{conf}(NT) = \{WA\}, \quad \text{conf}(Q) = \{NSW, NT\}, \quad \text{conf}(SA) = \{WA, NSW, NT, Q\}.$$

Backjump a Q : $\text{conf}(Q) \leftarrow \{WA, NSW, NT\}$ (WA non confina con Q , ma contribuisce indirettamente). Falliscono anche i valori di Q . Backjump a NT : $\text{conf}(NT) \leftarrow \{WA, NSW\}$. Falliscono anche i valori di NT . Backjump a NSW : $\text{conf}(NSW) \leftarrow \{WA\}$ — si scopre la relazione fra NSW e WA . Si prova $NSW=G$: si riesce a costruire una soluzione completa.

Registrare gli stati NOGOOD minimali permette all’algoritmo di evitare di ripetere in futuro gli stessi assegnamenti falliti: è una **forma di apprendimento**.

4.12 Applicazioni ed esempi aggiuntivi

Sudoku come CSP

Variabili: una per ogni casella x_{ij} . **Dominio:** $\{1, \dots, 9\}$. **Vincoli** (tutti alldifferent): per ogni riga, per ogni colonna, per ogni riquadro 3×3 .

N-regine su larga scala

Grazie a tecniche avanzate (local search, min-conflicts), alcuni algoritmi moderni risolvono il problema delle N regine con $N = 6\,000\,000$.

Applicazioni reali di tipo ricerca operativa

Location of facilities, job scheduling, car sequencing, cutting stock problem, vehicle routing, timetabling, rostering/crew scheduling (es. nel 1998 CSP realizzati in ECLiPSe sono stati usati per definire gli equipaggi dei treni delle Ferrovie dello Stato italiane).

Software e strumenti per CSP

Linguaggi di programmazione a vincoli (spesso moduli di Prolog); **ECLiPSe**; **CHR** (Constraint Handling Rules); **SAT solver** (basati sull’algoritmo **DPLL**, es. MiniSat); **GECODE**; **Answer Set Programming (ASP)**.

Domanda d’esame. Qual è la differenza pratica fra forward checking, arc consistency (AC-3) e path consistency, e quando conviene usare l’una o l’altra? — Le tre tecniche formano una gerarchia crescente di potenza (e di costo): il **forward checking** propaga solo dalla variabile appena assegnata ai suoi vicini diretti; è economico e si integra con MRV, ma non rileva inconsistenze fra due vicini non ancora assegnati. L’**arc consistency** (AC-3) propaga sistematicamente su tutti gli archi del grafo: costa $O(n^2 d^3)$ contro il costo minore del forward checking, ma è più efficace; resta comunque **incompleta**. La **path consistency** è più forte ancora, perché ragiona su triplette di variabili, ma il suo algoritmo (PC-2) ha complessità maggiore. In pratica: il forward checking si usa sempre “in linea” per il suo basso costo; l’arc consistency si usa quando serve una potatura più efficace; la path consistency (e la k -consistency in generale) restano soprattutto di interesse teorico.

Riepilogo e punti chiave — Modulo 4

- Un **CSP** è definito da variabili, domini e vincoli; una soluzione è un assegnamento **completo e consistente**. Grazie alla **commutatività**, si può sempre fissare prima la variabile e poi il valore, riducendo il branching da $n \cdot d$ a d per livello.

- Il **generate-and-test** è impraticabile; il **backtracking** è già molto più efficiente, ma soffre di **thrashing** senza euristiche/inferenza.
- **MRV** (fail-first) + **degree heuristic** per la variabile; **LCV** per il valore.
- **Consistenza locale** crescente: forward checking $\rightarrow$ node consistency $\rightarrow$ arc consistency/AC-3 ($O(n^2d^3)$, incompleta) $\rightarrow$ path consistency/PC-2. **k-consistency**: fortemente n -consistent si risolve senza backtracking, ma verificarlo è esponenziale.
- **Vincoli globali** (Alldifferent, Atmost) hanno algoritmi di propagazione dedicati.
- Il **backjumping** usa i **conflict set** per saltare alla variabile realmente responsabile; il **conflict-directed backjumping** gestisce gli **assegnamenti NOGOOD** — una forma di apprendimento.

Capitolo 5

Rappresentazione della conoscenza e Logica proposizionale

5.1 Perché rappresentare la conoscenza

Rappresentare la conoscenza significa dotarsi di un **linguaggio formale** in cui esprimere ciò che si sa del mondo, in modo tale da poter applicare a quella conoscenza dei **processi di ragionamento automatici (inferenze)**. Lo scopo dell'inferenza è duplice: derivare **nuova conoscenza**, cioè informazioni che erano implicite e diventano esplicite; e **prendere decisioni**.

Cosa vuol dire “ragionare”

Ragionare significa **rendere esplicita conoscenza che era implicita**. Esempio classico: so che *tutti i cetacei vivono in mare* e che *tutte le balene sono cetacei*. Posso concludere che *tutte le balene vivono in mare*, senza aver verificato personalmente ogni singola balena. Il punto cruciale è che questo tipo di ragionamento **lavora sulla forma** delle affermazioni, trattate come schemi astratti, indipendentemente dal contenuto specifico. Questo è ciò che rende il ragionamento **automatizzabile**: basta (1) un linguaggio, (2) delle regole di ragionamento, (3) un algoritmo che sappia applicare le regole.

Agenti basati sulla conoscenza

Un agente basato sulla conoscenza è caratterizzato da: **Knowledge Base (KB)**: insieme di formule nel linguaggio di rappresentazione, possedute dall'agente (la conoscenza iniziale è la **background knowledge**); **Tell (assert)**: meccanismo per aggiungere nuove formule; **Ask (query)**: meccanismo per interrogare la KB. Vincolo fondamentale: **ogni risposta a una ask deve essere conseguenza logica** delle tell fatte e della background knowledge.

Schema generale dell'agente (KB-Agent):

```
Agente ha: KB; Tempo = 0
function KB-Agent(percezione) returns azione:
1. tell(KB, costruisci-formulaP(percezione, tempo))
2. azione ← ask(KB, costruisci-interrogazioneA(tempo))
3. tell(KB, costruisci-formulaA(azione, tempo))
4. tempo ← tempo + 1
5. return azione
```

Una **versione 2** sostituisce la prima **tell** con **modify** (che può sia aggiungere sia **rimuovere** elementi) — necessario perché il mondo cambia, non solo si accumula.

Dato, informazione, conoscenza

Dato: il risultato grezzo della percezione sensoriale, privo di significato. **Informazione:** ciò che il dato rappresenta. **Conoscenza:** cattura le **relazioni** fra le informazioni (es. le regole per comporre lettere in parole e frasi).

Programmazione dichiarativa

Programmare un agente basato sulla conoscenza significa **specificare la KB** in forma dichiarativa: si dice *cosa* è vero/cosa fare, non *come* calcolarlo. Esempi di programmazione dichiarativa: XML, una query SQL, un'espressione regolare.

Esempio: il mondo dei blocchi

Predicati: $\text{holding}(x)$, handempty , $\text{clear}(x)$, $\text{ontable}(x)$, $\text{on}(x, y)$. Le azioni sono descritte con **precondizioni** ed **effetti** (add/delete), ad esempio l'azione $\text{pick}(x, y)$:

Preconditions: $\text{on}(x, y) \wedge \text{clear}(x) \wedge \text{handempty}$

Effects: $\text{add}(\text{clear}(y)), \text{add}(\text{holding}(x)), \text{delete}(\text{on}(x, y)), \text{delete}(\text{clear}(x)), \text{delete}(\text{handempty})$

5.2 Rappresentare la conoscenza tramite la logica

Un **linguaggio di rappresentazione** è lo strumento che permette di codificare la conoscenza in una forma su cui si può applicare un ragionamento automatico. I concetti chiave sono: **modello**, **conseguenza**, **inferenza**, **algoritmo di inferenza**, **grounding**.

Modello

Un **modello** è un “mondo possibile”: fissa il valore di verità di tutte le formule assegnando valori agli elementi da cui quei valori dipendono. Dati un modello m e una formula α , m è **un modello di α se α è vera in m** . Si indica con $M(\alpha)$ l'insieme di tutti i modelli di α .

Conseguenza logica (entailment, $\models$)

$A \models B$ (“ A implica logicamente B ”) significa: **in tutti i modelli in cui A è vera, è vera anche B** . Esempio: $(x + y = 4) \models (x + y < 5)$.

Attenzione alla **direzionalità**: $A \not\models B$ significa solo che *esiste almeno un modello* in cui A è vera e B è falsa.

Visione insiemistica: $A \models B$ corrisponde all'inclusione insiemistica $M(A) \subseteq M(B)$.

Equivalenza

$A \equiv B$ se e solo se $A \models B$ e $B \models A$, cioè le due formule sono vere esattamente negli stessi modelli: $M(A) = M(B)$.

Validità, insoddisfacibilità, soddisfacibilità

Validità (tautologia): una formula P è **valida** se è vera in **tutti** i modelli ($P \equiv \text{True}$). Esempio: $Q \vee \neg Q$. **Insoddisfacibilità (contraddizione):** P è **insoddisfacibile** se è **falsa** in tutti i modelli ($P \equiv \text{False}$). Esempio: $Q \wedge \neg Q$. **Soddisfacibilità:** P è **soddisfacibile** se esiste **almeno un** modello in cui è vera.

Relazione importante: P è **valida se e solo se $\neg P$ è insoddisfacibile**.

Inferenza

Inferenza: processo con cui, da una proposizione accettata come vera, si passa a una seconda proposizione la cui verità è derivata dalla prima. Punto essenziale: **l'inferenza è sintattica**, lavora sulla struttura delle formule, non sul loro “significato”.

Regole di inferenza principali: **Modus Ponens** (da $A \Rightarrow B$ e A si deriva B); **eliminazione della congiunzione** (da $A \wedge B$ si deriva B).

Formalizzazione: sia i un algoritmo di inferenza. $KB \vdash_i A$ significa che A è derivabile da KB usando l'algoritmo i .

Proprietà desiderate di un algoritmo di inferenza: **Correttezza (soundness):** l'algoritmo deriva *solo* conseguenze logiche vere — se $KB \vdash_i A$ allora $KB \models A$. **Completezza (completeness):** l'algoritmo è in grado di derivare *tutte* le conseguenze logiche — se $KB \models A$ allora $KB \vdash_i A$. Le logiche devono **sempre** garantire la correttezza; non sempre garantiscono la completezza.

Grounding

Il **grounding** cattura il legame fra la rappresentazione simbolica/formale e l'ambiente reale che essa rappresenta — possiamo pensarlo come derivante dalla percezione.

5.3 Logica proposizionale

È uno dei tipi di logica più semplici: **le formule non contengono variabili**.

Sintassi

Formule atomiche: i **simboli proposizionali** rappresentano ciascuno una formula elementare che può essere vera o falsa; True e False sono formule atomiche speciali.

Formule complesse, costruite componendo formule tramite operatori: **Negazione** ($\neg$): un **letterale** è una formula atomica, eventualmente negata; **Congiunzione** ($\wedge$); **Disgiunzione** ($\vee$); **Implicazione** ($\Rightarrow$); **Biimplicazione/equivalenza** ($\Leftrightarrow$).

Precedenza degli operatori (dal più forte al più debole): $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$.

Domanda d'esame. “Vero o falso? Si consideri la formula $\text{Piove} \wedge \text{Vento}$: è vera o falsa?” — La domanda è mal posta finché non si fissa un **modello**. Il valore di verità di una formula proposizionale non è assoluto: dipende dall'assegnamento di verità ai simboli atomici che la compongono. $\text{Piove} \wedge \text{Vento}$ è vera **solo** nel modello in cui sia Piove sia Vento sono vere (T,T); è falsa in tutti gli altri tre casi.

Semantica

Un **modello**, in logica proposizionale, è un **assegnamento di un valore di verità a ciascun simbolo proposizionale**. Con N simboli proposizionali ci sono 2^N modelli possibili.

Formule complesse (P, Q formule qualsiasi): $\neg Q$ è vera sse Q è falsa; $Q \wedge P$ è vera sse sia P sia Q sono vere; $Q \vee P$ è vera sse almeno una fra P e Q è vera; $Q \Rightarrow P$ è sempre vera, tranne quando Q è vera e P è falsa; $Q \Leftrightarrow P$ è vera sse P e Q hanno lo stesso valore di verità.

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$
F	F	T	F	F	T
F	T	T	F	T	T
T	F	F	F	T	F
T	T	F	T	T	T

L'implicazione: come interpretarla

$P \Rightarrow Q$ è **equivalente a** $\neg P \vee Q$. Se P è **falsa**, l'implicazione è comunque **vera** (vacuamente soddisfatta); se P è **vera**, allora Q deve **necessariamente** essere vera.

Attenzione: l'implicazione logica **non è una relazione causale**. È vera o falsa solo in base ai valori di verità di P e Q , indipendentemente dal legame di significato/causa.

Domanda d'esame. “È vera l'implicazione Torino è in Lombardia $\Rightarrow$ Giulio Cesare governò Roma?”
— Sì, è **vera**. ‘Torino è in Lombardia’ è **falsa** (Torino è in Piemonte), e quando l'antecedente è falso $P \Rightarrow Q$ è vera **indipendentemente** dal valore di verità di Q .

Verificare l'entailment: $KB \models P$?

Model checking: enumerare tutti i possibili modelli (2^N , esponenziale). **Theorem proving:** usare regole di inferenza per cercare una derivazione senza costruire esplicitamente i modelli.

Teorema di deduzione: date due formule R e Q , ($R \models Q$) se e solo se ($R \Rightarrow Q$) è valida.

Dimostrazione per refutazione: sfruttando che $(R \Rightarrow Q) \equiv (\neg R \vee Q)$, negando si ottiene $\neg(\neg R \vee Q) \equiv (R \wedge \neg Q)$ (De Morgan). Si arriva quindi a: date R e Q , ($R \models Q$) se e solo se $(R \wedge \neg Q)$ è insoddisfacibile. Per verificare $KB \models P$: (1) si assume per assurdo $\neg P$; (2) si dimostra che $KB \wedge \neg P$ è insoddisfacibile, cioè che se ne deriva **False**; (3) la ricerca della dimostrazione è formalmente analoga a una ricerca in uno spazio degli stati.

Questo ragionamento si applica a **logiche monotone**: se $KB \models P$ allora anche $KB \wedge Q \models P$.

Regole di inferenza (equivalenze logiche)

Equivalenza	Nome
$(\alpha \vee \beta) \equiv (\beta \vee \alpha)$	Commutatività $\vee$
$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha)$	Commutatività $\wedge$
$\neg(\neg\alpha) \equiv \alpha$	Eliminazione doppia negazione
$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha)$	Contrapposizione
$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta)$	Eliminazione implicazione
$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta)$	De Morgan
$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta)$	De Morgan
$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma))$	Distributività
$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha))$	Eliminazione bicondizionale

Se si usa solo un sottoinsieme delle regole, si rischia di perdere completezza: ad esempio, senza la regola del doppio negato non si può derivare $P \vdash \neg\neg P$.

La regola di risoluzione (Resolution)

La **risoluzione** è una singola regola di inferenza che, combinata con un algoritmo di ricerca completo, produce un algoritmo di inferenza **corretto e completo**. Si applica a **clausole**, cioè disgiunzioni di letterali. Date due clausole che contengono un **letterale complementare** P_i e Q_j :

$$\frac{(P_1 \vee \dots \vee P_i \vee \dots \vee P_n) \quad (Q_1 \vee \dots \vee Q_j \vee \dots \vee Q_m)}{(P_1 \vee \dots \vee P_{i-1} \vee P_{i+1} \vee \dots \vee P_n \vee Q_1 \vee \dots \vee Q_{j-1} \vee Q_{j+1} \vee \dots \vee Q_m)}$$

La clausola risultante (**resolvent**) contiene tutti i letterali delle due clausole originali **tranne** la coppia complementare; se un letterale compare più volte viene **fattorizzato**.

Esempio: da $A \vee B \vee \neg C$ e $C \vee A$, risolvendo su $C/\neg C$ si ottiene $A \vee B \vee A$, che fattorizzato diventa $A \vee B$.

Relazione con il modus ponens: il modus ponens è un **caso particolare** di risoluzione. Da C e $C \Rightarrow A$ (cioè $\neg C \vee A$), risolvendo su $C/\neg C$ si ottiene A .

Forma Normale Congiuntiva (CNF)

CNF: data una qualunque formula proposizionale, esiste sempre una **congiunzione di clausole** ad essa equivalente.

Algoritmo di traduzione in CNF (4 passi): (1) **Eliminare la biimplicazione:** $(\alpha \Leftrightarrow \beta) \rightarrow ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha))$. (2) **Eliminare l'implicazione:** $(\alpha \Rightarrow \beta) \rightarrow (\neg \alpha \vee \beta)$. (3) **Portare la negazione all'interno** (De Morgan, doppie negazioni). (4) **Distribuire l' $\vee$ sull' $\wedge$.**

Algoritmo di risoluzione proposizionale

```
function CP-RISOLUZIONE(KB, A) returns true | false
  clausole  $\leftarrow$  clausole della CNF di  $(KB \wedge \neg A)$ 
  new  $\leftarrow \{\}$ 
  loop do:
    for each coppia  $C_i, C_j$  in clausole do:
      resolvents  $\leftarrow$  IR-RISOLUZIONE( $C_i, C_j$ )
      if resolvents contiene la clausola vuota: return true
    new  $\leftarrow$  new  $\cup$  resolvents
    if new  $\subseteq$  clausole: return false
  clausole  $\leftarrow$  clausole  $\cup$  new
```

Il funzionamento è quello di una **dimostrazione per refutazione**: si nega la query, la si aggiunge alla KB in CNF, e si cerca di generare la **clausola vuota**.

Teorema di completezza della risoluzione: se un insieme di clausole è insoddisfacibile, la **chiusura della risoluzione** contiene la clausola vuota.

Esempio applicativo: pioggia, atmosfera, strada

Data una KB su pioggia/atmosfera/strada con fatti Sole e Vento, si vuole verificare $KB \models \neg \text{Piove}$. Negando il goal si ottiene Piove, e si deriva a catena: Piove risolve con $\neg \text{Piove} \vee \text{Atmosfera_umida} \rightarrow \text{Atmosfera_umida}$; questa risolve con $\neg \text{Atmosfera_asciutta} \vee \neg \text{Atmosfera_umida} \rightarrow \neg \text{Atmosfera_asciutta}$; questa risolve con $\neg \text{Sole} \vee \neg \text{Vento} \vee \text{Atmosfera_asciutta} \rightarrow \neg \text{Sole} \vee \neg \text{Vento}$; risolve con $\text{Sole} \rightarrow \neg \text{Vento}$; risolve con $\text{Vento} \rightarrow$ **clausola vuota**. Confermato: $KB \models \neg \text{Piove}$.

5.4 Clausole di Horn, Forward Chaining, Backward Chaining

Clausole di Horn

Una clausola di Horn è una disgiunzione di letterali in cui **al più uno** è positivo. Se contiene **esattamente un** letterale positivo, si dice **clausola definita**. Catturano implicazioni: $\neg B \vee C$ equivale a $B \Rightarrow C$; $\neg A \vee \neg B \vee C$ equivale a $A \wedge B \Rightarrow C$. Sono la **base della programmazione logica** (es. Prolog).

Perché sono importanti: permettono di verificare la conseguenza logica in **tempo lineare** rispetto alla dimensione della KB.

Forward Chaining (concatenazione in avanti)

Permette di derivare una query costituita da un **singolo simbolo proposizionale**, a partire da una KB di sole clausole di Horn. È un procedimento iterativo, **guidato dai dati (data-driven)**: (1) si parte dai fatti noti; (2) si applica **modus ponens**; (3) ogni volta che tutte le premesse di

un'implicazione risultano vere, si aggiunge il letterale conseguente ai fatti noti; (4) terminazione: query derivata $\rightarrow \text{true}$; nessun'altra inferenza possibile $\rightarrow \text{false}$. Ha **complessità lineare**.

Osservazioni: complessità lineare; **completo**; **“inconscio” (data-driven)**: guidato solo dai dati disponibili; adatto a problemi come il riconoscimento di oggetti; svantaggio: può attivare molte inferenze inutili.

Backward Chaining (concatenazione all'indietro)

Parte dalla formula da dimostrare (il **goal**): se già vero, termina con **true**; altrimenti cerca clausole di Horn di cui il goal è la conclusione, e cerca ricorsivamente di dimostrarne le premesse. È quindi un ragionamento **goal-driven**.

Osservazioni: ragionamento guidato dagli obiettivi; usato nel theorem proving e nella programmazione logica; spesso più efficiente del forward chaining perché il goal focalizza la ricerca; complessità meno che lineare in pratica.

Forward vs Backward chaining: confronto

Aspetto	Forward Chaining	Backward Chaining
Direzione	dai fatti verso le conclusioni	dal goal verso le premesse
Guida	data-driven	goal-driven
Completezza	completo	dimostra solo ciò che serve
Efficienza	lineare, inferenze inutili	spesso più efficiente
Uso tipico	riconoscimento pattern	theorem proving, Prolog

Riepilogo e punti chiave — Modulo 5

- Un agente basato sulla conoscenza è definito da **KB**, **tell** e **ask**, con il vincolo che ogni risposta sia conseguenza logica.
- **Modello**, **conseguenza logica** $\models$, **equivalenza** $\equiv$, **validità**, **insoddisfacibilità**, **soddisfacibilità**, **inferenza** sono i concetti fondamentali.
- Un buon algoritmo di inferenza deve essere **corretto**, idealmente **completo**.
- L'**implicazione** $P \Rightarrow Q \equiv \neg P \vee Q$ **non è causale**.
- Per $KB \models P$: model checking (esponenziale) o **theorem proving** via **refutazione**: si nega il goal e si dimostra insoddisfacibilità.
- La **CNF** è il prerequisito per la **risoluzione**, che combinata con ricerca è corretta e completa. Il **modus ponens** è un caso particolare di risoluzione.
- Le **clausole di Horn** permettono inferenza in tempo lineare: **forward chaining** (data-driven, completo) e **backward chaining** (goal-driven, più efficiente, usato in Prolog).

Capitolo 6

Logica del primo ordine (FOL)

“Rappresentiamo mondi complessi in cui gli oggetti sono in relazione gli uni con gli altri e studiamo come ragionare su queste rappresentazioni più espressive.” (C. Baroglio)

6.1 Perché non basta la logica proposizionale

Pregi della logica proposizionale (da mantenere)

La logica proposizionale ha tre proprietà preziose che **vogliamo conservare**: **È dichiarativa**: separa nettamente la conoscenza dall’inferenza. **È composizionale**: il valore di verità di una formula si ottiene componendo i valori di verità delle sue parti. **Non è ambigua**: ogni formula ha un significato preciso, indipendente dal contesto.

I limiti espressivi

Il problema è la **mancanza di espressività**: (1) **Non permette rappresentazioni compatte**. Per dire che “una persona quando è felice ascolta musica” dobbiamo scrivere una proposizione *per ogni singola persona* (es. $\text{ascoltaMusica}(\text{mia}) \Leftarrow \text{felice}(\text{mia}), \dots$). (2) **Non permette di esprimere relazioni fra elementi** (es. $\text{padre}(x, y)$): in proposizionale ogni fatto è un simbolo atomico indivisibile.

6.2 Dalla logica proposizionale a FOL

Logica proposizionale: ogni fatto è vero o falso, punto. **Logica del prim’ordine (FOL)**: il mondo è fatto di **oggetti** in **relazione** fra loro.

Elementi sintattici di FOL

Simboli di costante: Richard, John, 2 — nominano oggetti specifici. **Simboli di predicato**: Fratello, $<$, $>$ — esprimono relazioni/proprietà. **Simboli di funzione**: $+$, Antenato — restituiscono un oggetto a partire da altri oggetti. **Simboli di variabile**: x, y, z . **Connettivi**: $\Rightarrow \Leftarrow \wedge \vee \neg$. **Uguaglianza**: $=$. **Quantificatori**: $\forall \exists$ (il vero elemento nuovo).

Predicati vs funzioni

Entrambi operano su tuple di oggetti del dominio, ma restituiscono cose diverse: **Predicati**: catturano una proprietà e restituiscono **vero/falso** (es. $\text{Uomo}(\text{Andrea})$). **Funzioni**: restituiscono **un oggetto** del dominio (es. $\text{più}(3, 5)$).

Modelli e interpretazione

Un **modello** in FOL è una coppia $M = (D, I)$: D (dominio del discorso): insieme di ≥ 1 oggetti e delle loro relazioni; I (interpretazione): associazione fra i simboli del linguaggio e gli elementi/relazioni del dominio.

Un punto sottile ma importante: il valore di verità di una formula dipende interamente dall'interpretazione scelta. Se **cambio coerentemente i nomi dei simboli** ma mantengo la stessa struttura, il valore di verità **non cambia**. Se invece **cambio l'interpretazione**, il valore di verità **può cambiare**.

In breve: **una formula FOL non ha un valore di verità assoluto**, lo ha solo relativamente a un modello (D, I) fissato.

Quanti modelli ha una KB FOL?

In proposizionale, con N simboli, ci sono 2^N modelli enumerabili. In FOL, i modelli possibili possono essere **infiniti**.

Domanda d'esame. Perché in FOL non si può verificare la conseguenza logica per enumerazione dei modelli? — Perché se il dominio D è illimitato e una formula contiene quantificatori, per stabilirne il valore di verità servirebbe calcolare il valore di verità di infinite istanze. I modelli possibili in FOL possono quindi essere infiniti, e l'enumerazione smette di essere praticabile. Occorrono altre tecniche di inferenza (regole “liftate”, risoluzione, ecc.).

6.3 Termini e formule atomiche

Un **termine** è un'espressione che si riferisce a un oggetto del dominio. Le **funzioni** permettono di riferirsi a oggetti che *non hanno un nome proprio*: una funzione **non costruisce** l'oggetto restituito, si limita a **riferirlo** (es. $\text{GambaSinistra}(\text{John})$).

Termine ground: un termine che non contiene variabili.

Formula atomica: $\text{predicato}(t_1, \dots, t_n)$ oppure $t_1 = t_2$. È vera quando, dato un modello, la relazione denotata dal predicato è verificata dagli oggetti denotati dai termini.

6.4 Quantificatori

I quantificatori permettono di esprimere proprietà su **collezioni** di oggetti.

Quantificatore universale $\forall$

$\forall x F$ è vera in M se e solo se F è vera per **qualsiasi** interpretazione di x in M . Esempio: “Al corso di sistemi intelligenti tutti sono intelligenti”:

$$\forall x \text{Partecipa}(x, \text{SISINT}) \Rightarrow \text{Intelligente}(x).$$

Concettualmente si può “espandere” in una **congiunzione** su tutti gli oggetti del dominio.

Quantificatore esistenziale $\exists$

$\exists x F$ è vera in M se e solo se F è vera per **almeno una** interpretazione di x in M . Esempio: “Al corso di sistemi intelligenti qualcuno è intelligente”:

$$\exists x \text{Partecipa}(x, \text{SISINT}) \wedge \text{Intelligente}(x).$$

Si espande in una **disgiunzione**.

Regola pratica: $\forall$ va con $\Rightarrow$, $\exists$ va con $\wedge$

Domanda d'esame. Che differenza c'è fra $\forall x \text{Partecipa}(x, \text{SISINT}) \Rightarrow \text{Intelligente}(x)$ e $\forall x \text{Partecipa}(x, \text{SISINT}) \wedge \text{Intelligente}(x)$? Sono equivalenti? — **NO.** (1) La prima significa “tutti quelli che partecipano a SISINT sono intelligenti” — l'implicazione filtra correttamente solo i partecipanti. (2) La seconda significa “tutti (ogni oggetto del dominio) partecipano a SISINT E sono intelligenti” — affermazione quasi certamente falsa. **Motivo strutturale:** con $\forall$ la formula è valutata per *ogni* oggetto del dominio; usare $\wedge$ costringe *ogni* oggetto a soddisfare entrambe le condizioni, mentre di solito si vuole restringere il discorso a una sottoclasse, e questo si ottiene con l'implicazione.

Domanda d'esame. E per $\exists x \text{Partecipa}(x, \text{SISINT}) \Rightarrow \text{Intelligente}(x)$ vs $\exists x \text{Partecipa}(x, \text{SISINT}) \wedge \text{Intelligente}(x)$? — **NO, e qui l'errore è ancora più insidioso.** (1) La prima, per la legge $A \Rightarrow B \equiv \neg A \vee B$, equivale a $\exists x \neg \text{Partecipa}(x, \text{SISINT}) \vee \text{Intelligente}(x)$: basta che esista **un solo oggetto qualsiasi** che *non* partecipa al corso per rendere vera la formula, indipendentemente da chi sia intelligente! (2) La seconda dice correttamente “esiste (almeno) un partecipante a SISINT che è intelligente”. **Conclusione pratica:** usare $\forall \dots \Rightarrow \dots$ per affermazioni universali su una sottoclasse, e $\exists \dots \wedge \dots$ per affermazioni esistenziali su una sottoclasse.

Quantificatori annidati: l'ordine conta

Regole di equivalenza per quantificatori dello stesso tipo (commutano liberamente): $\exists x \exists y F \equiv \exists y \exists x F$; $\forall x \forall y F \equiv \forall y \forall x F$.

Ma **quantificatori di tipo diverso NON commutano**: $\forall x \exists y F$ (“per ogni x esiste un y che può dipendere da x ”, es. “tutti amano qualcuno, potenzialmente diverso per ciascuno”); $\exists y \forall x F$ (“esiste un y fisso che vale per tutti gli x ”, es. “esiste qualcuno amato da tutti”).

Domanda d'esame. Perché $\forall x \exists y \text{Ama}(x, y)$ e $\exists y \forall x \text{Ama}(x, y)$ non sono equivalenti? — Nel primo caso ogni x può scegliere un y diverso; nel secondo caso esiste un **singolo** oggetto y , fissato una volta per tutte, che soddisfa la relazione con *ogni* x . La seconda formula è molto più forte e implica la prima, ma non viceversa. L'ordine di annidamento determina la “portata” (scope) delle dipendenze fra variabili: la variabile quantificata più esternamente è fissata prima. Questo concetto tornerà cruciale nella skolemizzazione.

Quantificatori e negazione (leggi di De Morgan generalizzate)

$\forall x \neg F \equiv \neg \exists x F$; $\exists x \neg F \equiv \neg \forall x F$; $\forall x F \equiv \neg \exists x \neg F$; $\exists x F \equiv \neg \forall x \neg F$. In pratica, $\forall$ e $\exists$ sono duali fra loro esattamente come $\wedge$ e $\vee$ lo sono rispetto alla negazione in proposizionale.

Uguaglianza e quantificatori

L'**uguaglianza** ($=$) riguarda esclusivamente **termini**, non formule. Problema tipico: come esprimere “John ha almeno due fratelli”? Il tentativo $\exists y, z \text{Fratello}(\text{John}, y) \wedge \text{Fratello}(\text{John}, z)$ **non basta**, perché è soddisfatta anche se y e z vengono unificati con lo stesso oggetto. La correzione richiede esplicitamente $\neg(y = z)$:

$$\exists y, z \text{Fratello}(\text{John}, y) \wedge \text{Fratello}(\text{John}, z) \wedge \neg(y = z).$$

Il problema dell'unicità dei nomi

Fratello(John, Richard) $\wedge$ Fratello(John, Ramon) sembra dire che John ha due fratelli, ma **nella semantica standard di FOL** è soddisfatta anche se Richard e Ramon sono la **stessa persona**! Le formule per escludere questo diventano rapidamente complicate.

Database semantics

Per evitare queste complicazioni, la programmazione logica adotta spesso la **database semantics**, basata su tre assunzioni: (1) **Unicità dei nomi**: costanti diverse denotano sempre oggetti diversi. (2) **Closed-world assumption**: le formule atomiche di verità sconosciuta sono considerate false. (3) **Domain closure**: il modello non contiene più oggetti di quelli nominati dalle costanti presenti.

6.5 Inferenza in FOL: due approcci

(1) **Proposizionalizzazione** della KB e uso di un algoritmo per la logica proposizionale. (2) **Lifting** delle regole di inferenza al prim'ordine (con unificazione).

Sostituzione

Una **sostituzione** θ è un insieme $\{x_1/g_1, \dots, x_n/g_n\}$ dove le x_i sono variabili e le g_i sono termini ground. La notazione $F\theta$ indica la formula ottenuta applicando θ a F .

Proposizionalizzazione: UI ed EI

Regola di istanziazione universale (UI):

$$\frac{\forall x \alpha}{\text{SUBST}(\{x/g\}, \alpha)}$$

Da una formula universalmente quantificata si può inferire qualunque istanza ottenuta sostituendo alla variabile un **qualsiasi termine ground** del vocabolario.

Regola di istanziazione esistenziale (EI):

$$\frac{\exists x \alpha}{\text{SUBST}(\{x/k\}, \alpha)}$$

dove k deve essere una **costante nuova** (non ancora usata nella KB) — questo processo si chiama **skolemizzazione** (le costanti così introdotte sono dette **costanti di Skolem**).

Differenza fondamentale fra UI ed EI: UI produce tutte le istanze possibili; la nuova KB è **logicamente equivalente**. EI si applica **una sola volta** e poi la formula quantificata viene scartata; la nuova KB **non** è logicamente equivalente, ma è **soddisfacibile se e solo se** lo era la KB originaria.

Il problema delle funzioni e il teorema di Herbrand

Se il vocabolario contiene funzioni, esse possono essere annidate ricorsivamente, quindi l'insieme dei possibili termini ground diventa **potenzialmente infinito**.

Teorema di Herbrand: se una formula è conseguenza logica della KB FOL originaria, allora esiste una dimostrazione **finita** della sua verità a partire dalla KB proposizionalizzata. I termini vengono costruiti “in ampiezza” (breadth-first).

Conseguenza — semidecidibilità di FOL: Completezza: se una formula consegue dalla KB, la si troverà. Ma se la conseguenza **non** vale, la ricerca può proseguire **all'infinito**. Quindi: **FOL è semidecidibile** (si può confermare il “sì”, non sempre il “no”).

Inefficienza della proposizionalizzazione

Esempio:

$$\forall x \text{Re}(x) \wedge \text{Avido}(x) \Rightarrow \text{Malvagio}(x), \quad \text{Re}(\text{John}), \quad \text{Avido}(\text{John}), \quad \text{Fratello}(\text{Richard}, \text{John})$$

La proposizionalizzazione genera un'istanza dell'implicazione **per ogni costante del vocabolario**, incluso $\text{Re}(\text{Richard}) \wedge \text{Avido}(\text{Richard}) \Rightarrow \text{Malvagio}(\text{Richard})$, anche se è ovvio che solo John può risultare malvagio.

6.6 Modus Ponens Generalizzato (MPG)

Formula

$$\frac{p'_1, p'_2, \dots, p'_n, \quad (p_1 \wedge p_2 \wedge \dots \wedge p_n \Rightarrow q)}{q\theta}$$

dove $p'_i\theta = p_i\theta$ per ogni $i \in [1, n]$.

Se esiste una sostituzione θ tale che ciascuna p'_i unifica con la corrispondente p_i , allora si può concludere $q\theta$.

Esempio

Premesse: $\text{Re}(\text{John})$, $\text{Avido}(y)$, $\text{Re}(x) \wedge \text{Avido}(x) \Rightarrow \text{Malvagio}(x)$. Unificando $\text{Re}(\text{John})$ con $\text{Re}(x)$ (x/John) e $\text{Avido}(y)$ con $\text{Avido}(x)$ (y/John), $\theta = \{x/\text{John}, y/\text{John}\}$, si conclude $\text{Malvagio}(\text{John})$.

Lifting

Il **Modus Ponens Generalizzato** richiede che l'antecedente sia una **congiunzione di letterali positivi**, e generalizza la regola consentendo un numero arbitrario di premesse atomiche e operando su formule con **variabili**, tramite unificazione. Il termine “generalizzato” deriva dall’aver “sollevato” (**lifted**) la regola dalla logica proposizionale a quella del prim'ordine.

6.7 Unificazione

Definizione: l'unificazione è l'algoritmo chiave di tutte le tecniche di inferenza in FOL. Date due formule F_1 e F_2 : $\text{UNIFY}(F_1, F_2) = \theta$ tale che $F_1\theta = F_2\theta$. Se ne esistono più di una, si preferisce il **Most General Unifier (MGU)**.

Esempi passo-passo

1. $\text{UNIFY}(\text{Conosce}(\text{John}, x), \text{Conosce}(\text{John}, \text{Richard}))$: $\theta = \{x/\text{Richard}\}$.
2. $\text{UNIFY}(\text{Conosce}(\text{John}, x), \text{Conosce}(y, \text{Richard}))$: $\theta = \{x/\text{Richard}, y/\text{John}\}$.
3. $\text{UNIFY}(\text{Conosce}(\text{John}, x), \text{Conosce}(y, \text{MadreDi}(y)))$: y/John ; sostituendo, x deve unificare con $\text{MadreDi}(\text{John})$: $\theta = \{x/\text{MadreDi}(\text{John}), y/\text{John}\}$.
4. $\text{UNIFY}(\text{Conosce}(\text{John}, x), \text{Conosce}(x, \text{Richard}))$: **fallisce**, perché x dovrebbe assumere contemporaneamente due valori diversi. Si risolve con la **standardizzazione separata**: si rinominano le variabili di una formula in modo che non collidano mai con quelle di un'altra.

6.8 Clausole di Horn del prim'ordine

Formule atomiche (fatti): es. $\text{Avido}(x)$ (universalmente quantificata) o $\text{Avido}(\text{John})$ (ground).
Implicazioni il cui antecedente è una congiunzione di letterali positivi: $\text{Re}(x) \wedge \text{Avido}(x) \Rightarrow \text{Malvagio}(x)$.

KB di esempio: “Sotto Casa”

Enunciato: “La legge dice che è un reato per un negozio vendere alcolici a un minorenne. Marco, minorenne, possiede della birra. Tale birra gli è stata venduta dal minimarket Sotto Casa.”
Obiettivo: dimostrare che Sotto Casa è reo.

- C_1) $\text{Negozio}(x) \wedge \text{Vende}(x, y, z) \wedge \text{Alcolico}(y) \wedge \text{Minorenne}(z) \Rightarrow \text{Reo}(x)$
- C_2) $\text{Possiede}(\text{Marco}, B)$ (via EI su $\exists x \text{Possiede}(\text{Marco}, x) \wedge \text{Birra}(x)$)
- C_3) $\text{Birra}(B)$
- C_4) $\text{Possiede}(\text{Marco}, x) \wedge \text{Birra}(x) \Rightarrow \text{Vende}(\text{SottoCasa}, x, \text{Marco})$
- C_5) $\text{Birra}(x) \Rightarrow \text{Alcolico}(x)$
- C_6) $\text{Minimarket}(\text{SottoCasa})$
- C_7) $\text{Minimarket}(x) \Rightarrow \text{Negozio}(x)$
- C_8) $\text{Minorenne}(\text{Marco})$

Questa KB è composta da clausole di Horn del prim'ordine **senza funzioni**: questa sottoclasse (Horn + no funzioni) è chiamata **DATALOG**.

Forward Chaining in FOL

Un fatto è considerato una **rinomina** di un altro se identico a meno dei nomi delle variabili. In FOL si aggiunge un fatto inferito solo se **non è una rinomina** di un fatto già presente.

Esempio (continuazione KB Sotto Casa): da C_2, C_3, C_4 si applica MPG unificando x con B , ottenendo $\text{Vende}(\text{SottoCasa}, B, \text{Marco})$. Il processo continua costruendo un grafo AND-OR: si combina con C_5 per ottenere $\text{Alcolico}(B)$, con C_6/C_7 per ottenere $\text{Negozio}(\text{SottoCasa})$, con C_8 e infine con C_1 si conclude $\text{Reo}(\text{SottoCasa})$.

Proprietà: Correttezza: garantita. **Completezza:** se la KB è DATALOG, l'algoritmo è completo e **termina**. Se la KB contiene funzioni, per il teorema di Herbrand l'algoritmo può **non terminare**.

Backward Chaining (BC) in FOL

Come nel caso proposizionale, il ragionamento è **guidato dall'obiettivo**: (1) l'obiettivo viene inserito in uno **stack**; (2) iterativamente si estrae un obiettivo e si cercano clausole la cui **testa** sia **unificabile** con l'obiettivo; (3) quando lo stack è vuoto: successo.

Composizione delle sostituzioni: $\text{COMPOSE}(\theta_1, \theta_2) = \theta_3$ tale che $\text{SUBST}(\theta_3, F) = \text{SUBST}(\theta_2, \text{SUBST}(\theta_1, F))$.

Valutazione di BC: Corretto. Incompleto (depth-first, rischio loop infiniti). Generalmente **più efficiente** del forward chaining.

6.9 Risoluzione in FOL: lifting e refutazione

La combinazione risoluzione + refutazione, corretta e completa in proposizionale, può essere “liftata” anche a FOL: la KB va tradotta in **CNF**; le variabili sono implicitamente quantificate universalmente; ogni KB FOL può essere tradotta in una KB CNF **inferenzialmente equivalente**.

Procedura per tradurre FOL in CNF

Esempio guida: $\forall x [\forall y \text{Animale}(y) \Rightarrow \text{Ama}(x, y)] \Rightarrow [\exists y \text{Ama}(y, x)]$ (“tutti coloro che amano gli animali sono amati da qualcuno”).

Passo 1 — Elimina l'implicazione:

$$\forall x \neg [\forall y \neg \text{Animale}(y) \vee \text{Ama}(x, y)] \vee [\exists y \text{Ama}(y, x)]$$

Passo 2 — Sposta la negazione all'interno:

$$\forall x [\exists y \text{Animale}(y) \wedge \neg \text{Ama}(x, y)] \vee [\exists y \text{Ama}(y, x)]$$

Passo 3 — Standardizzazione delle variabili: le due occorrenze di $\exists y$ hanno scope diversi, vanno rinominate:

$$\forall x [\exists y \text{Animale}(y) \wedge \neg \text{Ama}(x, y)] \vee [\exists z \text{Ama}(z, x)]$$

Passo 4 — Skolemizzazione. Passo 5 — Cancella i quantificatori universali residui.

Passo 6 — Distribuisci $\vee$ su $\wedge$.

6.10 Skolemizzazione

Perché serve

Al passo 4, non si può applicare direttamente EI perché l'esistenziale è annidato dentro lo scope di un universale. Se, sbagliando, si sostituisse $\exists y \text{Animale}(y)$ con una singola costante A :

$$\forall x [\text{Animale}(A) \wedge \neg \text{Ama}(x, A)] \vee [\text{Ama}(B, x)]$$

significherebbe “**tutti** amano uno **stesso specifico** animale A ” — **sbagliato**.

Procedura (regola generale)

Si sostituisce ogni variabile quantificata esistenzialmente con una **funzione di Skolem** i cui argomenti sono **tutte le variabili universalmente quantificate nel cui scope ricade** l'esistenziale:

$$\begin{aligned} & \forall x_1, x_2, \dots [\exists y P(y, x_1, \dots) \dots \exists z Q(z, x_1, \dots)] \\ & \Downarrow \\ & \forall x_1, x_2, \dots [P(S_1(x_1, x_2, \dots), \dots) \dots Q(S_2(x_1, x_2, \dots), \dots)] \end{aligned}$$

Caso particolare: se l'esistenziale **non** ricade nello scope di alcun universale, la funzione di Skolem degenera in una **costante di Skolem** (=EI).

Applicando all'esempio guida: y e z ricadono nello scope di $\forall x$, quindi diventano funzioni di x :

$$\forall x [\text{Animale}(F(x)) \wedge \neg \text{Ama}(x, F(x))] \vee [\text{Ama}(G(x), x)]$$

Completando i passi 5 e 6:

$$[\text{Animale}(F(x)) \vee \text{Ama}(G(x), x)] \wedge [\neg \text{Ama}(x, F(x)) \vee \text{Ama}(G(x), x)]$$

Esempio più semplice, passo-passo

Consideriamo: $\forall x \exists y \text{persona}(x) \Rightarrow (\text{abita}(x, y) \wedge \text{paese}(y))$ (“ogni persona abita in un luogo che è in un paese”).

Errore da non fare: applicare EI ingenuamente eliminerebbe $\exists y$ con una singola costante K : $\forall x \text{persona}(x) \Rightarrow (\text{abita}(x, K) \wedge \text{paese}(K))$ — affermerebbe che **tutte le persone abitano nello stesso posto**.

Soluzione corretta: il luogo dipende dallo specifico individuo, quindi si introduce una funzione di Skolem $\text{luogo}(x)$:

$$\forall x \text{persona}(x) \Rightarrow (\text{abita}(x, \text{luogo}(x)) \wedge \text{paese}(\text{luogo}(x))).$$

Funzioni di Skolem con più argomenti

Esempio: in un rally, per ogni coppia di persone x, y esiste un'auto z di cui x è il pilota e y il navigatore:

$$\forall x, y \exists z \text{ persona}(x) \wedge \text{persona}(y) \wedge \text{auto}(z) \wedge \neg(x=y) \Rightarrow (\text{pilota}(x, z) \wedge \text{navigatore}(y, z)).$$

Skolemizzando (z dipende sia da x che da y):

$$\forall x, y \text{ persona}(x) \wedge \text{persona}(y) \wedge \text{auto}(A(x, y)) \wedge \neg(x=y) \Rightarrow (\text{pilota}(x, A(x, y)) \wedge \text{navigatore}(y, A(x, y))).$$

Caso limite: l'esistenziale “esterno” a tutti gli universali

Se la formula è $\forall x, y \exists z \text{ formula}(x, y, z)$: z è dentro lo scope di $x, y \Rightarrow$ funzione di Skolem $S(x, y)$.
Se invece è $\exists z [\forall x, y \text{ formula}(x, y, z)]$: qui è $\exists z$ a essere più esterno, quindi la funzione di Skolem **non ha argomenti** — degenera in una **costante di Skolem**.

Domanda d'esame. Da cosa dipendono gli argomenti di una funzione di Skolem, e perché a volte si riduce a una costante? — Gli argomenti sono esattamente le variabili quantificate universalmente il cui scope contiene il quantificatore esistenziale da eliminare, perché il valore “esistente” può, in linea di principio, variare al variare di quelle variabili universali già fissate più esternamente. Se non ci sono variabili universali che “contengono” l'esistenziale, il valore esistente è **unico e fisso**, e la funzione di Skolem degenera in una semplice **costante di Skolem**.

6.11 Binary resolution in FOL (lifting della risoluzione)

$$\frac{l_1 \vee \dots \vee l_k, \quad m_1 \vee \dots \vee m_n}{\text{SUBST}(\theta, l_1 \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n)}$$

dove θ è una sostituzione tale che, per una coppia di indici i, j , θ **unifica** l_i con $\neg m_j$.

Esempio: $\text{Re}(\text{John})$ e $\neg \text{Re}(x)$; $\theta = \{x/\text{John}\}$ fa risolvere le due clausole eliminando questi due letterali.

Osservazioni: le due clausole non devono condividere variabili (standardizzazione separata); va fatto anche il lifting della **fattorizzazione** (letterali unificabili, non solo sintatticamente uguali).

Binary resolution + **fattorizzazione** sono insieme **complete**.

Dimostrazione per refutazione: l'esempio “Curiosity ha ucciso il gatto?”

- A) $\forall x [\forall y \text{ Animale}(y) \Rightarrow \text{Ama}(x, y)] \Rightarrow [\exists y \text{ Ama}(y, x)]$
- B) $\forall x [\exists z \text{ Animale}(z) \wedge \text{Uccide}(x, z)] \Rightarrow [\forall y \neg \text{Ama}(y, x)]$
- C) $\forall x \text{ Animale}(x) \Rightarrow \text{Ama}(\text{Jack}, x)$
- D) $\text{Uccide}(\text{Jack}, \text{Tuna}) \vee \text{Uccide}(\text{Curiosity}, \text{Tuna})$
- E) $\text{Gatto}(\text{Tuna})$
- F) $\forall x \text{ Gatto}(x) \Rightarrow \text{Animale}(x)$

Query: **Curiosity ha ucciso il gatto?** Si aggiunge il goal negato $G) \neg \text{Uccide}(\text{Curiosity}, \text{Tuna})$.

Traduzione in CNF (con skolemizzazione: in A , $\exists y$ diventa $G(x)$; in B , $\exists z$ diventa $F(x)$):

- $A_1)$ $\text{Animale}(F(x)) \vee \text{Ama}(G(x), x)$
- $A_2)$ $\neg \text{Ama}(x, F(x)) \vee \text{Ama}(G(x), x)$
- $B)$ $\neg \text{Ama}(y, x) \vee \neg \text{Animale}(z) \vee \neg \text{Uccide}(x, z)$
- $C)$ $\neg \text{Animale}(x) \vee \text{Ama}(\text{Jack}, x)$
- $D)$ $\text{Uccide}(\text{Jack}, \text{Tuna}) \vee \text{Uccide}(\text{Curiosity}, \text{Tuna})$
- $E)$ $\text{Gatto}(\text{Tuna})$
- $F)$ $\neg \text{Gatto}(x) \vee \text{Animale}(x)$
- $G)$ $\neg \text{Uccide}(\text{Curiosity}, \text{Tuna})$

Catena di risoluzione: (1) E risolve con F (x/Tuna) $\rightarrow \text{Animale}(\text{Tuna})$. (2) D risolve con $G \rightarrow \text{Uccide}(\text{Jack}, \text{Tuna})$. (3) $\text{Animale}(\text{Tuna})$ risolve con B (z/Tuna) $\rightarrow \neg \text{Ama}(y, x) \vee \neg \text{Uccide}(x, \text{Tuna})$. (4) risolve con $\text{Uccide}(\text{Jack}, \text{Tuna})$ (x/Jack) $\rightarrow \neg \text{Ama}(y, \text{Jack})$. (5)–(7) si continua a risolvere con A_2 e A_1 (istanziate in Jack) fino a produrre, per fattorizzazione/risoluzione, la **clausola vuota**: $KB \wedge \neg Q$ è insoddisfacibile, dunque **Curiosity ha ucciso il gatto** è conseguenza logica della KB.

Refutation-completeness

La risoluzione **non** è in grado di enumerare *tutte* le conseguenze logiche di una KB, ma è **refutation-complete**: se una KB è **insoddisfacibile**, la risoluzione sarà **sempre in grado di derivare, in un numero finito di passi, la clausola vuota**.

6.12 Costruire una KB in FOL: ingegneria della conoscenza

Processo generale: (1) identificare l'uso della KB; (2) raccogliere la conoscenza rilevante; (3) definire un vocabolario di costanti, funzioni e predicati; (4) formalizzare la conoscenza in formule FOL. Per interrogare: (1) descrivere formalmente la specifica istanza; (2) interrogare la KB.

Riepilogo e punti chiave — Modulo 6

- FOL aggiunge **oggetti, relazioni (predicati), funzioni e quantificatori**, mantenendo le buone proprietà della proposizionale.
- Un **modello FOL** è la coppia $M = (D, I)$. Il valore di verità dipende sempre dal modello scelto.
- $\forall$ si accompagna a $\Rightarrow$, $\exists$ si accompagna a $\wedge$. L'**ordine dei quantificatori annidati di tipo diverso conta**.
- L'**uguaglianza** serve a distinguere oggetti; senza $\neg(x = y)$ esplicito, FOL standard non assume l'unicità dei nomi (a differenza della **database semantics**).
- L'inferenza avviene per **proposizionalizzazione** (completa per Herbrand ma inefficiente, e FOL è **semidecidibile**) o per **lifting** (più efficiente).
- Il **Modus Ponens Generalizzato** è il motore del **forward** e **backward chaining** su clausole di Horn. L'**unificazione** trova il Most General Unifier.
- La **risoluzione liftata** a FOL è **refutation-complete**. Per applicarla, la KB va portata in **CNF**, con **skolemizzazione** degli esistenziali: ogni $\exists y$ diventa una funzione di Skolem delle variabili universali nel cui scope ricade (o una costante, se non ricade in alcun $\forall$).

Capitolo 7

Tassonomie/Ontologie e Pianificazione automatica

7.1 Tassonomie e Ontologie

7.1.1 Cos'è una tassonomia

Una **tassonomia** è un'organizzazione gerarchica di categorie o concetti. Il mattone concettuale è la **relazione di sottoclasse** (relazione **Is-a**): se PalloneCalcio è sottoclasse di Pallone, allora **tutte le istanze di PalloneCalcio sono anche istanze di Pallone**.

Perché è utile: permette di caratterizzare le categorie tramite **proprietà** che valgono per tutte le istanze, ad esempio $\text{Member}(X, \text{Pallone}) \Rightarrow \text{Sferico}(X)$. Grazie alla relazione di sottoclasse, le istanze **ereditano** le proprietà delle sovraclassi: non serve ripetere la regola per ogni sottoclasse, perché vale già per **ereditarietà**.

7.1.2 Decomposizioni, disgiunzione, partizione

Quando si scompone una categoria C in sottocategorie $S = \{X_1, \dots, X_n\}$:

Disjoint(S): le categorie non hanno istanze in comune:

$$\forall X_i, X_j \in S, X_i \neq X_j \Rightarrow \text{Intersection}(X_i, X_j) = \{\}.$$

ExhaustiveDec(S,C): ogni istanza di C appartiene ad almeno una delle categorie di S :

$$\forall I (\text{Member}(I, C) \Leftrightarrow \exists X_i \text{ Is-a}(X_i, C) \wedge \text{Member}(I, X_i)).$$

Partition(S,C): S è sia disgiunto sia esaustivo:

$$\text{Partition}(S, C) \Leftrightarrow \text{Disjoint}(S) \wedge \text{ExhaustiveDec}(S, C).$$

7.1.3 Relazioni strutturali: Part-of e Bunch-of

Part-of è transitiva: $\text{Part-of}(X, Y) \wedge \text{Part-of}(Y, Z) \Rightarrow \text{Part-of}(X, Z)$. Le relazioni strutturali comprendono anche predicati dipendenti dal dominio, ad esempio:

$$\text{Flower}(x) \Rightarrow \exists \text{Corolla, Stelo } \text{Part-of}(\text{Corolla}, x) \wedge \text{Part-of}(\text{Stelo}, x) \wedge \text{On-top}(\text{Corolla}, \text{Stelo}).$$

Bunch-of (mucchio) serve quando si vuole dire che un oggetto è composto da parti **senza specificare le relazioni fra queste**: $\forall x \text{ In}(x, s) \Rightarrow \text{Part-of}(x, \text{Bunch-of}(s))$.

7.1.4 T-box e A-box

Un'ontologia si struttura sempre in due parti: **T-box** (terminological box): la parte **generale/intensionale**, fatta di definizioni, specializzazioni e proprietà. **A-box** (assertional box): la parte **estensionale**, contiene fatti su istanze specifiche, che devono essere coerenti con la T-box. Esempio: $T\text{-box} \rightarrow \text{Madre}(X) \Rightarrow \text{Donna}(X)$; $A\text{-box} \rightarrow \text{Madre}(\text{Anna})$.

7.1.5 Problematiche delle tassonomie/ontologie

Norma ed eccezioni: molte proprietà valgono “di default” ma ammettono eccezioni (gli uccelli di solito volano, ma struzzi, pinguini, kiwi, emù no). Le eccezioni possono **cancellare** proprietà ereditate.

Polisemia: una stessa parola può denotare concetti diversi e appartenere ad alberi tassonomici differenti (es. “cane” è animale, costellazione, persona vile, dente meccanico). Il motore inferenziale non deve mischiare le tassonomie: il problema aperto è come identificare i concetti in modo univoco.

7.1.6 Dalla tassonomia all'ontologia

Una tassonomia, essendo un **albero**, è un caso particolare e più limitato di struttura. Una base di conoscenza descrittiva può assumere una forma più generale, in cui i concetti e le relazioni formano un **grafo** invece che un albero: questo insieme più generale si chiama **ontologia** (o **rete semantica**).

Tassonomia vs Ontologia: la tassonomia cattura *solo* le relazioni gerarchiche Is-a; un'ontologia è più espressiva e può includere anche relazioni Part-of, relazioni di dominio specifiche, e in generale una rete arbitraria di concetti. **Le tassonomie sono quindi un caso speciale di ontologia.**

Tipi di interrogazione tipici su un'ontologia: (1) istanza appartiene a categoria? (2) istanza gode di proprietà? (3) differenza fra categorie? (4) identificazione di istanze.

7.1.7 Semantic Web e linguaggi per rappresentare ontologie

Il **Semantic Web** (termine coniato da Tim Berners-Lee) è l'estensione del web tradizionale in cui i contenuti pubblicati sono arricchiti da **metadati** che abilitano interpretazione, inferenza, interrogazione ed elaborazione automatica. I linguaggi sono standardizzati dal **W3C**.

RDF (Resource Description Framework): modello/linguaggio di rappresentazione **base**. La conoscenza è espressa tramite **triple** *soggetto – predicato – oggetto*. Soggetto, predicato e oggetto sono **IRI** (internationalized resource identifier) — questo risolve il problema di identificazione univoca dei concetti: ogni concetto ha un identificatore globale non ambiguo. **RDFS** permette di costruire tassonomie appoggiandosi a RDF. **SPARQL** è il linguaggio di interrogazione per dati RDF.

OWL 2 (Web Ontology Language): linguaggio dichiarativo pensato specificamente per **definire ontologie** tramite classi, proprietà, individui e valori. Modella la conoscenza con: **Entità** (individui, classi, proprietà), **Assiomi** (es. `ClassAssertion(:Persona : Maria)`), **Espressioni** (combinazioni complesse).

Costrutti principali: `Declaration`, `ClassAssertion`, `SubClassOf` (Is-a), `EquivalentClasses`, `DisjointClasses`, `ObjectPropertyAssertion`, `SubObjectPropertyOf`, `ObjectPropertyDomain/Range`, `ObjectIntersectionOf/UnionOf/ComplementOf`, `ObjectSomeValuesFrom` (esistenziale), `ObjectAllValuesFrom` (universale).

OWL 2 non è un framework per basi di dati: DB usa **closed world assumption** (un fatto non presente è falso) — OWL 2 usa **open world assumption** (un fatto non presente è

semplicemente sconosciuto/mancante). OWL non impone che le uniche proprietà di un individuo siano quelle della sua classe.

FOAF (Friend Of A Friend): ontologia per collegare persone sul web (classi Agent, Person, Group; proprietà name, knows, age).

7.1.8 Costruire un'ontologia: metodologia pratica

(1) **Identificazione dei concetti:** elencare i sostantivi rilevanti, poi identificare le sottoclassi. (2) **Identificazione delle proprietà:** elencare le relazioni (verbi). (3) **Riuso:** verificare ontologie esistenti riutilizzabili. (4) **Scrittura formale** in RDF/OWL — si ottiene la **T-box**. (5) **Annotazione dei dati** — si popola la **A-box**. (6) **Validazione e raffinamento**. (7) **Costruzione di un sistema di interrogazione**.

Strumenti citati: Protégé, reasoner (CEL, FaCT++, HermiT, Pellet, RacerPro), Alignment API, Apache Jena, AllegroGraph.

7.1.9 Relazioni fra ontologie

Identical: O_1 e O_2 sono la stessa ontologia. **Equivalent:** condividono vocabolario e assiomatizzazione ma sono espresse in linguaggi differenti. **Extension:** O_1 estende O_2 quando tutti i simboli di O_2 sono preservati in O_1 insieme a proprietà e relazioni, ma non vale il viceversa. **Weakly-Translatable:** è possibile tradurre espressioni di O_{source} in O_{dest} , ma **con perdita di informazione**. **Strongly-Translatable:** vocabolario totalmente mappabile, assiomatizzazione preservata, senza perdita né inconsistenze. **Approx-Translatable:** weak + possibili inconsistenze, per concetti solo affini (es. il coriandolo, affine a prezzemolo o pepe a seconda della tradizione).

Domanda d'esame. Qual è la differenza fra una tassonomia e un'ontologia? — Una tassonomia è una struttura **gerarchica ad albero** costruita esclusivamente tramite relazioni Is-a: ogni concetto eredita le proprietà del proprio “genitore”. Un'ontologia è un concetto più generale: un **grafo** di concetti collegati da relazioni di natura diversa (Is-a, Part-of, relazioni di dominio), non necessariamente organizzato ad albero. Ogni tassonomia è quindi un caso particolare di ontologia, ma non vale il viceversa.

Domanda d'esame. Perché Is-a e Part-of non bastano a rappresentare qualunque tipo di conoscenza? — Perché sono relazioni essenzialmente **statiche**, adatte a descrivere *cosa* è qualcosa o *di cosa è fatto*, ma non catturano il concetto di **azione** e di **cambiamento nel tempo**. Per questo serve un formalismo diverso, orientato alle azioni e agli stati: il **Situation Calculus**, ponte verso la pianificazione.

7.2 Pianificazione automatica

7.2.1 Dalle ontologie alla pianificazione: perché serve un nuovo formalismo

Pianificare significa costruire una **sequenza di azioni** che, applicata a partire da uno stato iniziale, soddisfa un certo obiettivo (goal).

Il mondo è descritto da un insieme di variabili, tipicamente nel linguaggio **PDDL** (*Planning Domain Definition Language*). Uno **stato** è una congiunzione di **atomi ground**, es. $\text{At}(\text{Camion1}, \text{Bari}) \wedge \text{At}(\text{Camion2}, \text{Lecce})$. Le azioni sono descritte in modo **schematico** (parametriche) e hanno un impatto **limitato** sul mondo. Aspetto pratico rilevante: se l'azione scelta viene davvero eseguita nel mondo reale, **potrebbe non essere possibile fare backtracking**.

7.2.2 Differenza rispetto alla ricerca classica

Nella pianificazione, gli stati e le azioni hanno una **rappresentazione logica strutturata** (predicati, precondizioni, effetti); l'obiettivo è tipicamente **complesso** e viene scomposto in sottoobiettivi; si presta esplicita attenzione al problema di **rappresentare gli effetti delle azioni** (frame problem).

7.2.3 Il Situation Calculus: fondamento logico della pianificazione

Il **Situation Calculus** è la rappresentazione logica (in FOL) su cui si basa storicamente il ragionamento su azioni:

Azione: in logica non è un predicato ma una **funzione** che restituisce un oggetto del dominio. Esempio: $\text{Move}(R, L_1, L_2)$. **Situazione:** lo stato risultante dall'esecuzione di una sequenza di azioni. **Fluente:** proprietà/relazione che può **cambiare valore** con l'esecuzione delle azioni; ha sempre come parametro la situazione, es. $\text{Holds}(\text{At}(R, \text{Loc}), s)$. **Predicato/funzione atemporale:** il cui valore **non** dipende dalle azioni eseguite.

Il tempo non è rappresentato esplicitamente, ma scandito dalla sequenza di eventi: si parte da una situazione iniziale S_0 e si applicano azioni $a_1, a_2, a_3, \dots$ generando $S_1, S_2, S_3, \dots$. La funzione $\text{Do}(\text{Azioni}, s)$ restituisce la situazione raggiunta:

$$\text{Do}([], s) = s, \quad \text{Do}([a \mid \text{resto}], s) = \text{Do}(\text{resto}, \text{Risultato}(a, s)).$$

Nota fondamentale: **due situazioni sono identiche solo se derivano dallo stesso stato iniziale tramite la stessa sequenza di azioni:**

$$\text{Do}(\text{Azioni}_1, S_1) = \text{Do}(\text{Azioni}_2, S_2) \Leftrightarrow (\text{Azioni}_1 = \text{Azioni}_2) \wedge (S_1 = S_2).$$

Tramite Do un agente può fare **proiezione**: ragionare sugli effetti futuri delle azioni.

Rappresentazione delle azioni tramite due tipi di assiomi:

Assioma di Applicabilità:

$$\forall \text{params}, s \text{ Applicable}(\text{Action}(\text{params}), s) \Leftrightarrow \text{Precond}(\text{params}, s).$$

Assioma di Effetto:

$$\forall \text{params}, s \text{ Applicable}(\text{Action}(\text{params}), s) \Rightarrow \text{Effects}(\text{params}, \text{Result}(\text{Action}(\text{params}), s)).$$

Esempio guida — mondo dei blocchi: l'azione $\text{Move}(X, Y, Z)$ sposta il blocco X da Y a Z .

Applicabilità: $\text{Applicable}(\text{Move}(x, y, z), s) \Leftrightarrow \text{Clear}(x, s) \wedge \text{Clear}(z, s) \wedge \text{On}(x, y, s) \wedge x \neq z \wedge y \neq z \wedge x \neq \text{Table}$.

Effetto: $\text{Applicable}(\text{Move}(x, y, z), s) \Rightarrow \text{On}(x, z, \text{Result}(\dots)) \wedge \text{Clear}(y, \text{Result}(\dots))$.

7.2.4 Il Frame Problem

Frame problem: le azioni hanno tipicamente un impatto **limitato** sul mondo — come rappresentare tutto ciò che **non** viene modificato da un'azione?

Dalla sola conoscenza di stato iniziale + assiomi di applicabilità/effetto **non si può derivare** ciò che rimane invariato. Due soluzioni:

(1) **Enumerare esplicitamente ciò che non cambia:** per ogni azione e per ogni fluente non toccato si scrive un assioma tipo

$$\forall \text{params}, \text{vars}, s \text{ fluent}(\text{vars}, s) \wedge \text{params} \neq \text{vars} \Rightarrow \text{fluent}(\text{vars}, \text{Result}(\text{Action}(\text{params}), s)).$$

Problema pratico: bisogna scrivere un assioma per **ogni combinazione azione $\times$ fluente non toccato** — il numero esplode.

(2) **Assioma di stato successore:** un unico schema generale per ciascun fluente che dice: “azione applicabile $\Rightarrow$ (fluente vero nella situazione risultante $\Leftrightarrow$ (l’azione lo rende vero $\vee$ (era vero $\wedge$ l’azione non l’ha reso falso)))”. Questo compatta in un solo schema effetti positivi e persistenza per default, evitando l’esplosione combinatoria.

Il Situation Calculus è completato da assiomi di **unique action names**: azioni con nomi diversi sono oggetti diversi.

7.2.5 Perseguire goal complessi: scomposizione in sottogoal

L’approccio tipico è **scomporlo in sottoobiettivi** e perseguirli, in linea di principio, uno alla volta. **Esempio (mondo dei blocchi):** stato iniziale con A e B sul tavolo, C sopra uno di essi; goal = “A su B su C”. Il goal si scompone in: Sottogoal 1: A su B; Sottogoal 2: B su C.

7.2.6 L’anomalia di Sussman: quando i sottogoal interagiscono

Non sempre il perseguimento dei sottoobiettivi è sequenzializzabile in modo indipendente.

Stato iniziale: C sul tavolo da solo, A sopra B sul tavolo. Goal: pila A su B su C. I due sottogoal (“A su B” e “B su C”) sono **in conflitto** se perseguiti nell’ordine sbagliato: se perseguo prima “B su C”, per liberare B devo spostare A altrove, disfando la pila A/B; se perseguo prima “A su B”, poi per “B su C” dovrei comunque spostare l’intera pila.

Soluzione — interleaving dei passi: la soluzione efficiente richiede di **intercalare** i passi provenienti dai due sottopiani. Sequenza corretta: (1) spostato C; (2) spostato A da sopra B; (3) spostato B su C; (4) spostato A su B.

Domanda d’esame. Cos’è l’anomalia di Sussman e cosa dimostra? — È l’esempio classico che mostra come la scomposizione di un obiettivo complesso in sottoobiettivi **non sia in generale risolvibile perseguendo i sottoobiettivi in sequenza indipendente**: risolvere un sottogoal può richiedere di disfare i progressi fatti per un altro sottogoal già raggiunto, e viceversa. Dimostra che i sottoobiettivi possono **interagire** fra loro, e che un planner corretto deve poter fare **interleaving** dei passi provenienti da piani diversi.

Domanda d’esame. Cos’è il frame problem e come si risolve? — È il problema di rappresentare esplicitamente **ciò che un’azione NON modifica**. Si risolve in due modi: (1) enumerando esplicitamente un assioma di frame per ogni coppia azione/fluente non toccato (esplode combinatoriamente); (2) tramite un **assioma di stato successore** unico per fluente, che stabilisce per default che un fluente resta vero se lo era prima e l’azione non lo ha reso falso.

7.2.7 Limiti pratici del Situation Calculus

Il Situation Calculus ha permesso di formalizzare rigorosamente il problema della pianificazione, ma nella **pratica** non è molto usato, perché **non esistono euristiche efficienti** che guidino la ricerca nello spazio (molto grande) delle sequenze di azioni possibili. I planner moderni usano rappresentazioni più specializzate (PDDL con algoritmi dedicati, planning graph, euristiche ad hoc).

Riepilogo e punti chiave — Modulo 7

- **Tassonomia** = organizzazione gerarchica (ad albero) di concetti tramite **Is-a**: le istanze ereditano le proprietà della superclasse. **Disjoint**, **ExhaustiveDec**, **Partition** formalizzano le decomposizioni.
- Relazioni **strutturali** (Part-of, transitiva; Bunch-of) affiancano Is-a. Ogni ontologia si divide in **T-box** e **A-box**.
- Un'**ontologia** generalizza la tassonomia: da albero a **grafo**. Il **Semantic Web** standardizza RDF, RDFS, OWL 2, SPARQL, FOAF.
- Is-a e Part-of **non bastano** a rappresentare le **azioni** → ponte verso la pianificazione.
- **Pianificare** = costruire una sequenza di azioni da stato iniziale a goal. Il **Situation Calculus** fornisce il fondamento in FOL: Azione, Situazione, Fluente, Do, assiomi di Applicabilità ed Effetto.
- Il **frame problem** si risolve con l'**assioma di stato successore**. L'**anomalia di Sussman** mostra che i sottogoal possono interagire, richiedendo **interleaving**.
- Il Situation Calculus è poco usato in pratica per mancanza di euristiche efficienti.

Capitolo 8

Machine Learning: Classificazione, Alberi di Decisione, Reti Neurali

8.1 Il problema della classificazione

Le classi non sono “naturalì”

Un punto concettuale sottolineato fin da subito: **le classi non esistono in natura**. Non sono insite negli esempi, non sono raggruppamenti naturali inequivocabili: sono **definite a tavolino** da chi costruisce il sistema, in funzione dello scopo per cui si vuole costruire il predittore. Il sistema **non ha altra conoscenza oltre ai dati** e impara esattamente quello che c'è nei dati per come sono etichettati, non quello che il progettista *pensa* di aver messo nei dati.

Definizione del problema

Dati: **Esempi** (istanze/record): gli oggetti da classificare. **Categorie o classi**: le etichette di appartenenza. Obiettivo: costruire una **rappresentazione astratta (modello)** che permetta di associare correttamente nuove istanze alla classe di appartenenza. Si parla di **apprendimento supervisionato** quando gli esempi hanno già associata la classe corretta.

Il problema si scompone in tre sotto-problemi: (1) Rappresentazione dei dati; (2) Analisi dei dati e costruzione delle definizioni; (3) Utilizzo della conoscenza acquisita.

Schema generale: training set, modello, test set

DATI TRAINING SET $\xrightarrow{\text{induzione}}$ MODELLO: classe = f(descrizione) $\xrightarrow{\text{deduzione}}$ CLASSE

Training (learning) set: la collezione di dati usati per l'apprendimento. Ogni istanza è una tupla (x, y) dove x è una tupla di valori di attributi descrittivi e y è la classe di appartenenza. Il processo di **induzione** costruisce il modello partendo dal training set. Il processo di **deduzione** applica il modello a nuove descrizioni. Il **test set** serve a valutare la bontà del modello appreso.

Modello predittivo vs modello descrittivo

Modello predittivo	Modello descrittivo
Usato per predire la classe di istanze ignote, non viste in fase di apprendimento	Usato come strumento esplicativo che evidenzia quali caratteristiche distinguono le categorie

Valutazione: matrice di confusione, accuratezza, error rate

Matrice di confusione (caso a 2 classi):

	classe predetta 1	classe predetta 2
classe reale 1	f_{11} (corretto)	f_{12} (errore)
classe reale 2	f_{21} (errore)	f_{22} (corretto)

Accuratezza = $(f_{11} + f_{22}) / (f_{11} + f_{22} + f_{12} + f_{21})$ — predizioni corrette su predizioni totali.
Error rate = $(f_{12} + f_{21}) / (f_{11} + f_{22} + f_{12} + f_{21})$.

Matrice dei costi

Non tutti gli errori hanno lo stesso peso. La **matrice dei costi** assegna un costo a ciascuna cella della matrice di confusione. Esempio classico: è più grave dire a un malato che è sano (falso negativo, costoso) o dire a una persona sana che è malata (falso positivo, meno grave)? Il costo complessivo si calcola sommando, cella per cella, $\text{costo}_{ij} \times \text{frequenza}_{ij}$.

Domanda d'esame. Perché un'accuratezza del 99.9% non è automaticamente sinonimo di “ottimo classificatore”? — Perché l'accuratezza va sempre letta insieme alla distribuzione delle classi nel dataset. Se il learning set è fortemente sbilanciato (es. 9990 istanze di classe A e solo 10 di classe B), un classificatore banale che risponde sempre “classe A” ottiene un'accuratezza del 99.9% pur non avendo imparato nulla di utile: non ha individuato nessun pattern discriminante, si limita a sfruttare lo sbilanciamento delle classi. Bisogna quindi guardare anche altre metriche (matrice di confusione completa, per-classe) e la composizione del dataset.

Insidie pratiche nella costruzione del classificatore

Esempio guida: riconoscere “frutta” da altre cose usando solo foto di mele → il modello impara a riconoscere solo mele, non la frutta in generale. Cause tipiche: **dati difficili da reperire** → dataset non rappresentativi; **bias mentale** di chi costruisce il dataset; **attribuire all'algoritmo capacità umane** — un umano generalizza usando moltissima conoscenza di base senza accorgersene; un algoritmo impara solo dai dati che vede.

Applicazioni ad alto impatto: riconoscimento facciale, concessione di mutui, assicurazioni sanitarie, applicazioni militari, agricoltura di precisione. Tema della **responsabilità etica**: nel caso della guida autonoma (variante del *trolley problem*), chi è responsabile della decisione presa da un agente autonomo? Si parla di *problem of the many hands*: la responsabilità è distribuita fra molti attori, rendendo difficile individuare un singolo responsabile.

Rote learning (apprendimento meccanico) — un “non-modello”

Strategia: memorizzare semplicemente tutte le istanze del training set. Non esiste un vero modello: il sistema “ricorda” i casi ma non generalizza. Data una nuova istanza, cerca un'istanza identica in memoria; se non la trova, cerca istanze **simili** (misura di distanza): se le istanze simili trovate hanno tutte la stessa classe, quella è l'output; se le classi sono diverse, serve una strategia di composizione: **votazione a maggioranza**, o **votazione pesata** (il peso dipende dalla distanza; i pesi vengono calcolati, usati e poi dimenticati, a differenza dei pesi di una rete neurale).

Questo approccio anticipa concettualmente algoritmi come k-NN, ma serve soprattutto a introdurre il concetto di **generalizzazione**: algoritmi di apprendimento diversi producono modelli di tipo diverso (alberi di decisione → albero; sistemi a regole → insiemi di regole if-then; reti neurali → matrici di numeri).

8.2 Alberi di decisione

Cosa sono

Gli alberi di decisione (*decision tree*, DT) sono strumenti di supporto alle decisioni basati su modelli strutturati ad albero. Applicati alla classificazione: si induce da esempi un albero di decisione che, dato un nuovo record, permette di predirne la classe seguendo un percorso di test sui valori degli attributi fino a una foglia.

Struttura

Nodi interni: rappresentano un **test** su un attributo descrittivo. **Foglie:** rappresentano una **decisione**, cioè l'assegnazione a una classe.

Esempio concreto — dataset **Iris** (3 classi: *setosa*, *versicolor*, *virginica*):

```
Petal width ≤ 0.6?  
├ sì → IRIS SETOSA  
└ no → Petal width ≤ 1.7?  
    ├── no → IRIS VIRGINICA  
    ├── sì → Petal length ≤ 4.9?  
    │   ├── sì → IRIS VERSICOLOR  
    │   └ no → Petal width ≤ 1.5?  
    │       ├── sì → IRIS VIRGINICA  
    │       └ no → IRIS VERSICOLOR
```

Algoritmi per apprendere alberi di decisione

Algoritmo di Hunt (lo schema ricorsivo di base), **ID3**, **C4.5**, **CART**.

L'algoritmo di Hunt

L'albero viene costruito **ricorsivamente**, suddividendo il learning set in sottoinsiemi via via più "puri". Notazione: D_t = sottoinsieme del learning set associato al nodo t ; $y = \{y_1, \dots, y_c\}$ = insieme delle etichette di classe possibili.

Procedura ricorsiva: (1) **Caso base:** se tutte le istanze in D_t appartengono alla stessa classe y_t , il nodo t diventa una **foglia** etichettata y_t . (2) **Passo ricorsivo:** altrimenti, si sceglie un attributo e si produce un **nodo figlio per ogni possibile valore** dell'attributo, ricorrendo su ciascun figlio.

Note: se una combinazione di valori non è rappresentata da nessuna istanza, si associa una **classe di default**; se tutte le istanze associate a un nodo hanno tuple identiche ma classi diverse, il nodo diventa foglia con la classe di **maggioranza**; restano da definire i **criteri di arresto** e **come scegliere l'attributo di split**.

Strategia greedy e problemi aperti

La costruzione dell'albero segue una strategia **greedy**: ad ogni passo si seleziona l'attributo di split che massimizza localmente una misura di riferimento, senza backtracking.

Tipi di split in base al tipo di attributo

Attributi binari: lo split produce esattamente 2 figli. **Attributi nominali:** *split multivalore* (un figlio per ciascun valore) o *split binario* (un valore vs il resto, con $2^{k-1} - 1$ raggruppamenti possibili per k valori). **Attributi ordinali:** il raggruppamento deve **rispettare l'ordinamento**. **Attributi continui:** *split binario* con soglia $A \leq v$, o *split multivalore* con più soglie (discretizzazione).

Quale attributo scegliere per lo split? Misure di impurità

Criterio generale (Rasoio di Occam): a parità di prestazioni, si preferiscono alberi più compatti. **Idea intuitiva:** sono preferiti gli split che producono nodi figli con **minore confusione**, cioè con grado di purezza maggiore.

Si definisce, per un nodo t , $p(i|t)$ = probabilità che un elemento estratto casualmente dal nodo t sia di classe i .

Entropia:

$$\text{Entropia}(t) = - \sum_{i=0}^{c-1} p(i|t) \cdot \log_2 p(i|t)$$

(con la convenzione $0 \cdot \log_2(0) = 0$). Caso a 2 classi: distribuzioni $(0, 1)$ o $(1, 0)$: **purezza massima**, Entropia = 0 (valore minimo). Distribuzione $(0.5, 0.5)$: **massima confusione**, Entropia = 1 (valore massimo).

Gini index: $\text{Gini}(t) = 1 - \sum_i p(i|t)^2$ (misura tipicamente usata da CART).

Errore di classificazione: terza misura alternativa.

Calcolo del guadagno (gain) di uno split

$$\text{guadagno} = I(\text{parent}) - \sum_{j=1}^k \frac{N(v_j)}{N} \cdot I(v_j)$$

dove $I(\text{parent})$ = impurità del nodo genitore, k = numero di nodi figli, N = numero di record nel nodo genitore, $N(v_j)$ = numero di record del nodo figlio j -mo, $I(v_j)$ = impurità del nodo figlio j -mo. Si sceglie lo split che **massimizza il guadagno**.

Information gain: quando la misura di impurità è l'entropia:

$$\text{gain} = \text{entropia}(\text{parent}) - \sum_{j=1}^k \frac{N(v_j)}{N} \cdot \text{entropia}(v_j).$$

Attenzione — limite dell'information gain (e di Gini): queste misure tendono a **favorire attributi con molti valori diversi**. Caso estremo: un identificatore univoco (es. numero di matricola) annulla completamente l'entropia dei figli, dando un information gain massimo — ma **non è affatto un attributo significativo**, anzi porta a overfitting totale. Una possibile soluzione è restringersi a **split binari**.

Domanda d'esame. Perché un identificatore univoco (es. matricola) non va mai scelto come attributo di split, nonostante massimizzi l'information gain? — Perché produce tanti nodi figli quante sono le istanze, ciascuno con una sola istanza e quindi entropia zero (purezza massima) — l'information gain risulta massimo. Ma un simile split non generalizza affatto: il “modello” ottenuto equivale a un dizionario che memorizza ogni singola istanza (overfitting estremo), incapace di classificare correttamente istanze nuove mai viste. Questo mostra il limite intrinseco di entropia/Gini, che favoriscono attributi con molti valori distinti indipendentemente dalla loro reale capacità discriminante.

Criteri di arresto (cenni)

Il caso base naturale è quando un nodo è già puro o quando non è più possibile scindere il nodo (istanze identiche ma classi diverse, gestito con la classe di maggioranza).

8.3 Reti neurali

Ispirazione biologica

Le reti neurali (Neural Networks, NN) si ispirano al modo in cui i **neuroni biologici** agiscono e interagiscono fra loro. È importante ricordare che i **neuroni artificiali non sono modelli fedeli** dei neuroni biologici. Tra i primi modelli proposti: il **Perceptron** di Rosenblatt e la *B-machine* di Turing.

Il neurone artificiale (Perceptron)

Un **perceptron** è un elemento computazionale dotato di una piccola memoria (i pesi), in grado di calcolare una funzione di attivazione applicata a una combinazione pesata dei valori in ingresso:

$$\text{net} = \sum_{i=1}^n w_i \cdot X_i, \quad Y = f(\text{net}).$$

Funzioni di attivazione: Funzione gradino (step): la funzione originariamente usata da Rosenblatt; produce valori discreti (0 oppure 1). Non è derivabile, poco adatta a tecniche basate sul gradiente. **Funzione sigmoide:**

$$Y = f(\text{net}) = \frac{1}{1 + e^{-\alpha(\text{net}-\theta)}}$$

dove θ è la soglia (bias) e α controlla la pendenza. Il vantaggio della sigmoide è di essere **derivabile**. Al crescere di α , la sigmoide tende alla forma della funzione gradino.

Passata forward e interpretazione geometrica

Cosa codifica un perceptron? Un **test lineare**, cioè un **iperpiano** nello spazio degli input:

$$w_1x_1 + w_2x_2 + \dots = 0.$$

Ciò che cade **sopra** l'iperpiano fa attivare il neurone (output 1); ciò che cade **sotto** non lo attiva (output 0). I pesi **caratterizzano il neurone** e sono **persistenti** (a differenza dei pesi “usa e getta” della votazione pesata nel rote learning).

Caratteristiche del perceptron

Adatto a compiti di tipo numerico; risolve solo problemi **linearmente separabili**; la conoscenza è data dai pesi, persistenti; apprendimento da esempi, supervisionato; “Imparare” = individuare la posizione corretta dell'iperpiano.

Apprendimento del perceptron

Regola di aggiornamento del peso sulla connessione dall'input j -mo:

$$w_j^{(k+1)} = w_j^{(k)} + \eta \cdot (d - o) \cdot x_j.$$

Il peso viene aggiornato sommando l'errore, moltiplicato per un fattore di scalamento η (learning rate) e per il valore della componente j -ma dell'input.

Teorema di Novikoff: se il problema è **linearmente separabile**, questo algoritmo di apprendimento **converge** alla soluzione; se il problema non è linearmente separabile, l'algoritmo **non converge**.

Passata backward: dopo la passata forward il perceptron produce un output o ; l'errore $(d - o)$ è l'informazione usata per modificare i pesi. **Epoca di apprendimento:** l'elaborazione di **tutte** le istanze del learning set costituisce un'epoca.

Limiti del singolo perceptron: lo XOR

A_1	A_2	XOR
0	0	0
1	0	1
0	1	1
1	1	0

Interpretando le coppie (A_1, A_2) come coordinate di punti nel piano: i punti positivi e negativi **non sono linearmente separabili** da nessuna retta. Un singolo perceptron non può quindi apprendere lo XOR. **Intuizione della soluzione:** se se ne avessero **due** disponibili, con la capacità di combinare le loro risposte, il problema si risolverebbe. Questa intuizione porta all'introduzione delle reti **multi-strato**.

Reti neurali: definizione generale e topologia

Una **rete neurale** è un **approssimatore universale di funzioni**, di natura distribuita, costituita da un insieme di neuroni collegati secondo una **topologia** (**a strati**, o **a vicinato**).

Multi-Layer Perceptron (MLP)

Il **MLP** è un modello di rete neurale con topologia **a strati**, di tipo **feed-forward**: **Livello di input**: neuroni con funzione **identità**. **Livello/i hidden**: perceptron veri e propri, tipicamente con **sigmoide**. **Livello di output**: combina i risultati hidden. Tipicamente la rete è **fully connected**.

Codifica delle classi nell'output di un MLP

1 classe o 2 classi: basta 1 solo neurone di output. ≥ 3 **classi**: *codifica binaria compatta* ($\lceil \log_2(\text{classi}) \rceil$ neuroni) oppure *codifica one-hot* (un neurone per classe) — **l'alternativa più usata in pratica**.

Domanda d'esame. Perché per un problema a più classi si preferisce spesso un neurone di output per classe (one-hot) invece della codifica binaria compatta? — Con la codifica compatta, un piccolo errore in un solo neurone può far scivolare l'output verso il codice di una classe completamente diversa. Con la codifica one-hot, ogni neurone rappresenta direttamente e in modo indipendente il grado di confidenza verso una specifica classe: è più robusta a piccoli errori, più facile da interpretare e da addestrare con la delta rule/backpropagation standard.

Cosa imparano i diversi hidden layer

Con più livelli hidden: il **primo layer** traccia dei confini (separazioni lineari elementari); il **secondo layer** combina questi confini per costruire delle **forme**; il **terzo layer** combina le forme per creare forme arbitrariamente complesse.

Potere espressivo dell'MLP

Gli **MLP a 3 livelli** con neuroni sigmoide sono **approssimatori universali di funzioni**, a patto di poter utilizzare un numero di neuroni hidden sufficientemente grande. La conoscenza appresa è memorizzata nella **matrice dei pesi**.

Apprendimento dell'MLP: backpropagation

Per ogni istanza si effettuano due passate: (1) **Passata forward**: l'istanza viene elaborata strato per strato. (2) **Passata backward**: l'errore commesso viene usato per modificare i pesi, procedendo a ritroso.

Discesa del gradiente

L'errore complessivo E dipende dalla matrice dei pesi W . Imparare significa cercare la configurazione di pesi che minimizza E :

$$\Delta w_{ji} = -\eta \cdot \frac{\partial E(w)}{\partial w_{ji}}.$$

Il gradiente $\nabla f = (\partial f / \partial x_1, \dots, \partial f / \partial x_n)$ indica la direzione di massima crescita; usato con segno negativo, permette di muoversi verso un minimo.

Errore globale dell'MLP

$$E = \frac{1}{2} \sum_{i=1}^p (t_i - y_i)^2,$$

dove p = numero di neuroni di output, t_i = target, y_i = output effettivo.

Il problema della distribuzione del credito (credit assignment)

L'errore è calcolabile direttamente solo per i neuroni del **livello di output**. Come si distribuisce il "biasimo" per l'errore fra i pesi degli strati hidden? Questo è il problema centrale risolto dalla **backpropagation**.

Delta rule per i neuroni di output:

$$\Delta w_{ji} = \eta \cdot \delta_j \cdot x_{ji}, \quad \delta_j = y_j \cdot (1 - y_j) \cdot (t_j - y_j).$$

Delta rule per i neuroni hidden:

$$\Delta w_{ki} = \eta \cdot \delta_k \cdot x_{ki}, \quad \delta_k = y_k \cdot (1 - y_k) \cdot \sum_{j \in I_k} \delta_j \cdot w_{kj},$$

dove I_k è l'insieme dei neuroni del livello successivo a cui k è connesso: il δ di un neurone hidden si calcola "propagando all'indietro" i δ dei neuroni a cui è connesso nel livello successivo, pesati per la forza delle rispettive connessioni.

Oltre la classificazione: la regressione

Una rete neurale può risolvere sia problemi di **classificazione** sia problemi di **regressione**: costruire un'approssimazione di una funzione continua $y = f(x)$ la cui forma analitica non è nota. Il target è un valore continuo anziché una classe discreta.

Calcolo dell'errore in regressione: si usa l'**Errore Quadratico Medio (MSE)**:

$$MSE(y) = \frac{\sum_{i=1}^n (y_o - y_d)^2}{n},$$

somma dei quadrati degli errori divisa per il numero di istanze.

Riepilogo e punti chiave — Modulo 8

- **Classificazione:** le classi non sono naturali; si apprende un modello dal *training set* (induzione) e lo si valuta sul *test set* (deduzione). Attenzione a dataset sbilanciati: un'accuratezza alta non implica un buon modello.
- **Alberi di decisione:** costruiti ricorsivamente con l'**algoritmo di Hunt**; scelta dell'attributo **greedy** basata su misure di impurità (**entropia**, **Gini**) combinate nel **guadagno/information gain**. Bias verso attributi con molti valori.
- **Perceptron:** calcola $net = \sum w_i X_i$, applica **sigmoide**; codifica un **iperpiano**; risolve solo problemi **linearmente separabili** (limite: **XOR**). Converge solo se linearmente separabile (Novikoff).
- **MLP:** feed-forward a strati, con almeno 3 livelli e sigmoide è **approssimatore universale**. Apprendimento per **backpropagation**: discesa del gradiente, *delta rule* per output e hidden.
- Le reti neurali risolvono anche problemi di **regressione**, valutati con l'**MSE**.